

FRONT-END VÀ BACK-END



TÓM TẮT

Phần B là phần dài nhất và là chỗ lĩnh vực này thật sự sống. Nó chia đôi theo đúng ranh giới mà mọi hệ thống SLAM tự chia: front-end lo việc *tìm ra phép đo* từ pixel, back-end lo việc *ghép phép đo thành trạng thái*. Sáu chương đi theo thứ tự một khung hình được xử lý: trích và khớp đặc trưng (chương 7), chọn dạng phân dư (chương 8), chọn kiến trúc ước lượng (chương 9), khởi động hệ thống từ số không (chương 10), phát hiện và đóng vòng khi quay lại chỗ cũ (chương 11), và tìm lại mình khi lạc (chương 12). Nếu chỉ nhớ một điều: *front-end quyết định hệ thống hỏng theo kiểu nào, back-end quyết định nó chính xác tới đâu* — và trong triển khai thật, kiểu hỏng quan trọng hơn độ chính xác.

Ranh giới front-end / back-end không phải một quy ước tùy tiện mà là một đường phân chia có cơ sở. Front-end giải bài toán **data association** — pixel nào ở khung này tương ứng với pixel nào ở khung kia, hoặc với landmark nào trong bản đồ. Đó là một bài toán *tổ hợp*, có lời giải đúng hoặc sai, và sai thì sai thảm họa. Back-end giải bài toán **ước lượng** — cho các tương ứng, tìm trạng thái tốt nhất. Đó là bài toán *liên tục*, có lời giải chính xác hơn hoặc kém chính xác hơn, và sai thì sai từ từ.

Hai loại sai ấy đòi hai loại công cụ, và đó là lý do sâu xa của sự phân chia. Chống lỗi tổ hợp cần dư thừa và kiểm định: RANSAC, kiểm tra hình học, ngưỡng chấp nhận bảo thủ. Chống lỗi liên tục cần mô hình đúng và trọng số đúng. Dùng công cụ của bên này cho bên kia thì hỏng — và một trong những lỗi thiết kế phổ biến là hy vọng back-end sẽ “chịu đựng được” một front-end tồi. Nó chịu được vài phần trăm outlier; nó không chịu được một loop closure sai, và B/11 nói vì sao.

Sáu chương có mức đầu tư rất khác nhau. Chương 7 và 8 là hai chương mà mọi hệ thống đều phải quyết định, và quyết định của chúng định hình mọi thứ còn lại — đó là lý do chúng đứng đầu. Chương 9 là chương hợp nhất: nó cho thấy bộ lọc, cửa sổ trượt, smoothing gia tăng và full batch đều là cùng một bài toán MAP của A/04 với các lựa chọn khác nhau về marginalize cái gì và tuyến tính hoá lại bao nhiêu lần; hiểu điều đó thì việc chuyển qua lại giữa các thể giới trở nên tự nhiên. Chương 10 và 11 là hai chương về hai khoảnh khắc đặc biệt của một hệ thống — lúc bắt đầu và lúc quay lại — và cả hai đều là chỗ A/05 trả công. Chương 12 là chương về khoảnh khắc thứ ba, lúc lạc đường, và nó là chương gần với sản phẩm nhất trong phần này.

Thứ tự đọc đề xuất là tuần tự. Một ngoại lệ: nếu bạn đã đọc ../IMU_Fusion/, chương 9 ở đây chồng lấn với chương 8 và 9 ở đó; phần chồng lấn được viết lại từ góc nhìn thị giác, và chỗ nào cây kia nói sâu hơn thì ở đây trở sang.

Bảng 1. Bản đồ Phần B: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
7	07-dac-trung-va-dat	Detector và descriptor cổ điển lần học được; KLT và optical flow; phân bố đặc trưng; RANSAC và họ của nó; tinh chỉnh subpixel	• • •
8	08-mo-hinh-phan-du	Indirect, direct, semi-direct; tham số hoá landmark; structureless; phần dư độ sâu, đường thẳng, mặt phẳng; prior học được	• • •
9	09-paradigm-uoc-luo	Bộ lọc (EKF, MSCKF, InEKF); cửa sổ trượt; fixed-lag; iSAM2; full batch; pose graph; dạng căn bậc hai; certifiable	• • •
10	10-khoi-tao	Khởi tạo mono hai khung và model selection; khởi tạo từ stereo/depth; khởi tạo VI; từ prior học được; phát hiện thất bại và khởi tạo lại	• •
11	11-loop-closure-va-	BoW và place recognition học được; kiểm định hình học; $Sim(3)$; pose graph; back-end bền (switchable, DCS, GNC, RRR); perceptual aliasing	• • •
12	12-relocalization	BoW + PnP; localization phân cấp; scene coordinate regression; vì sao APR không tốt; kidnapped robot; kiểm định chống reloc sai	• •

Chương 7, 8, 9 là bộ ba xương sống và nên đọc liền nhau: chúng lần lượt trả lời “phép đo đến từ đâu”, “phép đo có dạng gì”, và “ghép phép đo thế nào”. Chương 11 cũng là xương sống nhưng vì lý do khác — nó là chương duy nhất mà một lỗi đơn lẻ có thể phá huỷ toàn bộ bản đồ, nên phần về back-end bền ở đó đáng đọc kỹ hơn về ngoài. Chương 10 và 12 có thể đọc lướt lần đầu rồi quay lại khi bạn thật sự phải xây dựng hai mô-đun đó; cả hai đều là nơi A/05 trả công và là nơi nguyên tắc “biết từ chối quan trọng hơn luôn trả lời” được áp dụng cụ thể nhất.

Đặc trưng và data association (Features and Data Association)

PHỤ THUỘC A/01-hình-học-da-thi (kiểm định hình học là bước cuối của mọi pipeline khớp điểm); A/02-mô-hình-camera (gỡ méo trước khi dùng hình học); A/06-bat-dinh-va-lan-truyen-sai-so (σ của phép đo pixel chính là thứ chương này quyết định, và nó là trọng số cho mọi thứ ở back-end).

TÓM TẮT

Chương này về bài toán mà phần còn lại của cây coi như đã giải xong: *pixel nào ở khung này là cùng một điểm vật lý với pixel nào ở khung kia?* Đây là bài toán *tổ hợp* — có lời giải đúng hoặc sai — khác hẳn bài toán ước lượng liên tục ở back-end, và khác biệt ấy quyết định mọi công cụ dùng để giải nó. Nội dung có bốn phần. Một: cách tìm điểm đáng theo dõi, từ tiêu chí Harris/Shi-Tomasi tới detector học được, và tiêu chí thật để chọn giữa chúng. Hai: cách khớp chúng — mô tả rồi so khớp, hay theo dõi trực tiếp bằng optical flow — và vì sao hai cách phù hợp với hai chế độ hoạt động khác nhau. Ba: hai thứ bị coi nhẹ nhất mà ảnh hưởng lớn nhất, là *phân bố không gian* của đặc trưng và *độ chính xác subpixel*; cái thứ nhất quyết định điều kiện số của Hessian, cái thứ hai quyết định trực tiếp σ tức trọng số của mọi phần dư. Bốn: loại bỏ outlier bằng RANSAC và họ của nó, với phân tích vì sao cỡ mẫu tối thiểu quan trọng hơn người ta nghĩ. Nếu chỉ nhớ một điều: *front-end quyết định hệ thống hỏng theo kiểu nào, back-end chỉ quyết định nó chính xác tới đâu* — và trong sản phẩm, kiểu hỏng quan trọng hơn.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đặc trưng (feature)** — một vùng ảnh nhỏ có tính chất đủ đặc biệt để nhận ra lại ở ảnh khác. Gồm hai phần tách rời: *detector* tìm ra nó ở đâu, *descriptor* mô tả nó trông thế nào.
- **bài toán cửa sổ (aperture problem)** — nhìn qua một lỗ nhỏ vào một cạnh thẳng, ta chỉ biết cạnh dịch chuyển vuông góc với nó bao nhiêu, không biết nó trượt dọc bao nhiêu. Đây là lý do góc theo dõi được còn cạnh thì không, và nó là gốc của mọi tiêu chí detector.
- **ma trận cấu trúc (structure tensor)** — ma trận $M = \sum_w \nabla I \nabla I^T$ lấy trên một cửa sổ. Hai giá trị riêng của nó nói vùng đó là vùng phẳng (cả hai nhỏ), cạnh (một lớn một nhỏ), hay góc (cả hai lớn).
- **optical flow** — trường vectơ mô tả chuyển động biểu kiến của các mẫu cường độ giữa hai ảnh. KLT theo dõi một mảng nhỏ bằng cách giải trực tiếp cho độ dịch làm cường độ khớp nhất.
- **ratio test** — quy tắc của Lowe: chỉ chấp nhận khớp nếu khoảng cách tới ứng viên gần nhất nhỏ hơn đáng kể so với ứng viên gần thứ hai. Nó loại các đặc trưng lặp lại, thứ mà khoảng cách tuyệt đối không loại được.
- **RANSAC** — lập: lấy mẫu tối thiểu, khớp mô hình, đếm inlier, giữ mô hình tốt nhất. Nó tìm *tập nhất quán lớn nhất*, không phải nghiệm chính xác nhất — nên nó luôn phải theo sau bởi một bước tinh chỉnh.
- **tinh chỉnh subpixel** — ước lượng vị trí đặc trưng chính xác hơn một pixel, thường bằng cách khớp parabol vào hàm điểm số quanh cực đại. Nó quyết định trực tiếp σ của phép đo.

CÂU HỎI MỞ ĐẦU

- Q1. Thuật toán năm điểm cần năm mẫu, tám điểm cần tám. Với 50% outlier, chênh lệch số vòng lặp RANSAC giữa hai lựa chọn ấy là bao nhiêu — và vì sao con số đó lớn hơn nhiều so với trực giác “chỉ hơn ba điểm”?
- Q2. Hai hệ thống trích cùng số lượng đặc trưng với cùng chất lượng khớp. Một hệ chính xác hơn hẳn. Nếu một khác biệt về front-end có thể giải thích điều đó mà *không* liên quan gì tới detector hay descriptor.
- Q3. Tinh chỉnh subpixel cải thiện σ từ 0,5 px xuống 0,15 px. Vì sao con số đó ảnh hưởng tới *mọi* thứ ở back-end, và nó ảnh hưởng thế nào tới hiệp phương sai đầu ra?
- Q4. SuperPoint và LightGlue cho nhiều khớp đúng hơn ORB một cách rõ rệt trong điều kiện khó. Vì sao nhiều hệ sản phẩm vẫn dùng ORB, và câu trả lời “vì GPU đắt” là chưa đủ?
- Q5. Hành lang văn phòng có hàng chục cánh cửa giống hệt nhau. Front-end khớp điểm giữa hai cánh cửa khác nhau một cách hoàn hảo — khoảng cách descriptor rất nhỏ. Vì sao ratio test không cứu được ở đây, và thứ gì mới cứu được?

1.1 Đặt vấn đề

Phần A xây dựng toàn bộ bộ máy ước lượng trên một giả thiết mà nó không bao giờ kiểm: rằng ta biết pixel nào tương ứng với pixel nào. A/01 mục 7 nêu nó là giả thiết ngầm thứ tư; chương này là chỗ trả nợ.

Bài toán gốc: cho hai ảnh của cùng một cảnh chụp từ hai chỗ khác nhau, tìm các cặp pixel là ảnh của cùng một điểm vật lý. Nghe đơn giản, và nó khó vì bốn lý do chồng lên nhau. *Thứ nhất, không gian tìm kiếm lớn*: một ảnh 640×480 có ba trăm nghìn pixel, nên ghép cặp vét cạn là chín mươi tỉ phép so sánh. *Thứ hai, phần lớn pixel không phân biệt được*: một pixel giữa bức tường trắng giống hết hàng nghìn pixel khác. *Thứ ba, vẻ ngoài thay đổi*: cùng một điểm nhìn từ góc khác, dưới ánh sáng khác, ở tỉ lệ khác, trông không giống nhau. *Thứ tư, và đây là lý do nghiêm trọng nhất, có những thứ trông giống nhau mà không phải cùng một điểm* — cửa sổ trên một toà nhà, viên gạch trên sàn, cánh cửa trong hành lang.

Ba lý do đầu là lý do *khó*; lý do thứ tư là lý do *nguy hiểm*, và nó khác về bản chất. Với ba lý do đầu, thất bại nghĩa là không tìm được khớp, và hệ thống biết mình không tìm được. Với lý do thứ tư, thất bại nghĩa là tìm được một khớp *sai* trông rất thuyết phục, và hệ thống không biết. Đây là **perceptual aliasing**, và nó là kẻ thù số một của SLAM trong môi trường nhân tạo — chính là môi trường mà robot sản phẩm hoạt động.

Ai gặp bài toán này? Mọi ứng dụng thị giác hình học, và giải pháp được phát triển chủ yếu trong cộng đồng thị giác máy tính chứ không phải robot học. Lịch sử có ba giai đoạn đáng biết vì mỗi giai đoạn để lại một bài học. Giai đoạn một, trước SIFT: theo dõi mảng ảnh trực tiếp (KLT), hoạt động tốt khi chuyển động nhỏ và hỏng hoàn toàn khi lớn. Giai đoạn hai, SIFT [1] và con cháu: mô tả bất biến với tỉ lệ và xoay, cho phép khớp giữa hai ảnh bất kỳ; đây là bước nhảy làm SfM quy mô lớn khả thi. Giai đoạn ba, từ khoảng 2018: detector và matcher học được, cải thiện rõ rệt ở điều kiện khó.

Cách tiếp cận hiển nhiên — và cách gần như ai cũng nghĩ tới đầu tiên — là làm descriptor *tốt hơn* cho tới khi không còn khớp sai. Cách đó không bao giờ thành công, và lý do đáng nói rõ vì nó định hình toàn bộ kiến trúc: *không có descriptor nào phân biệt được hai cánh cửa thật sự giống hệt nhau*, vì thông tin phân biệt không nằm trong vùng ảnh cục bộ. Nó nằm ở *ngữ cảnh* — cánh cửa này ở đâu so với các thứ khác — và ngữ cảnh là thông tin hình học, không phải thông tin về ngoại.

Đó là lý do mọi pipeline khớp điểm nghiêm túc đều có *hai tầng*: tầng vẻ ngoài đề xuất ứng viên, tầng hình học kiểm định chúng. Tầng thứ hai không phải một bước làm sạch tùy chọn mà là một phần thiết yếu, và nó là chỗ A/01 quay lại phục vụ chương này.

TRỰC GIÁC

Hình dung bạn phải nối hai bức ảnh chụp cùng một căn phòng từ hai chỗ đứng, bằng cách vẽ các đường nối những điểm tương ứng.

Bạn không bắt đầu bằng cách nhìn từng pixel. Bạn tìm những chỗ *đáng nhớ*: góc bàn, mép tranh, vết ố trên tường. Những chỗ ấy có hai tính chất — chúng khác biệt với xung quanh, và chúng vẫn nhận ra được từ góc nhìn khác. Đó là công việc của detector.

Rồi với mỗi chỗ đáng nhớ, bạn ghi lại nó *trông thế nào* theo một cách không phụ thuộc vào việc bạn đứng đâu — “một góc vuông, sáng ở trên, tối ở dưới”. Đó là descriptor.

Rồi bạn ghép: với mỗi chỗ ở ảnh một, tìm chỗ ở ảnh hai có mô tả giống nhất. Và đây là chỗ bạn sẽ mắc lỗi, vì trong phòng có bốn cái ghế giống hệt nhau.

Cách bạn — một con người — phát hiện lỗi ấy rất đáng chú ý: bạn *không* nhìn kỹ hơn vào cái ghế. Bạn lùi lại và nhìn *tất cả* các đường nối cùng lúc. Nếu hai mươi đường nối đều đi theo một quy luật chung và một đường đi ngược hẳn, bạn biết ngay đường đó sai — không phải vì nó trông sai, mà vì nó *không phù hợp với số đông*.

Đó chính xác là RANSAC, và đó là lý do tăng hình học không thể thiếu. Thông tin để phát hiện khớp sai không nằm trong khớp đó; nó nằm trong quan hệ giữa nó với tất cả các khớp khác.

1.2 Tìm điểm đáng theo dõi

A. Vì sao góc chứ không phải cạnh

Tiêu chí chọn đặc trưng có một cơ sở toán học gọn và đáng tự dẫn một lần.

Xét một cửa sổ nhỏ và hỏi: dịch nó đi một đoạn $\Delta \mathbf{u}$ thì cường độ đổi bao nhiêu? Tổng bình phương chênh lệch, khai triển bậc nhất:

$$E(\Delta \mathbf{u}) = \sum_w [I(\mathbf{u} + \Delta \mathbf{u}) - I(\mathbf{u})]^2 \approx \Delta \mathbf{u}^\top M \Delta \mathbf{u}, \quad M = \sum_w \nabla I \nabla I^\top.$$

Ma trận M cỡ 2×2 là **ma trận cấu trúc**. Hai giá trị riêng của nó phân loại vùng ảnh: cả hai nhỏ nghĩa là vùng phẳng — dịch đi đâu cũng không đổi, không theo dõi được; một lớn một nhỏ nghĩa là cạnh — dịch vuông góc thì đổi, dịch dọc thì không, đây chính là **bài toán cửa sổ**; cả hai lớn nghĩa là góc — dịch theo hướng nào cũng đổi, theo dõi được theo cả hai chiều.

Shi và Tomasi [2] đề nghị dùng trực tiếp $\min(\lambda_1, \lambda_2)$ làm điểm số; Harris dùng một tổ hợp của định thức và vết để tránh tính giá trị riêng. Hai tiêu chí rất gần nhau trong thực hành.

Điều đáng nhận ra, và nó nối chương này với A/04: M chính là *ma trận thông tin của bài toán ước lượng vị trí đặc trưng*. Bài toán “đặc trưng này dịch đi đâu” là một bài toán MAP nhỏ, và M là Hessian của nó. Cạnh không theo dõi được vì M suy biến theo hướng dọc cạnh — đó đúng là hiện tượng ở A/05, gặp ở quy mô một cửa sổ 7×7 pixel. Cùng một nguyên lý xuất hiện ở hai thang bậc khác nhau, và nhận ra điều đó là dấu hiệu đã đọc đúng cả hai chương.

B. Từ cổ điển tới học được

FAST [3] kiểm một vòng tròn 16 pixel quanh ứng viên và hỏi có đủ số pixel liên tiếp sáng hơn hoặc tối hơn tâm không — nhanh tới mức gần như miễn phí, không bắt biến gì cả. **ORB** [4] thêm hướng vào FAST và ghép với descriptor nhị phân BRIEF có xoay; kết quả là một cặp detector-descriptor cực nhanh, khớp bằng khoảng cách Hamming. **SIFT** [1] tìm cực trị trong không gian tỉ lệ và mô tả bằng histogram gradient; chậm hơn nhiều bậc nhưng bất biến tỉ lệ và xoay thật sự.

SuperPoint [5] học cả detector lẫn descriptor bằng tự giám sát. **SuperGlue** [6] thay bước khớp bằng một mạng chú ý nhìn *toàn bộ* hai tập điểm cùng lúc, nên nó khai thác được ngữ cảnh — đây là bước tiến khái niệm quan trọng, vì nó là lần đầu thông tin ngữ cảnh được đưa vào tầng về ngoài thay vì chỉ ở tầng hình học. **LightGlue** [7] làm cùng việc rẻ hơn nhiều với cơ chế thoát sớm. **LoFTR** [8] bỏ hẳn detector và khớp trực tiếp ở mức dày — hoạt động tốt ở vùng ít vân, nơi mọi detector đều thất bại.

CẠM BÃY

Tôi **không** đặt bảng so sánh số liệu giữa các detector ở đây, và lý do thuộc về phương pháp chứ không phải sự lười.

Các con số công bố (số khớp đúng, độ lặp lại, tỉ lệ thành công) phụ thuộc cực mạnh vào bộ dữ liệu, vào ngưỡng, vào cách đếm, và vào việc ai chạy baseline. Hai bài báo báo cáo hai con số cho cùng một cặp phương pháp trên cùng bộ dữ liệu thường *không so sánh được với nhau*, và việc đặt chúng cạnh nhau trong một bảng tạo ra ảo giác so sánh — đúng thứ mà `research-rules.md` mục 6.2 cảnh báo.

Điều *có thể* nói một cách trung thực là các đánh đổi về cơ chế, và đó là nội dung Bảng 1.1. Điều *không thể* nói là “X tốt hơn Y bao nhiêu phần trăm” mà không kèm điều kiện đo đầy đủ.

Cách đúng để chọn cho dự án của bạn không phải đọc bảng của người khác: đó là **đo trên dữ liệu của chính bạn**, trong chính điều kiện triển khai, với chính phần cứng mục tiêu. Chương D/21 lập luận rằng đây không phải lời khuyên cầu toàn mà là yêu cầu bắt buộc, vì khoảng cách giữa EuRoC và một nhà kho thật lớn hơn khoảng cách giữa hai detector bất kỳ.

Bảng 1.1. Các họ front-end theo cơ chế, không theo con số. Cột cuối là tiêu chí chọn thực dụng.

Họ	Cơ chế	Hỏng khi	Chọn khi
FAST + BRIEF/ORB	Kiểm vòng tròn cường độ; descriptor nhị phân	Đổi tỉ lệ mạnh; đổi ánh sáng lớn; vùng ít vân	CPU hạn chế; tốc độ khung cao; chuyển động vừa phải
SIFT và họ	Cực trị không gian tỉ lệ; histogram gradient	Vùng ít vân; đổi về ngoài lớn (mùa, ngày đêm)	Cần bất biến tỉ lệ thật; SfM ngoại tuyến
KLT / optical flow	Theo dõi mảng trực tiếp bằng gradient	Chuyển động lớn; đổi ánh sáng; che khuất	Khung liên tiếp gần nhau; muốn rẻ và chính xác subpixel
Học được (SuperPoint+Li)	Detector và matcher học từ dữ liệu; matcher dùng ngữ cảnh	Ngoài phân bố huấn luyện; cần GPU; khó dự đoán kiểu hỏng	Có GPU; điều kiện khó; cần độ bền với đổi về ngoài
Detector-free (LoFTR, MAST3R)	Khớp dày trực tiếp, không qua điểm	Rất đắt; khó đưa vào vòng lặp thời gian thực	Vùng ít vân; localization ngoại tuyến

Về câu hỏi mở đầu thứ tư — vì sao sản phẩm vẫn dùng ORB — câu trả lời “vì GPU đắt” chỉ là một phần. Ba lý do còn lại đáng giá hơn. *Tính dự đoán được*: ORB hỏng theo những cách đã biết rõ và lặp lại được, còn mạng học được hỏng theo những cách phụ thuộc vào việc dữ liệu hiện tại có giống dữ liệu huấn luyện không — và không ai đo được điều đó tại hiện trường. *Tính tất định*: cùng đầu vào cho cùng đầu ra, bit-exact, điều kiện bắt buộc để debug một lỗi từ hiện trường

(D/20). *Chi phí cập nhật*: đối một mô hình học được đòi hỏi tái kiểm định toàn bộ, còn đối một ngưỡng thì không. Ba lý do đó thuộc về D/21 và chúng thường quyết định hơn độ chính xác.

1.3 Khớp: mô tả rời so, hay theo dõi trực tiếp

A. Hai chế độ hoạt động

Có hai cách hoàn toàn khác nhau để tìm tương ứng, và chúng phù hợp với hai tình huống khác nhau — nhằm chúng là một lỗi thiết kế phổ biến.

Khớp bằng descriptor không giả định gì về vị trí: trích đặc trưng ở cả hai ảnh, mô tả chúng, rồi tìm ứng viên gần nhất trong không gian descriptor. Dùng được cho hai ảnh *bất kỳ*, kể cả cách nhau hàng giờ hoặc chụp từ hai thiết bị khác nhau. Chi phí: trích đặc trưng ở cả hai ảnh, và tìm kiếm trong không gian nhiều chiều.

Theo dõi bằng optical flow giả định chuyển động nhỏ: lấy vị trí đặc trưng ở khung trước làm khởi đầu, rồi giải cho độ dịch làm cường độ khớp nhất. KLT làm điều này bằng Gauss-Newton trên một mảng nhỏ, với tháp ảnh để chịu được chuyển động lớn hơn [9]. Rẻ hơn nhiều vì không phải trích và mô tả lại; chính xác subpixel tốt hơn vì nó tối ưu trực tiếp trên cường độ. Hồng khi chuyển động quá lớn hoặc ánh sáng đổi.

Quy tắc chọn gọn: *theo dõi giữa các khung liên tiếp, khớp descriptor khi cần nối hai khung xa nhau*. Đây là lý do VINS-Mono dùng KLT cho tracking [10] còn ORB-SLAM dùng descriptor — hai lựa chọn kiến trúc khác nhau phù hợp với hai ưu tiên khác nhau, không phải một cái đúng một cái sai. Nhưng mọi hệ thống đều cần descriptor ở đâu đó, vì loop closure (B/11) và relocalization (B/12) bản chất là khớp giữa hai khung rất xa nhau về thời gian.

Baker và Matthews [11] chỉ ra một biến thể quan trọng: *inverse compositional*, trong đó Hessian được tính *một lần* trên mảng tham chiếu thay vì tính lại mỗi vòng lặp. Cùng kết quả, chi phí thấp hơn nhiều bậc. Đây là loại tối ưu hoá đáng biết vì nó không đánh đổi gì cả.

B. Ratio test và giới hạn của nó

Sau khi tìm ứng viên gần nhất, câu hỏi là có chấp nhận không. Dùng ngưỡng tuyệt đối trên khoảng cách descriptor hoạt động kém, vì thang khoảng cách khác nhau giữa các loại đặc trưng và giữa các vùng ảnh.

Ratio test của Lowe so khoảng cách tới ứng viên gần nhất với ứng viên gần *thứ hai*, và chỉ chấp nhận nếu tỉ số đủ nhỏ. Trực giác: nếu đặc trưng thật sự đặc biệt thì ứng viên đúng phải gần hơn hẳn mọi ứng viên khác; nếu có hai ứng viên gần ngang nhau thì ta không có cơ sở chọn. Đây là một phép thử *tương đối*, nên nó tự thích ứng với thang khoảng cách địa phương — và đó là lý do nó bền hơn ngưỡng tuyệt đối rất nhiều.

Nhưng nó có một giới hạn quan trọng, và đó là câu trả lời cho câu hỏi mở đầu thứ năm. Ratio test loại được trường hợp *có nhiều ứng viên giống nhau trong tầm nhìn*. Nó không loại được trường hợp cánh cửa thứ hai không nằm trong tập ứng viên — chẳng hạn vì nó ở ngoài khung hình hiện tại, hoặc vì ta chỉ tìm kiếm trong một vùng nhỏ. Lúc đó ứng viên sai là ứng viên duy nhất, tỉ số rất tốt, và khớp được chấp nhận với độ tin cậy cao.

Thứ cứu được chỉ có thể là hình học: kiểm định rằng khớp này nhất quán với đa số các khớp khác (mục 5), hoặc nhất quán với một tiên nghiệm về chuyển động. Đây lại là bài học ở khối *Trực giác*: thông tin để phát hiện khớp sai không nằm trong khớp đó.

1.4 Hai thứ bị coi nhẹ nhất

A. Phân bố không gian

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một trong những điểm có tỉ lệ giá trị trên độ phức tạp cao nhất trong cả cây.

Cách cài đặt ngây thơ: tính điểm số cho mọi ứng viên, sắp xếp, lấy N cái tốt nhất. Kết quả gần như luôn là các đặc trưng *tụ lại* ở vùng có vân mạnh — một tấm poster trên tường, một tán lá — và cả vùng còn lại của ảnh không có đặc trưng nào.

Vì sao điều đó tệ? Quay lại Jacobian ở A/03 công thức (??): đóng góp của một quan sát vào ma trận thông tin phụ thuộc vào *vị trí của nó trong ảnh* qua các số hạng X/Z và Y/Z . Các đặc trưng tụ ở một chỗ đóng góp các hàng Jacobian gần như *tuyến tính phụ thuộc* nhau, nên chúng thêm rất ít thông tin mới. Hệ quả là Λ có số điều kiện lớn, và bài toán gần suy biến theo một hướng — đúng loại suy biến số học ở A/05 mục 2C. Một trăm đặc trưng trải đều cho nhiều thông tin hơn năm trăm đặc trưng tụ một chỗ.

Có một lý do thứ hai, độc lập và cũng quan trọng: *độ bền*. Nếu mọi đặc trưng ở trên một vật, và vật đó bị che hoặc bắt đầu di chuyển, hệ thống mất tất cả cùng lúc. Phân bố đều là một dạng đa dạng hoá rủi ro.

Ba cách cài đặt, theo thứ tự tinh vi. *Lưới*: chia ảnh thành ô, giữ tối đa k đặc trưng tốt nhất mỗi ô. Rẻ, hiệu quả, và đủ cho hầu hết trường hợp. *Quadtree*: chia thích ứng, tinh hơn ở vùng đông — ORB-SLAM dùng cách này [12]. *ANMS* (adaptive non-maximal suppression): giữ đặc trưng theo bán kính triệt tiêu giảm dần, cho phân bố đều nhất nhưng đắt nhất.

B. Độ chính xác subpixel

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và nó là điểm nối trực tiếp nhất giữa front-end và A/06.

Vị trí đặc trưng ở độ phân giải pixel nguyên cho $\sigma \approx 0,5$ px (sai số lượng tử hoá của một biến phân bố đều trên một pixel là $1/\sqrt{12} \approx 0,29$ px, nhưng trong thực tế sai số lớn hơn thế vì còn nhiễu và lượng tử hoá không hoàn hảo). Tinh chỉnh subpixel — khớp một parabol vào hàm điểm số quanh cực đại, hoặc chạy vài vòng KLT — giảm con số đó xuống mức thường được nêu là 0,1 tới 0,2 px.

CẠM BẮY

Hai con số vừa nêu cần một cảnh báo. Khoảng 0,1–0,2 px là con số **thường được nêu trong cộng đồng** cho tinh chỉnh subpixel trên ảnh có vân tốt, nhưng **tôi không dẫn được một nguồn đo có điều kiện đầy đủ cho nó**, và nó phụ thuộc mạnh vào độ tương phản, mức nhiễu cảm biến, và độ sắc nét của đặc trưng.

Theo `learning-rules.md` mục 5, một con số không có điều kiện đo là một con số vô nghĩa, nên cách đúng là: **tự đo σ của chính front-end của bạn**, trên chính dữ liệu của bạn. Cách đo rẻ nhất: chụp một cảnh tĩnh với camera cố định trên chân máy, theo dõi đặc trưng qua vài trăm khung, và tính độ lệch chuẩn của vị trí từng đặc trưng. Con số thu được là σ *thật* của bạn, đã bao gồm cả nhiễu cảm biến lẫn giới hạn thuật toán.

Con số đó đáng đo vì nó là **trọng số $\Omega = \sigma^{-2}$** cho mọi phần dư chiếu ở back-end. Đặt sai nó thì mọi hiệp phương sai đầu ra sai theo, và bạn rơi vào đúng nguồn số hai của bệnh quá tự tin ở A/06 mục 4B. Đây là một trong những phép đo rẻ nhất và bị bỏ qua nhiều nhất trong cả quy trình.

Vì sao σ quan trọng tới vậy? Ba đường. *Một*: nó vào thẳng Ω_k trong hệ chuẩn (??), nên nó quyết định trọng số tương đối giữa nhân tử thị giác và mọi nhân tử khác — IMU, odometry, GNSS. *Hai*: hiệp phương sai đầu ra tỉ lệ với σ^2 , nên giảm σ ba lần thì hiệp phương sai giảm chín lần; đó là một cải thiện lớn hơn hầu hết các thay đổi thuật toán. *Ba*: qua σ_d trong công thức (??), nó quyết định trực tiếp tầm hữu ích của stereo. Với $b = 12$ cm, $f = 400$ px, ở $z = 5$ m: $\sigma = 0,5$ px cho $\sigma_z \approx 26$ cm, còn $\sigma = 0,15$ px cho $\sigma_z \approx 8$ cm. Cùng phần cứng, ba lần khác biệt về chất lượng độ sâu.

1.5 Loại bỏ outlier

A. RANSAC và vì sao cỡ mẫu quan trọng

RANSAC [13] lặp bốn bước: lấy ngẫu nhiên s tương ứng (cỡ mẫu tối thiểu của mô hình), khớp mô hình, đếm inlier, giữ mô hình nhiều inlier nhất.

Số vòng lặp cần để có xác suất p lấy được ít nhất một mẫu toàn inlier, với tỉ lệ inlier w :

$$N = \frac{\log(1-p)}{\log(1-w^s)}. \quad (1.1)$$

Suy dẫn: xác suất một mẫu toàn inlier là w^s ; xác suất N mẫu đều hỏng là $(1-w^s)^N$; đặt bằng $1-p$ và lấy log. (Cửa a.)

Ví dụ số trả lời câu hỏi mở đầu thứ nhất. Với $w = 0,5$ và $p = 0,99$:

Thuật toán năm điểm ($s = 5$): $w^s = 0,5^5 = 0,03125$, nên $N = \log(0,01)/\log(0,96875) \approx -4,605/(-0,03175) \approx 146$ vòng.

Thuật toán tám điểm ($s = 8$): $w^s = 0,5^8 = 0,00391$, nên $N \approx -4,605/(-0,003914) \approx 1177$ vòng.

Tám lần khác biệt cho ba điểm mẫu thêm. (Cửa kiểm chứng b, tính tay.) Và với $w = 0,3$, chênh lệch thành 170 so với 6300 vòng — gần bốn mươi lần. Đây là lý do thực dụng chính để dùng thuật toán năm điểm, và nó lớn hơn nhiều so với ấn tượng “chỉ bớt ba điểm”. Quan hệ là hàm mũ theo s , không tuyến tính.

B. Cải tiến và giới hạn

PROSAC [14] lấy mẫu theo thứ tự chất lượng khớp thay vì ngẫu nhiên đều — vì khớp có điểm số tốt có xác suất là inlier cao hơn. Hội tụ nhanh hơn nhiều khi thang điểm có ý nghĩa. **MAGSAC++** [15] bỏ ngưỡng inlier cứng bằng cách tích phân trên các mức ngưỡng — giải quyết một vấn đề thực tế đáng kể, là ngưỡng inlier thường được chọn tùy tiện và kết quả nhạy với nó. **LO-RANSAC** chạy một bước tinh chỉnh cục bộ trên tập inlier mỗi khi tìm được mô hình tốt hơn, cải thiện rõ với chi phí nhỏ.

Ba giới hạn của RANSAC đáng nhớ vì chúng là nơi nó thất bại một cách im lặng.

Một: nó tìm *tập nhất quán lớn nhất*, không phải nghiệm đúng. Nếu outlier có cấu trúc — tất cả nằm trên một vật đang di chuyển — thì chúng nhất quán với nhau, và nếu vật đó chiếm đa số khung hình, RANSAC sẽ trung thành chọn vật động làm inlier. Kết quả sai nhưng nhất quán, tức im lặng. Xử lý ở D/19.

Hai: nó cần một mô hình. Trong cấu hình suy biến ở A/01 mục 6, mô hình không xác định duy nhất, nên RANSAC báo nhiều inlier cho một mô hình vô nghĩa. Đây chính là lý do khối *Cắm bẫy* ở chương đó cấm dùng số inlier làm thước đo chất lượng.

Ba: nghiệm của nó chỉ để khởi tạo. Mô hình khớp từ mẫu tối thiểu không dùng *tất cả* inlier, nên nó không tối ưu. Luôn phải theo sau bởi một bước tinh chỉnh trên toàn bộ inlier — và bước đó chính là bài toán MAP nhỏ của A/04.

C. Tiên nghiệm chuyển động: cách rẻ nhất cải thiện front-end

Một kỹ thuật đơn giản mà hiệu quả bất ngờ, và nó là một trong những chỗ mà kiến trúc hệ thống trả công.

Nếu có tiên nghiệm về chuyển động giữa hai khung — từ IMU, từ odometry bánh xe, hoặc chỉ từ mô hình vận tốc không đổi — ta dự đoán được mỗi landmark sẽ chiếu về *đâu* trong khung mới. Thay vì tìm kiếm toàn ảnh, ta tìm trong một vùng nhỏ quanh vị trí dự đoán.

Ba lợi ích cùng lúc, và cả ba đều lớn. *Rẻ hơn*: vùng tìm kiếm nhỏ hơn hàng chục lần. *Ít khớp sai hơn*: bớt ứng viên nghĩa là bớt cơ hội nhầm — và điều này tấn công trực tiếp vào perceptual aliasing, vì cánh cửa thứ hai thường nằm ngoài vùng tìm kiếm. *Tỉ lệ inlier cao hơn*: w tăng, nên theo (1.1) số vòng RANSAC giảm theo hàm mũ.

Đây là một ví dụ rõ ràng của việc hợp nhất cảm biến giúp *front-end* chứ không chỉ back-end — một điểm hay bị bỏ qua khi người ta nghĩ về IMU chỉ như một nguồn ràng buộc bổ sung trong đồ thị. Chi tiết ở C/15 và ../IMU_Fusion/.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [16]	ORB + quadtree phân bố; tìm kiếm có hướng dẫn bằng mô hình chuyển động; RANSAC PnP	Mục 4A và 5C trong một hệ thống hoàn chỉnh
VINS-Mono [10]	KLT tracking + lưới phân bố; IMU dự đoán vùng tìm kiếm	Lựa chọn kiến trúc đối lập với ORB-SLAM — xem mục 3A
COLMAP [17]	SIFT + ratio test + kiểm định hình học đa mô hình	Pipeline hai tầng đầy đủ nhất trong phần mềm mã nguồn mở
HLoc [18]	SuperPoint + SuperGlue/LightGlue	Front-end học được trong pipeline localization thật
SVO-Pro [19]	Trích đặc trưng thưa + căn chỉnh direct; tinh chỉnh subpixel bằng tối ưu cường độ	Mình hoạ mục 4B: độ chính xác subpixel đến từ tối ưu trực tiếp trên cường độ

MINI PROJECT SLAM · Đo σ thật của front-end của bạn, rồi đo ảnh hưởng của phân bố đặc trưng · 3 ngày

Mục tiêu	biến hai con số mà mọi người đoán — σ subpixel và ảnh hưởng của phân bố — thành hai con số bạn đo được trên phần cứng của mình.
Dữ liệu	camera của bạn trên chân máy cố định, chụp một cảnh tĩnh có vân đa dạng (vài trăm khung); cộng thêm một chuỗi EuRoC để có chân trị.
Các bước	<i>Phần một</i> : theo dõi đặc trưng qua toàn bộ chuỗi tĩnh, tính độ lệch chuẩn vị trí của từng đặc trưng, vẽ histogram — đó là σ thật của bạn; lặp lại có và không có tinh chỉnh subpixel, và với đặc trưng có độ tương phản cao và thấp riêng. <i>Phần hai</i> : trên chuỗi EuRoC, chạy BA với hai chiến lược chọn đặc trưng — “top-N theo điểm số” và “lưới phân bố” — với <i>cùng tổng số</i> đặc trưng; đo số điều kiện của Λ và ATE cho cả hai.
Xong khi	bạn có một con số σ của riêng mình kèm điều kiện đo, và bạn dùng nó làm trọng số trong mọi thứ về sau thay vì một hằng số đoán. Và bạn có hai đường cong số điều kiện cho thấy phân bố lưới cho Λ tốt hơn đáng kể ở cùng số đặc trưng — nếu không quan sát được, hãy kiểm xem cảnh của bạn có vân quá đều không, và đó cũng là một kết quả đáng ghi.

Nối với A/06-bat-dinh-va-lan-truyen-sai-so dùng σ này làm đầu vào cho toàn bộ ngân sách sai số; C/18-danh-gia-va-phuong-phap-luan biến phần hai thành một ablation study có kỷ luật.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Từ $N = \log(1 - p) / \log(1 - w^s)$ với $w = 0,5$, $p = 0,99$: năm điểm cần khoảng 146 vòng, tám điểm cần khoảng 1177 — *tám lần*. Với $w = 0,3$ thì chênh gần bốn mươi lần. Lớn hơn trực giác vì quan hệ là *hàm mũ* theo s : mỗi điểm thêm vào nhân xác suất thành công của một mẫu với w , nên với $w = 0,5$ thì ba điểm thêm chia xác suất cho tám. Đây là lý do thực dụng chính để dùng thuật toán năm điểm, cùng với lý do quan trọng không kém là nó không suy biến với cảnh phẳng (A/01 mục 6B).
- A2. *Phân bố không gian* của đặc trưng. Hệ tụ đặc trưng ở vùng vãn mạnh cho các hàng Jacobian gần tuyến tính phụ thuộc — theo công thức Jacobian ở A/03, đóng góp phụ thuộc vị trí trong ảnh — nên Λ có số điều kiện lớn và bài toán gần suy biến. Một trăm đặc trưng trải đều cho nhiều thông tin hơn năm trăm đặc trưng tụ một chỗ. Lý do thứ hai, độc lập: nếu mọi đặc trưng ở trên một vật thì khi vật bị che hoặc di chuyển, hệ mất tất cả cùng lúc. Cách chữa rẻ nhất là lưới; tinh hơn thì quadtree hoặc ANMS.
- A3. Vì σ là trọng số $\Omega = \sigma^{-2}$ cho mọi phần dư chiều, nên nó quyết định trọng số *tương đối* giữa nhân tử thị giác và mọi nhân tử khác trong đồ thị. Hiệp phương sai đầu ra tỉ lệ σ^2 , nên giảm σ hơn ba lần thì hiệp phương sai giảm hơn mười lần — lớn hơn hầu hết cải tiến thuật toán. Và qua σ_d trong $\sigma_z = z^2 \sigma_d / (bf)$, nó quyết định tầm hữu ích của stereo: ở $z = 5$ m với $b = 12$ cm, $f = 400$ px, $\sigma = 0,5$ px cho $\sigma_z \approx 26$ cm còn $\sigma = 0,15$ px cho ≈ 8 cm. Cùng phần cứng, ba lần khác biệt.
- A4. “GPU đắt” chỉ là một phần. Ba lý do quan trọng hơn, và cả ba thuộc về D/21. *Tính dự đoán được*: ORB hỏng theo cách đã biết và lặp lại được; mạng học được hỏng khi dữ liệu lệch khỏi phân bố huấn luyện, và không ai đo được điều đó tại hiện trường. *Tính tất định*: cùng đầu vào cho cùng đầu ra bit-exact, điều kiện bắt buộc để tái hiện một lỗi từ hiện trường. *Chi phí cập nhật*: đổi mô hình đòi tái kiểm định toàn bộ; đổi ngưỡng thì không. Trong sản phẩm, ba yếu tố đó thường quyết định hơn độ chính xác trung bình.
- A5. Vì ratio test so ứng viên gần nhất với ứng viên gần *thứ hai trong tập ứng viên*. Nếu cánh cửa thứ hai không nằm trong tập — ngoài khung hình, hoặc ngoài vùng tìm kiếm — thì ứng viên sai là ứng viên duy nhất, tỉ số rất tốt, và khớp được chấp nhận với độ tin cậy cao. Thứ cứu được chỉ có thể là *hình học*: kiểm định rằng khớp này nhất quán với đa số khớp khác (RANSAC), hoặc nhất quán với tiên nghiệm chuyển động. Nguyên tắc chung: thông tin để phát hiện một khớp sai không nằm trong khớp đó mà nằm trong quan hệ của nó với các khớp khác — và ở quy mô lớn hơn, đây chính là vấn đề perceptual aliasing ở B/11.

NÓI LẠI THẬT ĐƠN GIẢN

Đưa cho một người hai bức ảnh chụp cùng một căn phòng từ hai chỗ đứng và bảo họ nối các điểm tương ứng. Họ sẽ làm ba việc, theo đúng thứ tự.

Đầu tiên họ tìm những chỗ *đáng nhớ*. Không phải giữa bức tường trắng — chỗ đó nhìn đâu cũng như nhau. Họ tìm góc bàn, mép khung tranh, vết bẩn trên sàn. Nguyên tắc: một chỗ đáng nhớ là chỗ mà dịch đi một chút theo *bất kỳ* hướng nào thì nó trông khác ngay. Một đoạn mép bàn thẳng thì không đạt — trượt dọc theo mép, nó vẫn y như cũ.

Thứ hai họ ghép. Với mỗi chỗ đáng nhớ ở ảnh một, tìm chỗ giống nhất ở ảnh hai. Và ở bước này họ chắc chắn sẽ mắc lỗi, vì trong phòng có bốn cái ghế giống hệt nhau.

Thứ ba — và đây là bước quan trọng nhất — họ *lùi lại nhìn tất cả các đường nối cùng lúc*. Nếu hai mươi đường đi theo một quy luật chung và một đường đi ngược hẳn, họ gạch đường đó đi. Không phải vì nhìn kỹ hơn vào cái ghế, mà vì đường đó không hợp với số đông.

Đó là toàn bộ chương này, và bài học lớn nhất nằm ở bước ba: *không có cách nào nhìn kỹ hơn vào một cái ghế để biết nó là cái nào*. Thông tin đó không tồn tại trong cái ghế. Nó chỉ tồn tại trong việc cái ghế ấy ở đâu so với mọi thứ khác.

Còn hai chi tiết nhỏ mà ít ai để ý nhưng lại quan trọng hơn vẻ ngoài. Một: đừng chọn tất cả các chỗ đáng nhớ ở cùng một góc ảnh — trải chúng ra khắp bức ảnh, vì mỗi vùng nói cho bạn một chuyện khác nhau. Hai: hãy xác định vị trí của chúng *chính xác nhất có thể*, chính xác hơn cả một ô pixel. Nghe như chuyện vụn vặt, nhưng con số ấy nhân lên qua toàn bộ phép tính phía sau, và cải thiện nó ba lần thì kết quả cuối cải thiện gấp mười.

Chỗ không rút gọn được: chỗ không rút gọn được là những căn phòng mà *mọi thứ* đều lặp lại — một hành lang kho hàng dài với các kệ giống hệt nhau. Ở đó bước ba cũng thất bại, vì số đông cũng có thể sai một cách nhất quán. Đó là bài toán khó nhất trong triển khai thật, và B/11 nói rằng nó vẫn chưa có lời giải tốt.

Một câu: Không có gì trong một khớp cho biết nó đúng hay sai — chỉ có quan hệ của nó với mọi khớp khác mới cho biết.

1.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *không descriptor nào phân biệt được hai vật thật sự giống hệt nhau, nên tăng hình học là bắt buộc* — gần như đúng theo định nghĩa nếu “giống hệt” hiểu chặt. Nó trở nên có thể bác bỏ khi ta hỏi về trường hợp thực tế: hai cánh cửa gần giống nhau. Ở đó, một descriptor đủ tốt có thể khai thác các khác biệt nhỏ — vết xước, khác biệt ánh sáng — và mệnh đề bị bác bỏ nếu một hệ thuần về ngoài đạt tỉ lệ khớp sai thấp ngang hệ có kiểm định hình học trên môi trường lặp lại. Tôi cho là không, nhưng LoFTR và MAST3R đi theo hướng khai thác ngữ cảnh rộng hơn, nên ranh giới đang dịch chuyển và tôi không chắc nó dừng ở đâu (*tôi suy ra, chưa đối chiếu nguồn*).

Mệnh đề thứ hai — *phân bố không gian ảnh hưởng tới số điều kiện của Λ* — suy ra trực tiếp từ cấu trúc Jacobian (của a) và kiểm được bằng mini project (của b). Nó bị bác bỏ nếu thí nghiệm cho thấy số điều kiện không khác nhau đáng kể giữa hai chiến lược chọn ở cùng số đặc trưng. Một trường hợp mà nó *thật sự* không đúng: khi cảnh có vân đều khắp nơi, top-N tự nhiên đã trải đều, và hai chiến lược trùng nhau.

Mệnh đề thứ ba — *tính chỉnh subpixel cho σ khoảng 0,1–0,2 px* — là con số tôi không có nguồn đo, và tôi đã nói rõ điều đó ở khối *Cạm bẫy*. Nó dễ bác bỏ nhất trong chương: chỉ cần đo, và mini project chính là phép đo. Nếu σ thật của bạn là 0,4 px thì mọi tính toán ngân sách sai số dựa trên 0,15 px đều sai gấp bảy lần về phương sai.

Một mệnh đề mềm: chương này khẳng định *front-end quyết định kiểu hồng, back-end quyết định độ chính xác*. Điều đó bị bác bỏ nếu trong một khảo sát ca hồng thật, phần lớn thất bại bắt nguồn từ back-end — phân kỳ số học, cực tiểu địa phương — chứ không từ data association. Tôi không có số liệu và đây là chỗ tôi phát biểu dựa trên cấu trúc của vấn đề (*tôi suy ra, chưa đối chiếu nguồn*); D/21 đề nghị cách thu số liệu đó.

1.7 Kết nối

Nói lên trên. A/01-hình-hoc-da-thi cung cấp tầng kiểm định hình học ở mục 5, và đồng thời nhận từ chương này đầu vào của nó — hai chương nối vòng với nhau. A/02-mo-hình-camera là bước gỡ méo phải chạy trước khi dùng bất kỳ ràng buộc hình học nào. A/03-lie-group-va-toi-uu-tren-da-tap giải thích vì sao phân bố đặc trưng ảnh hưởng tới điều kiện số: qua cấu trúc của Jacobian. A/05-observability-gauge-va-suy-bien là chỗ ma trận cấu trúc M ở mục 2A được nhìn đúng bản chất — một ma trận thông tin suy biến ở quy mô nhỏ. A/06-bat-dinh-va-lan-truyen-sai-so là chương nhận σ từ mục 4B và biến nó thành ngân sách sai số.

Nổi xuống dưới. [B/08-mo-hinh-phan-du](#) là lựa chọn tiếp theo: một khi có tương ứng, phần dư viết thế nào — và với direct method thì chương này gần như biến mất, vì không còn bước trích đặc trưng. [B/09-paradigm-uoc-luong-back-end](#) nhận đầu ra của chương này làm đầu vào, và giả định rằng outlier đã được loại — giả định mà mục 5B cảnh báo là không bao giờ đúng hoàn toàn. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) là chương mà bài toán của chương này xuất hiện lại ở quy mô lớn hơn và với hậu quả nghiêm trọng hơn: khớp sai giữa hai khung liên kế chỉ làm hỏng một quan sát, còn khớp sai giữa hai địa điểm làm hỏng cả bản đồ. [B/12-relocalization](#) cũng vậy.

Nổi xa hơn. C/15-hop-nhat-da-cam-bien là chỗ mục 5C mở rộng: tiên nghiệm chuyển động từ bất kỳ cảm biến nào đều cải thiện front-end theo ba đường cùng lúc. C/17-hoc-may-trong-slam bàn kỹ hơn về front-end học được và lập luận rằng đây là chỗ học máy đóng góp rõ ràng nhất cho SLAM. D/19-ben-vung-trong-the-gioi-thuc xử lý ba giới hạn của RANSAC ở mục 5B, đặc biệt là trường hợp vật động chiếm đa số. D/20-toi-uu-hieu-nang-va-he-thong là chỗ chi phí của front-end trở thành ràng buộc cứng — nó thường chiếm phần lớn ngân sách tính toán của cả hệ.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao cạnh không theo dõi được còn góc thì được? Trả lời bằng ma trận cấu trúc, rồi nói nó liên hệ thế nào với A/05.
2. Tự dẫn lại công thức số vòng RANSAC, rồi tính cho $s = 4$ (homography) với $w = 0,4$, $p = 0,99$.
3. Nêu hai chế độ khớp — descriptor và optical flow — và nói mỗi cái phù hợp với tình huống nào và vì sao.
4. Ratio test cứu được trường hợp nào và không cứu được trường hợp nào? Nêu cơ chế duy nhất cứu được trường hợp thứ hai.
5. Vì sao phân bố đặc trưng ảnh hưởng tới độ chính xác nhiều hơn số lượng đặc trưng? Nêu hai lý do độc lập.
6. σ subpixel của bạn là bao nhiêu, bạn đo nó thế nào, và nó ảnh hưởng tới ba thứ nào ở back-end?
7. Nêu ba giới hạn của RANSAC, và với mỗi cái nêu một tình huống thực tế mà nó thất bại im lặng.

Nguồn

- [1] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, mức tin A.
- [2] J. Shi and C. Tomasi, “Good Features to Track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 1994.
- [3] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2006.
- [4] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2011.
- [5] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [6] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [7] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [8] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [9] J.-Y. Bouguet, “Pyramidal Implementation of the Affine Lucas Kanade Feature Tracker: Description of the Algorithm,” Intel Corporation, tech. rep., 2001, mức tin B; báo cáo kỹ thuật, là tài liệu tham chiếu de-facto cho KLT thấp.
- [10] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [11] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; nguồn của inverse compositional.
- [12] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [13] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [14] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2005.
- [15] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2020.
- [16] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.

- [17] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [18] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [19] C. Forster, Z. Zhang, M. Gassner, et al., "SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.
- [20] G. Klein and D. Murray, "Parallel Tracking and Mapping for Small AR Workspaces," in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A; công trình tách tracking khỏi mapping thành hai luồng, 2007.
- [21] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," in *European Conference on Computer Vision (ECCV)*, mức tin A, 2014.
- [22] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [23] J. Civera, A. J. Davison, and J. M. M. Montiel, "Inverse Depth Parametrization for Monocular SLAM," *IEEE Transactions on Robotics*, vol. 24, no. 5, pp. 932–945, 2008, mức tin A; nguồn của lập luận vì sao inverse depth tuyến tính hoá tốt hơn.
- [24] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của cách khử landmark bằng chiếu lên null space, 2007.
- [25] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2024.
- [26] L. Piccinelli, Y.-H. Yang, C. Sakaridis, et al., "UniDepth: Universal Monocular Metric Depth Estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [27] A. Rosinol, M. Abate, Y. Chang, et al., "Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [28] M. Kaess, H. Johannsson, R. Roberts, et al., "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [29] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên Sim(3), 2010.
- [30] V. Usenko, N. Demmel, D. Schubert, et al., "Visual-Inertial Mapping with Non-Linear Factor Recovery," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [31] N. Demmel, C. Sommer, D. Cremers, et al., "Square Root Bundle Adjustment for Large-Scale Reconstruction," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [32] D. M. Rosen, L. Carlone, A. S. Bandeira, et al., "SE-Sync: A Certifiably Correct Algorithm for Synchronization over the Special Euclidean Group," *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019, mức tin A.
- [33] R. Hartley, J. Trumpf, Y. Dai, et al., "Rotation Averaging," *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.

- [34] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [35] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [36] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.
- [37] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012, mức tin A.
- [38] D. Nistér and H. Stewénus, “Scalable Recognition with a Vocabulary Tree,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2006.
- [39] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018, mức tin A.
- [40] M. J. Milford and G. F. Wyeth, “SeqSLAM: Visual Route-Based Navigation for Sunny Summer Days and Stormy Winter Nights,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [41] Y. Latif, C. Cadena, and J. Neira, “Robust Loop Closing over Time for Pose Graph SLAM,” *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013, mức tin A; thuật toán RRR.
- [42] N. Sünderhauf and P. Protzel, “Switchable Constraints for Robust Pose Graph SLAM,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2012.
- [43] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., “Robust Map Optimization Using Dynamic Covariance Scaling,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2013.
- [44] H. Yang, P. Antonante, V. Tzoumas, et al., “Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [45] M. J. Black and A. Rangarajan, “On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision,” *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [46] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [47] E. Brachmann and C. Rother, “Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022, mức tin B.
- [48] E. Brachmann, T. Cavallari, and V. A. Prisacariu, “Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2023.
- [49] A. Kendall, M. Grimes, and R. Cipolla, “PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2015.
- [50] T. Sattler, Q. Zhou, M. Pollefeys, et al., “Understanding the Limitations of CNN-Based Absolute Camera Pose Regression,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của khẳng định APR không tốt hơn image retrieval, 2019.

Mô hình phần dư (Residual Models)

PHỤ THUỘC A/02-mo-hinh-camera (mô hình quang trắc — bắt buộc cho direct); A/04-uoc-luong-map-va-bundle-adjustment (khung MAP — chương này chỉ thay hàm h mà không đổi gì khác); A/06-bat-dinh-va-lan-truyen-sai-so (phân bố lệch của độ sâu — lập luận đầy đủ cho nghịch đảo độ sâu); B/07-dac-trung-va-data-association (đầu vào cho indirect; direct thì bỏ qua bước này).

TÓM TẮT

Khung MAP ở A/04 là một cái khuôn: đưa vào một hàm đo h và một trọng số Ω , nhận về một bộ ước lượng. Chương này bàn về việc chọn h , và nó cho thấy khuôn ấy tổng quát tới mức nào — toàn bộ sự khác nhau giữa ORB-SLAM và DSO nằm ở một lựa chọn duy nhất trong chương này, chứ không nằm ở bộ giải. Nội dung có ba phần. Một: ba họ phần dư — indirect (sai số chiếu lại), direct (sai số cường độ), semi-direct — với phân tích cơ chế chứ không phải bảng số liệu, vì các con số công bố của hai họ này gần như không so sánh được. Hai: tham số hoá landmark, nơi một chi tiết thống kê ở A/06 quyết định một lựa chọn kiến trúc; và kỹ thuật structureless, nơi landmark biến mất hoàn toàn khỏi vector trạng thái nhờ một phép chiếu. Ba: các loại phần dư khác — độ sâu, đường thẳng, mặt phẳng, prior học được — và cái giá ẩn của chúng, vốn là phá vỡ cấu trúc thừa mà A/04 phụ thuộc vào. Nếu chỉ nhớ một điều: *lựa chọn phần dư quyết định hệ thống cần gì từ thế giới* — indirect cần vận có cấu trúc, direct cần ánh sáng ổn định, và đó là lý do không cái nào thắng ở mọi nơi.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phần dư chiếu lại (reprojection residual)** — hiệu giữa tọa độ pixel quan sát được của một đặc trưng và tọa độ mà mô hình dự đoán. Đơn vị pixel, hai chiều cho mỗi quan sát.
- **phần dư quang trắc (photometric residual)** — hiệu giữa *giá trị cường độ* của một pixel ở khung này và giá trị của pixel tương ứng ở khung kia. Đơn vị mức xám, một chiều cho mỗi pixel.
- **ngịch đảo độ sâu (inverse depth)** — tham số hoá một điểm bằng $\rho = 1/z$ thay vì z , thường kèm một khung *neo* và một hướng tia. Điểm ở vô hạn tương ứng $\rho = 0$, một giá trị hoàn toàn hợp lệ chứ không phải kỳ dị.
- **khung neo (anchor frame)** — khung hình mà tọa độ của landmark được biểu diễn tương đối với nó. Chọn khung đầu tiên nhìn thấy điểm làm neo giữ cho tham số hoá ổn định số học.
- **structureless** — kỹ thuật loại landmark khỏi vector trạng thái bằng cách chiếu phần dư lên không gian rỗng của Jacobian theo landmark. Kết quả là một ràng buộc chỉ giữa các tư thế.
- **độ sáng affine (affine brightness)** — mô hình $I' = aI + b$ cho thay đổi cường độ giữa hai khung do phơi sáng hoặc gain tự động. Hai tham số mỗi khung, đưa vào chính bài toán tối ưu.
- **gradient ảnh** — điều kiện tồn tại của direct method: phần dư quang trắc chỉ ràng buộc được tư thế ở những pixel có gradient khác không. Vùng phẳng đóng góp đúng bằng không.

CÂU HỎI MỞ ĐẦU

- Q1. Direct method dùng *mọi* pixel có gradient, indirect chỉ dùng vài trăm đặc trưng. Vì sao direct không thắng tuyệt đối, và thứ gì nó phải trả giá để có được nhiều dữ liệu như vậy?
- Q2. Tham số hoá một điểm bằng z và bằng $1/z$ mô tả cùng một điểm và cho cùng một nghiệm. Vậy vì sao lựa chọn giữa chúng lại đổi kết quả, và đổi ở đâu?

- Q3. MSCKF khử landmark khỏi vector trạng thái bằng phép chiếu null-space. Nếu landmark biến mất thì thông tin về nó đi đâu, và vì sao kỹ thuật này cho một bộ lọc chi phí tuyến tính thay vì bậc hai?
- Q4. Bạn thêm một ràng buộc “hai landmark này nằm trên cùng một mặt phẳng”. Ràng buộc đúng, và nó làm bài toán chậm đi nhiều bậc. Vì sao?
- Q5. Một mô hình học được cho độ sâu metric của mỗi pixel. Nghe như nó giải quyết bài toán tỉ lệ của mono. Nêu điều kiện để đưa nó vào đồ thị một cách hợp lệ, và nói vì sao đưa vào sai còn tệ hơn không dùng.

2.1 Đặt vấn đề

A/04 dựng một khuôn tổng quát: cực tiểu hoá $\sum_k \mathbf{r}_k^\top \Omega_k \mathbf{r}_k$. Khuôn ấy không nói $\mathbf{r}_k$ là gì. Chương này bàn đúng câu hỏi đó, và câu trả lời hoá ra là chỗ phân chia lớn nhất trong lịch sử Visual SLAM.

Bài toán gốc phát biểu thế này. Ta có một chuỗi ảnh — những mảng số. Ta muốn một hàm biến giả thuyết về tư thế và cấu trúc thành một dự đoán *kiểm được với ảnh*. Có hai cách làm việc đó, và chúng khác nhau ở chỗ *trừu tượng hoá ở đâu*.

Cách thứ nhất: *trừu tượng hoá sớm*. Biến ảnh thành một tập điểm có toạ độ và mô tả (B/07), rồi so sánh toạ độ dự đoán với toạ độ quan sát. Đây là **indirect**, và nó thống trị từ PTAM [20] tới ORB-SLAM [12].

Cách thứ hai: *không trừu tượng hoá*. So sánh trực tiếp giá trị cường độ: nếu tư thế đúng thì pixel này ở khung một phải có cùng độ sáng với pixel tương ứng ở khung hai. Đây là **direct**, và nó được LSD-SLAM [21] và DSO [22] phát triển thành một dòng hoàn chỉnh.

Ai gặp sự lựa chọn này? Mọi người thiết kế một hệ SLAM, và đó là một trong hai hoặc ba quyết định kiến trúc lớn nhất — nó lan toả vào mọi thứ khác, từ yêu cầu phần cứng tới cách hệ thống hỏng.

Điều đáng nói ngay là vì sao tranh luận này *không* giải quyết được bằng cách đo. Các con số công bố cho hai họ này gần như không so sánh được: chúng chạy trên các bộ dữ liệu khác nhau, với các mức hiệu chuẩn quang trắc khác nhau (DSO cần nó, ORB-SLAM không), và với các định nghĩa thất bại khác nhau. Đặt hai con số cạnh nhau tạo ra ảo giác so sánh — đúng thứ *research-rules.md* mục 6.2 cảnh báo là chỗ dễ tự lừa mình nhất.

Điều *có thể* nói một cách trung thực là mỗi họ *cần gì từ thế giới*, và cần gì thì hỏng khi thiếu cái đó. Indirect cần vắn có cấu trúc đủ để trích ra đặc trưng lặp lại được; nó bất chấp thay đổi ánh sáng trong giới hạn rộng. Direct cần ánh sáng ổn định hoặc một mô hình quang trắc đúng; nó bất chấp việc thiếu góc cạnh, chỉ cần có gradient. Hai yêu cầu ấy độc lập với nhau, nên có những môi trường mà cái này sống cái kia chết, và ngược lại. Đó là phát biểu trung thực nhất mà tôi đưa ra được, và nó hữu ích hơn một bảng số liệu.

TRỰC GIÁC

Hình dung bạn phải kiểm tra xem một bản đồ có khớp với thực địa không.

Cách indirect: bạn chọn ra hai mươi mốc dễ nhận — ngã tư, tháp nước, cây đa đầu làng — rồi kiểm xem từng mốc có ở đúng chỗ bản đồ nói không. Ưu điểm: mỗi mốc là một phép kiểm rõ ràng, và bạn không quan tâm hôm nay trời nắng hay mưa. Nhược điểm: bạn đã *vứt bỏ* toàn bộ phần còn lại của thực địa — những con đường, những cánh đồng — và nếu vùng đó không có mốc nào dễ nhận thì bạn bó tay.

Cách direct: bạn không chọn mốc nào. Bạn chồng bản đồ lên ảnh chụp từ trên cao và đo xem chúng lệch nhau bao nhiêu ở *mọi* chỗ có sự biến đổi — mọi ranh giới, mọi bờ ruộng. Ưu điểm: bạn dùng được rất nhiều thông tin, kể cả ở vùng không có mốc nào. Nhược điểm: nếu ảnh chụp hôm nay sáng hơn ảnh gốc — vì trời nắng hơn, vì máy chỉnh phơi sáng — thì mọi chỗ đều lệch, và bạn sẽ kết luận nhầm rằng bản đồ sai. Đó là toàn bộ sự đánh đổi. Cách một bên với thay đổi bề ngoài nhưng vứt bỏ phần lớn dữ liệu. Cách hai dùng gần hết dữ liệu nhưng phải biết chính xác điều kiện chụp. Và một chi tiết quyết định về cách hai: ở giữa một cánh đồng phẳng lì, không có gì để so. Vùng không có sự biến đổi đóng góp *đúng bằng không*. Nên direct không thật sự dùng “mọi pixel” — nó dùng mọi pixel *có gradient*, và ở một bức tường trắng thì con số đó cũng bằng không, y như indirect.

2.2 Ba họ phần dư

A. Indirect: sai số chiếu lại

$$\mathbf{r}_{ij}^{\text{ind}} = \mathbf{z}_{ij} - \pi(T_i^{-1} \mathbf{P}_j) \in \mathbb{R}^2.$$

Jacobian đã dẫn ở A/03 công thức (??). Trọng số $\Omega = \sigma^{-2}I$ với σ là độ chính xác subpixel đo ở B/07 mục 4B.

Ba tính chất quyết định. *Bên với vẻ ngoài:* descriptor được thiết kế bất biến với ánh sáng và một phần với góc nhìn, nên thay đổi phơi sáng không ảnh hưởng gì. *Ít phần dư:* vài trăm quan sát mỗi khung thay vì vài chục nghìn — rẻ, và cấu trúc thưa của A/04 giữ nguyên hoàn hảo. *Mất thông tin:* toàn bộ ảnh bị rút xuống vài trăm tọa độ; mọi thứ ở vùng ít vân bị vứt.

Điểm thứ ba là điểm đáng suy nghĩ nhất. Một bức tường trắng có gradient rất yếu nhưng *không phải bằng không*, và direct khai thác được phần thông tin ít ỏi đó trong khi indirect thì không — vì detector cần một cực đại rõ ràng để tồn tại. Đây là lý do direct thường sống lâu hơn trong hành lang trống, và nó là một lợi thế thật chứ không phải lý thuyết.

B. Direct: sai số cường độ

$$r_{ij}^{\text{dir}} = I_i(\mathbf{u}) - I_j(\pi(T_{ij} \pi^{-1}(\mathbf{u}, \rho))) \in \mathbb{R}.$$

Một chiều cho mỗi pixel. Trọng số dựa trên nhiễu cảm biến và độ tin cậy của gradient.

Ba yêu cầu mà direct đặt ra, và chúng là cái giá của nó.

Một — mô hình quang trắc. Nếu phơi sáng đổi giữa hai khung thì *mọi* phần dư lệch cùng một lượng, và bộ giải sẽ diễn giải điều đó thành một chuyển động. DSO xử lý bằng hai cách đồng thời: hiệu chuẩn ngoại tuyến vignette và CRF (A/02 mục 5B), và ước lượng trực tuyến hai tham số affine (a, b) cho mỗi khung [22]. Cách thứ hai đáng chú ý vì nó là luận đề trung tâm áp dụng ở quy mô nhỏ: thay vì fix một tham số ta không biết, thả nó ra làm biến.

Hai — khởi tạo tốt. Hàm mục tiêu quang trắc *rất* không lồi: bề mặt cường độ của một ảnh thật có hàng nghìn cực tiểu địa phương. Direct chỉ hội tụ trong một bán kính hẹp, nên nó bắt buộc phải dùng thấp ảnh (coarse-to-fine) và một tiên nghiệm chuyển động. Đây là giới hạn nghiêm trọng nhất và là lý do direct dễ mất bám khi chuyển động nhanh.

Ba — gradient. Pixel không có gradient cho Jacobian bằng không, đóng góp đúng bằng không. Nên “dùng mọi pixel” thực ra là “dùng mọi pixel có gradient”, và DSO chọn có chọn lọc vài nghìn pixel phân bố đều thay vì dùng hết — một quyết định hội tụ về chính nguyên tắc phân bố ở B/07 mục 4A.

C. Semi-direct

SVO [19] kết hợp theo một cách không phải thoả hiệp mà là phân công theo thể mạnh: *direct* cho *tracking*, *indirect* cho *mapping*.

Logic: *tracking* cần *nhANH* và chỉ giải sáu ẩn với khởi tạo tốt từ khung trước — đúng chế độ mà *direct* mạnh. *Mapping* cần *bền* vì landmark sống lâu và phải nhận ra lại sau nhiều khung — đúng chế độ mà *indirect* mạnh, vì descriptor cho phép nhận ra lại sau khi mất bám.

Kết quả là một hệ rất nhanh với độ bền chấp nhận được. Đây là một ví dụ tốt về việc *nhận ra hai bài toán con có yêu cầu khác nhau* thay vì chọn một công cụ cho cả hai — một khuôn mẫu thiết kế xuất hiện lại ở B/09 và C/13.

Bảng 2.1. Ba họ phần dư, so sánh theo *cơ chế và yêu cầu*, không theo con số. Cột “cần gì từ thế giới” là cột quyết định lựa chọn.

Họ	Cần gì từ thế giới	Hỏng khi	Được gì
Indirect	Vân có cấu trúc đủ để trích đặc trưng lặp lại được	Vùng ít vân; vân lặp lại (perceptual aliasing)	Bền với đổi ánh sáng và phơi sáng; rẻ; cấu trúc thừa hoàn hảo; dễ nhận ra lại sau mất bám
Direct	Ánh sáng ổn định hoặc mô hình quang trắc đúng; gradient khác không	Đổi phơi sáng không mô hình hoá; chuyển động nhanh (không lỗi); vật động	Dùng được vùng gradient yếu; độ chính xác subpixel ngăm; không cần bước trích đặc trưng
Semi-direct	Cả hai, nhưng ở hai chỗ khác nhau	Kế thừa cả hai chế độ hỏng, nhẹ hơn	Nhanh như <i>direct</i> , bền như <i>indirect</i> ở chỗ cần bền

2.3 Tham số hoá landmark

A. Vì sao XYZ không đủ

Cách hiển nhiên là ba toạ độ Euclid. Nó hỏng với điểm xa, và lý do đã được dẫn đầy đủ ở A/06 mục 3B — đây là câu trả lời cho câu hỏi mở đầu thứ hai.

Tóm tắt lập luận. Quan hệ giữa phép đo (disparity, hoặc parallax) và z là *nghịch đảo*. Nhiều Gauss trên phép đo, đi qua một hàm nghịch đảo, cho phân bố *lệch* trên z : đuôi phía xa dài hơn nhiều. Ở giới hạn parallax rất nhỏ, phân bố của z thậm chí không có kỳ vọng hữu hạn. Hệ quả: một ellipsoid Gauss trong toạ độ XYZ là mô tả *sai* về bất định của một điểm xa, và bộ giải làm việc với một mô hình sai.

Có một hệ quả thứ hai, thực dụng hơn: *điểm ở vô hạn không biểu diễn được*. Trong SLAM ngoài trời, nhiều điểm gần như ở vô hạn, và chúng cực kỳ hữu ích cho việc ước lượng xoay (A/03 mục 5, khối $[P_c]_{\times}$). Với tham số hoá XYZ, ta buộc phải hoặc gán cho chúng một z hữu hạn tuỳ tiện, hoặc vứt bỏ chúng. Cả hai đều lãng phí.

B. Nghịch đảo độ sâu có neo

Tham số hoá đúng [23]: biểu diễn điểm bằng một khung *neo* a , một hướng tia $\mathbf{m}$ trong khung đó, và một nghịch đảo độ sâu ρ :

$$\mathbf{P} = \mathbf{c}_a + \frac{1}{\rho} R_a \mathbf{m}.$$

Nếu hướng tia lấy trực tiếp từ quan sát đầu tiên và giữ cố định, chỉ còn *một* ẩn ρ — giảm từ ba xuống một, một khoản tiết kiệm đáng kể khi có hàng trăm nghìn landmark.

Ba lợi ích. *Phân bố gần Gauss*: ρ tỉ lệ tuyến tính với disparity, nên nhiều Gauss trên phép đo cho phân bố gần Gauss trên ρ — xấp xỉ của A/06 trở lại hợp lệ. *Điểm ở vô hạn hợp lệ*: $\rho = 0$ là một giá trị bình thường, không phải kỳ dị, nên một điểm rất xa vẫn nằm trong bài toán và vẫn đóng góp cho ước lượng xoay. *Tuyến tính hoá tốt hơn*: hàm đo gần tuyến tính theo ρ hơn nhiều so với theo z , nên Gauss-Newton hội tụ tốt hơn ở chính chế độ khó.

Cái giá: cần theo dõi khung neo, và khi khung neo bị marginalize thì phải chuyển landmark sang neo mới — một phép toán không miễn phí và là một nguồn lỗi cài đặt. Đây là loại phức tạp mà một hệ sản phẩm phải chấp nhận, và là lý do C/13 coi quản lý bản đồ là một chương riêng.

C. Structureless: khi landmark biến mất

MSCKF [24] đi xa hơn: *không* đưa landmark vào vector trạng thái. Đây là câu trả lời cho câu hỏi mở đầu thứ ba.

Ý tưởng. Với một landmark quan sát từ n khung, xếp toàn bộ phần dư thành một vectơ:

$$\mathbf{r} \approx H_x \delta_x + H_f \delta_f + \mathbf{n},$$

với δ_f là nhiễu loạn của landmark. Lấy N là ma trận có các cột trực chuẩn sinh ra *không gian rộng trái* của H_f , rồi nhân trái:

$$N^\top \mathbf{r} = N^\top H_x \delta_x + N^\top \mathbf{n}.$$

Landmark biến mất hoàn toàn. Cái còn lại là một ràng buộc *chỉ giữa các tư thế* — chính là thông tin mà landmark mang về hình học tương đối, tách khỏi vị trí cụ thể của nó.

Thông tin đi đâu? Nó không mất: nó được chuyển thành ràng buộc giữa các tư thế. Về mặt đại số, đây là phần bù Schur của A/04 mục 4C viết ở dạng khác — khử một khối biến nhưng giữ nguyên ảnh hưởng của nó lên phần còn lại. Khác biệt duy nhất là ở đây ta *không* thể ngược để lấy lại landmark, vì ta không cần nó.

Vì sao điều này quan trọng cho bộ lọc: chi phí của EKF là bậc ba theo kích thước trạng thái, nên một trạng thái chứa hàng nghìn landmark là bất khả thi. Structureless giữ trạng thái chỉ gồm một cửa sổ tư thế, nên chi phí không phụ thuộc số landmark. Đây chính là điều làm MSCKF khả thi trong khi EKF-SLAM thì không — và là một trong những kỹ thuật đẹp nhất trong cả cây, vì nó đổi một bài toán bậc ba thành bậc thấp bằng một phép chiếu.

Cái giá: landmark không được ước lượng, nên không có bản đồ để tái sử dụng. MSCKF là một hệ *odometry*, không phải SLAM đầy đủ — nó không đóng vòng được vì không có gì để nhận ra lại. Đây là một đánh đổi kiến trúc thật, và B/09 bàn về nó.

2.4 Các loại phần dư khác và cái giá ẩn

A. Phần dư độ sâu và ICP

Với RGB-D hoặc stereo, ta có phép đo độ sâu trực tiếp. Cách đưa vào đơn giản: thêm một chiều vào phần dư chiếu, $r_z = z_{\text{đo}} - z_{\text{dự đoán}}$, với trọng số theo công thức $\sigma_z \approx z^2 \sigma_d / (bf)$ ở A/06.

Điểm đáng nhấn mạnh: trọng số ấy *phải* phụ thuộc z . Gán một σ_z không đổi cho mọi điểm là một trong những lỗi phổ biến nhất với RGB-D, và nó gây hại theo cả hai chiều — điểm gần bị tin quá ít, điểm xa bị tin quá nhiều. Cái thứ hai nguy hiểm hơn: nó bơm sai lệch có hệ thống vào bài toán và là một nguồn của bệnh quá tự tin ở A/06 mục 4B.

ICP dùng phần dư hình học giữa hai đám mây điểm. Nó là chuẩn mực trong LiDAR SLAM và xuất hiện trong RGB-D SLAM dày. Điểm cần biết cho chương này: ICP có bài toán data association của riêng nó — việc ghép điểm gần nhất — và bài toán đó có cùng cấu trúc và cùng chế độ hỏng như B/07.

B. Đường thẳng và mặt phẳng: cái giá ẩn

Trong môi trường nhân tạo, đường thẳng và mặt phẳng có mặt khắp nơi và ổn định hơn điểm. Việc thêm chúng vào có vẻ hiển nhiên có lợi, và nó có — nhưng có một cái giá ít được nói tới, và đó là câu trả lời cho câu hỏi mở đầu thứ tư.

Quay lại A/04 mục 4B: phần bù Schur hoạt động được vì khối V là khối chéo, và V khối chéo vì không nhân tử nào nối hai landmark với nhau. Một ràng buộc “ba điểm này cùng nằm trên một mặt phẳng” là đúng một nhân tử nối ba landmark. Khối V mất tính khối chéo, nghịch đảo của nó không còn tách được thành các phép nghịch đảo 3×3 độc lập, và toàn bộ lợi thế tính toán của Schur biến mất.

Đây là một ví dụ quan trọng của một nguyên tắc chung: *thêm ràng buộc đúng không miễn phí*. Ràng buộc cấu trúc cho thông tin thêm nhưng phá vỡ tính thưa. Cách xử lý thực tế là mô hình hoá mặt phẳng như một *landmark riêng* với các điểm nối vào nó — lúc đó cấu trúc thưa khôi phục được một phần, vì các điểm lại chỉ nối với một biến chung thay vì nối với nhau. Ràng buộc Manhattan hoặc Atlanta (các mặt phẳng vuông góc hoặc song song với nhau) thì mạnh hơn nữa và cũng nguy hiểm hơn: chúng đúng trong nhiều toà nhà và sai trong nhiều toà nhà khác, và khi sai thì chúng bơm vào một sai lệch có hệ thống mà không có gì báo.

C. Prior độ sâu học được

Các mô hình độ sâu metric đơn ảnh gần đây — Depth Anything v2 [25], UniDepth [26] — cho một ước lượng độ sâu có đơn vị mét từ một ảnh duy nhất. Nghe như nó giải quyết bài toán tỉ lệ của mono, và đây là câu trả lời cho câu hỏi mở đầu thứ năm.

Cách đưa vào đồ thị đúng nguyên tắc: thêm một nhân tử prior trên ρ của mỗi landmark, với hiệp phương sai phản ánh độ tin cậy của mô hình. Điều này hợp lệ và hữu ích. Nhưng nó có ba điều kiện, và vi phạm bất kỳ điều kiện nào thì đưa vào còn tệ hơn không dùng.

Một — hiệp phương sai phải trung thực. Nếu mô hình sai nhiều hơn ta khai báo, prior kéo nghiệm đi và không có gì cảnh báo. Vấn đề: các mô hình độ sâu học được *không* đi kèm một bất định đã hiệu chuẩn, và bất định của chúng thay đổi mạnh theo loại cảnh — tốt trong nhà kiểu phòng khách, tệ với bề mặt phản chiếu hoặc cảnh ngoài phân bố huấn luyện. C/17 bàn về điều này.

Hai — sai số phải gần độc lập giữa các pixel. Không đúng chút nào: một mô hình đơn ảnh sai cả một vùng cùng lúc, theo một cách tương quan mạnh. Đưa vào như các prior độc lập là *đếm thừa* thông tin nghiêm trọng — nguồn số một của bệnh quá tự tin ở A/06 mục 4B.

Ba — phải có cổng phát hiện khi mô hình ra ngoài phân bố. Đây là điều kiện khó nhất và chưa ai giải tốt.

Cách dùng an toàn nhất trong thực tế: dùng prior học được để *khởi tạo* tỉ lệ và độ sâu, rồi giảm mạnh trọng số của nó (hoặc bỏ hẳn) khi đã có đủ quan sát hình học. Như vậy ta lấy được phần giá trị lớn nhất — vượt qua chỗ khó là khởi tạo — mà không để một mô hình không được hiệu chuẩn chi phối nghiệm dài hạn.

CẠM BÃY

Một nhân tử có hiệp phương sai sai còn tệ hơn không có nhân tử đó, và đây là nguyên tắc quan trọng nhất của mục 4.

Lý do nằm ở chính khung MAP. Trọng số $\Omega = \Sigma^{-1}$ quyết định nhân tử này được tin bao nhiêu *so với* mọi nhân tử khác. Khai Σ quá nhỏ nghĩa là bạn ra lệnh cho bộ giải tin nguồn này hơn tất cả — và nếu nguồn ấy có sai lệch có hệ thống, sai lệch đó sẽ chảy vào toàn bộ nghiệm, kéo theo cả những phần vốn được ước lượng tốt.

Điều làm nó đặc biệt nguy hiểm: *không có gì cảnh báo*. Phần dư của các nhân tử khác sẽ tăng lên, nhưng bộ giải diễn giải điều đó là “các nguồn kia kém chính xác” chứ không phải “nguồn này sai”. Bạn thấy một hệ thống hội tụ đẹp về một nghiệm sai.

Quy tắc thực hành: trước khi thêm bất kỳ loại nhân tử mới nào vào đồ thị, hãy trả lời được *hiệp phương sai của nó từ đâu ra*. Nếu câu trả lời là “tôi chỉnh cho tới khi nó chạy” thì bạn đã có một hằng số tùy tiện trong hệ thống, và ba năm sau không ai biết vì sao nó bằng con số đó. Với nguồn học được, câu trả lời trung thực thường là “tôi không biết”, và lúc đó cách đúng là dùng nó làm khởi tạo chứ không làm ràng buộc.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [16]	Indirect thuần; landmark XYZ với quản lý cẩn thận	Chứng minh indirect vẫn cạnh tranh khi front-end và quản lý bản đồ tốt
DSO [22]	Direct thừa; nghịch đảo độ sâu có neo; hai tham số affine mỗi khung	Cài đặt tham chiếu của mục 2B; đọc cách nó chọn pixel để thấy nguyên tắc phân bố
SVO-Pro [19]	Semi-direct: direct cho tracking, indirect cho mapping	Mục 2C — phân công theo thể mạnh thay vì thỏa hiệp
MSCKF [24]	Structureless qua chiếu null-space	Mục 3C — kỹ thuật làm bộ lọc khả thi
VINS-Mono [10]	Nghịch đảo độ sâu có neo trong cửa sổ trượt	Cho thấy chi phí thật của việc chuyển neo khi marginalize
Kimera [27]	Thêm nhân tử mặt phẳng và ngữ nghĩa vào đồ thị GTSAM	Ví dụ về cái giá ẩn ở mục 4B trong một hệ thật

MINI PROJECT SLAM · So sánh ba tham số hoá landmark trên cùng dữ liệu, đặc biệt ở điểm xa · 3 ngày

Mục tiêu	tự thấy vì sao nghịch đảo độ sâu thắng, và thấy nó thắng ở đâu — vì nó không thắng ở mọi nơi.
Dữ liệu	tự sinh, để kiểm soát được phân bố độ sâu: một quỹ đạo, và ba bản đồ — toàn điểm gần (1–3 m), toàn điểm xa (20–100 m), và trộn cả hai.
Các bước	dựng BA với ba tham số hoá: XYZ, nghịch đảo độ sâu ba ẩn, và nghịch đảo độ sâu một ẩn có neo cố định tia; với mỗi tổ hợp (tham số hoá $\times$ bản đồ), đo số vòng lặp tới hội tụ, ATE cuối, số điều kiện của Λ , và — quan trọng nhất — chạy Monte-Carlo để so <i>phân bố</i> sai số độ sâu của các điểm xa với một Gauss.
Xong khi	bạn có một bảng ba nhân ba, và bạn quan sát được: với bản đồ gần, ba tham số hoá gần như tương đương; với bản đồ xa, XYZ hội tụ chậm hơn rõ rệt và phân bố sai số của nó <i>lệch</i> trong khi phân bố theo ρ thì gần Gauss. Cái sau là kết quả quan trọng nhất và là thứ xác nhận A/06 mục 3B bằng số của chính bạn.
Nối với	B/09-paradigm-uoc-luong-back-end chọn kiến trúc dùng các tham số hoá này; C/13-quan-ly-ban-do xử lý chi phí thật của việc chuyển khung neo.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì “mọi pixel” thực ra là “mọi pixel có gradient khác không” — vùng phẳng đóng góp Jacobian bằng không, nên ở một bức tường trắng direct cũng nghèo thông tin như indirect. Và cái giá phải trả có ba phần: (a) *mô hình quang trắc* phải đúng, vì thay đổi phơi sáng làm mọi phần dư lệch cùng lượng và bộ giải diễn giải đó là chuyển động; (b) *khởi tạo tốt* là bắt buộc, vì hàm mục tiêu quang trắc cực kỳ không lồi với hàng nghìn cực tiểu địa phương — đây là giới hạn nghiêm trọng nhất và là lý do direct dễ mất bám khi chuyển động nhanh; (c) khó nhận ra lại sau mất bám, vì không có descriptor để so.
- A2. Vì nghiệm thì trùng nhau nhưng *đường đi tới nghiệm* và *mô tả bất định* thì không. Quan hệ phép đo- z là nghịch đảo, nên nhiều Gauss trên phép đo cho phân bố *lệch* trên z (ở giới hạn parallax nhỏ, không có cả kỳ vọng hữu hạn), trong khi trên $\rho = 1/z$ thì gần Gauss. Hệ quả: với XYZ, xấp xỉ Gauss của hiệp phương sai là mô tả sai, và tuyến tính hoá tệ hơn nên Gauss-Newton hội tụ chậm hơn — cả hai đúng ở chế độ khó nhất là điểm xa. Thêm nữa, XYZ không biểu diễn được điểm ở vô hạn, trong khi $\rho = 0$ là giá trị hợp lệ, nên ta giữ được các điểm xa vốn rất tốt cho ước lượng xoay.
- A3. Thông tin không mất — nó được chuyển thành một ràng buộc *chỉ giữa các tư thế*, chính là phần thông tin mà landmark mang về hình học tương đối, tách khỏi vị trí cụ thể của nó. Về đại số, đây là phần bù Schur ở A/04 viết ở dạng khác, khác duy nhất ở chỗ ta không thể ngược để lấy lại landmark. Nó cho bộ lọc chi phí không phụ thuộc số landmark vì chi phí EKF là bậc ba theo kích thước trạng thái, và trạng thái bây giờ chỉ gồm một cửa sổ tư thế. Cái giá: không có bản đồ để tái sử dụng, nên MSCKF là odometry chứ không phải SLAM — nó không đóng vòng được.
- A4. Vì nó phá vỡ tính khối chéo của khối V trong Hessian. Phần bù Schur ở A/04 mục 4C hoạt động được *chỉ vì* không nhân tử nào nối hai landmark với nhau, nên V^{-1} tách thành các phép nghịch đảo 3×3 độc lập. Một ràng buộc đồng phẳng là một nhân tử nối nhiều landmark, và nó phá đúng tính chất đó — toàn bộ lợi thế tính toán biến mất. Cách xử lý: mô hình hoá mặt phẳng như một *landmark riêng* với các điểm nối vào nó, để các điểm lại chỉ nối với một biến chung thay vì nối trực tiếp với nhau.
- A5. Ba điều kiện. (a) *Hiệp phương sai trung thực* — nhưng các mô hình độ sâu học được không đi kèm bất định đã hiệu chuẩn, và bất định của chúng thay đổi mạnh theo loại cảnh. (b) *Sai số gần độc lập giữa các pixel* — hoàn toàn không đúng, vì một mô hình đơn ảnh sai cả một vùng theo cách tương quan mạnh, nên đưa vào như các prior độc lập là đếm thừa thông tin nghiêm trọng. (c) *Có cổng phát hiện khi ra ngoài phân bố*. Đưa vào sai tệ hơn không dùng vì trọng số quá lớn sẽ kéo cả nghiệm theo một sai lệch có hệ thống, và bộ giải sẽ diễn giải phần dư tăng của các nguồn khác là “các nguồn kia kém chính xác”. Cách an toàn: dùng làm khởi tạo rồi hạ mạnh trọng số khi đã có đủ quan sát hình học.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn muốn biết mình đã đi bao xa, bằng cách so hai bức ảnh chụp trước và sau. Có hai cách làm, và chúng khác nhau ở một điểm duy nhất.

Cách một: chọn ra vài chục thứ dễ nhận trong ảnh — góc cửa sổ, mép bàn — rồi xem chúng dịch đi đâu. Bạn không quan tâm hôm nay đèn sáng hơn hay tối hơn, vì góc cửa sổ vẫn là góc cửa sổ. Nhưng nếu bạn đang nhìn một bức tường trắng trơn thì không có gì để chọn, và bạn bó tay.

Cách hai: không chọn gì cả. Chồng hai ảnh lên nhau và trượt qua trượt lại cho tới khi chúng khớp nhất. Bạn dùng được cả những chỗ mờ nhạt mà cách một bỏ qua. Nhưng nếu ai đó vừa bật thêm đèn, hai ảnh sẽ không bao giờ khớp — và bạn sẽ kết luận nhầm rằng mình đã di chuyển.

Cả hai cách đều đúng, và không cách nào thắng ở mọi nơi. Cách một cần thể giới *có hình thù rõ*; cách hai cần ánh sáng *đúng thay đổi*. Hai đòi hỏi đó chẳng liên quan gì tới nhau, nên có phòng cách này sống cách kia chết, và ngược lại. Đó là lý do tranh luận “cái nào tốt hơn” kéo dài hai mươi năm không có kết quả — vì nó là câu hỏi sai.

Còn một chuyện nữa, về cách ghi lại vị trí của một điểm mốc. Nếu điểm mốc ở gần, ghi “cách ba mét” là hợp lý. Nếu nó ở rất xa — một ngọn núi — thì “cách mười ki-lô-mét” hay “cách hai mươi ki-lô-mét” đều là những phỏng đoán tệ như nhau, và ghi con số nào cũng gây hiểu lầm rằng ta biết. Cách khôn hơn là ghi *độ gần*: “rất gần”, “khá gần”, “gần như không gần chút nào”. Với ngọn núi,

câu cuối là một mô tả hoàn toàn chính xác — và quan trọng là, nó vẫn cho biết ngọn núi nằm về hướng nào, thứ mà ta biết rất rõ.

Chỗ không rút gọn được: chỗ không rút gọn được là việc ước lượng bất định của một mô hình học được. Một cái máy nói “chỗ này cách ba mét” — nhưng nó chắc tới mức nào? Nó không biết, và ta cũng không có cách nào biết hộ nó. Đó là lý do phần cuối chương này khuyên dùng các mô hình ấy để khởi động chứ không để quyết định, và là một trong tám khoảng trống ở D/22.

Một câu: Chọn phần dư là chọn hệ thống của bạn sẽ đòi hỏi gì ở thế giới — và thế giới không phải lúc nào cũng đáp ứng.

2.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *indirect và direct đòi hỏi hai thứ độc lập từ thế giới, nên không cái nào thắng ở mọi nơi* — bị bác bỏ nếu một hệ thuộc một họ vượt trội hệ thuộc họ kia trên mọi loại môi trường trong một khảo sát có kiểm soát, với cùng phần cứng và cùng mức hiệu chuẩn. Tôi không biết một khảo sát như vậy tồn tại, và tôi nghi ngờ nó sẽ khó thực hiện công bằng vì DSO cần hiệu chuẩn quang trắc mà ORB-SLAM không cần — nên “cùng mức hiệu chuẩn” không định nghĩa được rõ ràng. Đây là một lý do cấu trúc khiến tranh luận này khó giải quyết bằng thực nghiệm (*tôi suy ra, chưa đổi chiều nguồn*).

Mệnh đề thứ hai — *ngịch đảo độ sâu cho phân bố gần Gauss còn XYZ thì lệch* — là hệ quả toán học của việc lan truyền qua hàm nghịch đảo (cửa a, dẫn ở A/06) và kiểm được bằng Monte-Carlo trong mini project (cửa b). Nó bị bác bỏ nếu thí nghiệm cho thấy hai phân bố lệch như nhau — điều sẽ xảy ra nếu parallax luôn lớn, tức nếu bạn không có điểm xa thật sự.

Mệnh đề thứ ba — *ràng buộc cấu trúc phá vỡ tính thừa* — là một sự kiện về cấu trúc ma trận, kiểm được bằng cách nhìn mẫu khác không. Điều *có thể* sai là kết luận thực dụng rằng cái giá đó đáng kể: với bài toán nhỏ hoặc với số ràng buộc ít, nó có thể không đo được. Tôi chưa đo.

Mệnh đề thứ tư — *một nhân tử có hiệp phương sai sai tệ hơn không có nhân tử đó* — là mệnh đề tôi tin nhất và cũng là mệnh đề khó kiểm nhất, vì nó đòi so sánh với một chân trị. Nó bị bác bỏ nếu một hệ thêm prior học được với hiệp phương sai đoán bừa vẫn cho ATE tốt hơn hệ không dùng, một cách nhất quán qua nhiều điều kiện. Điều này *có thể* xảy ra khi prior đủ tốt và bài toán gốc đủ thiếu ràng buộc — và nếu nó xảy ra thì kết luận đúng không phải “hiệp phương sai không quan trọng” mà là “bài toán gốc thiếu thông tin tới mức bất kỳ prior nào cũng giúp”, tức một chẩn đoán thuộc A/05.

2.6 Kết nối

Nối lên trên. A/04-uoc-luong-map-va-bundle-adjustment là khuôn mà chương này đổ vào, và điều đáng chú ý nhất là khuôn ấy *không đổi gì* dù ta thay indirect bằng direct — đó là bằng chứng mạnh nhất cho tính tổng quát của nó. A/02-mo-hinh-camera mục 5B là điều kiện tiên quyết cho direct. A/06-bat-dinh-va-lan-truyen-sai-so mục 3B cho lập luận đầy đủ về nghịch đảo độ sâu, và mục 4B cho lý do vì sao hiệp phương sai sai lại nguy hiểm. B/07-dac-trung-va-data-association là đầu vào của indirect, và điều thú vị là với direct thì cả chương đó gần như biến mất — direct không có bước trích đặc trưng, nhưng nó trả giá bằng yêu cầu khởi tạo tốt.

Nối xuống dưới. B/09-paradigm-uoc-luong-back-end nhận structureless ở mục 3C như một kỹ thuật nền tảng, và bàn về hệ quả kiến trúc của nó — odometry so với SLAM đầy đủ. B/10-khoi-tao chịu ảnh hưởng trực tiếp: direct cần khởi tạo tốt hơn nhiều so với indirect, nên

chiến lược khởi tạo phụ thuộc vào lựa chọn ở chương này. C/15-hop-nhat-da-cam-bien mở rộng nguyên tắc ở khối *Cam bẫy* mục 4: mỗi nguồn thêm vào đồ thị cần hiệp phương sai đúng và cổng loại outlier riêng. C/16-dense-mapping-va-bieu-dien dùng phần dư quang trắc và độ sâu ở quy mô dày. C/17-hoc-may-trong-slam bàn kỹ hơn về prior học được ở mục 4C, đặc biệt về bài toán hiệu chuẩn bất định.

D/19-ben-vung-trong-the-gioi-thuc là chỗ hai họ phần dư lộ ra chế độ hỏng thật: direct rất nhạy với vật động (vì một vật động làm hỏng cường độ của cả một vùng) và với thay đổi ánh sáng đột ngột. D/20-toi-uu-hieu-nang-va-he-thong là chỗ chi phí tính toán của lựa chọn này được cân: direct có nhiều phần dư hơn hàng chục lần nhưng mỗi phần dư rẻ hơn, và kết quả ròng phụ thuộc vào phần cứng.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu điều mà indirect cần từ thế giới và điều mà direct cần, rồi nói vì sao hai điều đó độc lập với nhau.
2. “Direct dùng mọi pixel.” Câu đó sai ở chỗ nào, và sự thật là gì?
3. Vì sao nghịch đảo độ sâu tuyến tính hoá tốt hơn XYZ với điểm xa? Trả lời bằng ngôn ngữ hình dạng phân bố, không bằng ngôn ngữ “nó chạy tốt hơn”.
4. Structureless khử landmark khỏi trạng thái. Thông tin đi đâu, và kỹ thuật này liên hệ thế nào với phần bù Schur ở A/04?
5. Vì sao thêm một ràng buộc đồng phẳng — một ràng buộc *đúng* — lại làm bài toán chậm đi nhiều bậc?
6. Nêu ba điều kiện để đưa một prior độ sâu học được vào đồ thị một cách hợp lệ, và nói điều kiện nào khó thoả nhất.
7. Vì sao một nhân tử có hiệp phương sai sai lại tệ hơn không có nhân tử đó, và vì sao không có gì cảnh báo khi điều đó xảy ra?

Nguồn

- [1] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, mức tin A.
- [2] J. Shi and C. Tomasi, “Good Features to Track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 1994.
- [3] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2006.
- [4] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2011.
- [5] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [6] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [7] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [8] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [9] J.-Y. Bouguet, “Pyramidal Implementation of the Affine Lucas Kanade Feature Tracker: Description of the Algorithm,” Intel Corporation, tech. rep., 2001, mức tin B; báo cáo kỹ thuật, là tài liệu tham chiếu de-facto cho KLT thấp.
- [10] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [11] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; nguồn của inverse compositional.
- [12] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [13] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [14] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2005.
- [15] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2020.
- [16] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.

- [17] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [18] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [19] C. Forster, Z. Zhang, M. Gassner, et al., "SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.
- [20] G. Klein and D. Murray, "Parallel Tracking and Mapping for Small AR Workspaces," in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A; công trình tách tracking khỏi mapping thành hai luồng, 2007.
- [21] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," in *European Conference on Computer Vision (ECCV)*, mức tin A, 2014.
- [22] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [23] J. Civera, A. J. Davison, and J. M. M. Montiel, "Inverse Depth Parametrization for Monocular SLAM," *IEEE Transactions on Robotics*, vol. 24, no. 5, pp. 932–945, 2008, mức tin A; nguồn của lập luận vì sao inverse depth tuyến tính hoá tốt hơn.
- [24] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của cách khử landmark bằng chiếu lên null space, 2007.
- [25] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2024.
- [26] L. Piccinelli, Y.-H. Yang, C. Sakaridis, et al., "UniDepth: Universal Monocular Metric Depth Estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [27] A. Rosinol, M. Abate, Y. Chang, et al., "Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [28] M. Kaess, H. Johannsson, R. Roberts, et al., "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [29] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên Sim(3), 2010.
- [30] V. Usenko, N. Demmel, D. Schubert, et al., "Visual-Inertial Mapping with Non-Linear Factor Recovery," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [31] N. Demmel, C. Sommer, D. Cremers, et al., "Square Root Bundle Adjustment for Large-Scale Reconstruction," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [32] D. M. Rosen, L. Carlone, A. S. Bandeira, et al., "SE-Sync: A Certifiably Correct Algorithm for Synchronization over the Special Euclidean Group," *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019, mức tin A.
- [33] R. Hartley, J. Trumpf, Y. Dai, et al., "Rotation Averaging," *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.

- [34] P. Geneva, K. Eickenhoff, W. Lee, et al., "OpenVINS: A Research Platform for Visual-Inertial Estimation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [35] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [36] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.
- [37] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012, mức tin A.
- [38] D. Nistér and H. Stewénus, "Scalable Recognition with a Vocabulary Tree," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2006.
- [39] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018, mức tin A.
- [40] M. J. Milford and G. F. Wyeth, "SeqSLAM: Visual Route-Based Navigation for Sunny Summer Days and Stormy Winter Nights," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [41] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013, mức tin A; thuật toán RRR.
- [42] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2012.
- [43] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2013.
- [44] H. Yang, P. Antonante, V. Tzoumas, et al., "Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [45] M. J. Black and A. Rangarajan, "On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision," *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [46] T. Schneider, M. Dymczyk, M. Fehr, et al., "maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [47] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022, mức tin B.
- [48] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2023.
- [49] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2015.
- [50] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-Based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của khẳng định APR không tốt hơn image retrieval, 2019.

Paradigm ước lượng back-end (Back-End Estimation Paradigms)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (MAP, Schur, marginalization — chương này là các lựa chọn khác nhau trong khung đó); A/05-observability-gauge-va-suy-bien (nhất quán và FEJ); B/08-mo-hinh-phan-du (structureless là kỹ thuật nền cho MSCKF). Phần quán tính tương ứng ở ../IMU_Fusion/ chương 8 và 9.

TÓM TẮT

Người mới học SLAM gặp một danh sách các kiến trúc back-end trông không liên quan: EKF, MSCKF, cửa sổ trượt, fixed-lag, iSAM2, full batch BA, pose graph. Mỗi cái có bài báo riêng, ký hiệu riêng, và cộng đồng riêng. Chương này lập luận rằng chúng là *một* bài toán MAP của A/04 với các lựa chọn khác nhau trên đúng hai trục: *marginalize những gì*, và *tuyến tính hoá lại bao nhiêu lần*. Hiểu hai trục đó thì danh sách sụp thành một bảng, và việc chuyển qua lại giữa các thể giới trở nên tự nhiên. Nội dung có bốn phần. Một: chứng minh sự tương đương, và chỉ ra chính xác chỗ bộ lọc đánh đổi cái gì. Hai: bốn kiến trúc chính đặt trên hai trục ấy. Ba: pose graph như một trường hợp riêng quan trọng — nó là một bài toán MAP mà mọi landmark đã bị marginalize. Bốn: hai hướng phát triển đáng biết là dạng căn bậc hai (ổn định số học) và certifiable (bảo đảm nghiệm toàn cục). Nếu chỉ nhớ một điều: *bộ lọc là bài toán tối ưu đã marginalize toàn bộ quá khứ và chỉ tuyến tính hoá một lần* — mọi khác biệt về chi phí và về tính nhất quán đều là hệ quả của câu đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bộ lọc (filter)** — bộ ước lượng chỉ giữ trạng thái hiện tại và hiệp phương sai của nó, cập nhật tuần tự khi có phép đo mới. Mọi thông tin quá khứ được nén vào hai đại lượng đó.
- **smoothing** — bộ ước lượng giữ *nhiều* trạng thái quá khứ và giải lại cho tất cả khi có phép đo mới. Kết quả tốt hơn vì một phép đo muộn có thể sửa một ước lượng sớm.
- **cửa sổ trượt (sliding window)** — smoothing trên một cửa sổ kích thước cố định; trạng thái ra khỏi cửa sổ được marginalize thành nhân tử prior.
- **fixed-lag smoothing** — tên khác của cửa sổ trượt, nhấn mạnh rằng ước lượng cho một thời điểm được chốt sau một độ trễ cố định.
- **smoothing gia tăng (incremental smoothing)** — giữ *toàn bộ* lịch sử nhưng chỉ tính lại phần bị ảnh hưởng khi có phép đo mới. iSAM2 làm điều này qua cấu trúc Bayes tree.
- **pose graph** — đồ thị nhân tử chỉ gồm các tư thế, với các cạnh là ràng buộc tư thế tương đối. Landmark đã bị khử; đây là dạng nén nhất của bài toán còn giữ được tính nhất quán toàn cục.
- **dạng căn bậc hai (square-root form)** — làm việc với thừa số R sao cho $\Lambda = R^T R$ thay vì với Λ . Số điều kiện giảm còn căn bậc hai, nên ổn định số học tốt hơn hẳn.
- **certifiable** — thuật toán không chỉ trả về nghiệm mà còn trả về *chứng chỉ* rằng nghiệm đó là cực tiểu toàn cục — điều mà Gauss-Newton không bao giờ cho được.

CÂU HỎI MỞ ĐẦU

- Q1. EKF-SLAM và bundle adjustment cùng giải một bài toán MAP. Vậy khác biệt thật nằm ở đâu, và nêu chính xác hai lựa chọn phân biệt chúng.

- Q2. EKF-SLAM có chi phí bậc hai theo số landmark mỗi bước. Vì sao lại là bậc hai, khi mà trận thông tin của bài toán thì thưa?
- Q3. MSCKF giữ một cửa sổ tư thế và khử landmark. Nó rẻ và chính xác. Vì sao nó *không* phải một hệ SLAM, và điều đó có hậu quả gì cho sản phẩm?
- Q4. iSAM2 giữ toàn bộ lịch sử nhưng chạy thời gian thực. Nghe như nó có tất cả mà không mất gì. Điều gì nó vẫn phải trả giá, và khi nào cái giá đó trở nên nghiêm trọng?
- Q5. Dạng căn bậc hai cho cùng nghiệm với cùng chi phí bậc độ lớn, nhưng ổn định số học tốt hơn hẳn. “Tốt hơn hẳn” ở đây là bao nhiêu, và nó đến từ đâu?

3.1 Đặt vấn đề

Ai học SLAM theo thứ tự lịch sử sẽ gặp một danh sách gây bối rối. Đầu tiên là EKF-SLAM, thống trị những năm 1990 và 2000. Rồi PTAM [20] chứng minh rằng bundle adjustment chạy được thời gian thực, và cộng đồng chuyển sang tối ưu hoá. Rồi MSCKF [24] cho thấy bộ lọc vẫn cạnh tranh nếu làm đúng. Rồi iSAM2 [28] cho thấy smoothing đầy đủ cũng chạy thời gian thực được. Bốn câu chuyện, bốn cộng đồng, bốn bộ ký hiệu.

Cách học đó để lại một ấn tượng sai: rằng đây là những phương pháp *cạnh tranh* và ta phải chọn một. Ấn tượng ấy làm người học phải học lại từ đầu mỗi lần đổi hệ thống, và tệ hơn, nó che mất việc các đánh đổi là gì.

Bài toán gốc của chương: *tồn tại một khung nào mà trong đó cả bốn đều là trường hợp riêng không?* Câu trả lời là có, và khung đó chính là bài toán MAP ở A/04. Mọi kiến trúc trong danh sách trên đều cực tiểu hoá cùng một hàm mục tiêu; chúng khác nhau ở hai lựa chọn kỹ thuật:

Trục một — *marginalize* những gì. Giữ lại bao nhiêu quá khứ trong bài toán? Từ “không giữ gì” (bộ lọc) tới “giữ tất cả” (full batch).

Trục hai — *tuyến tính hoá lại bao nhiêu lần*. Jacobian được tính tại điểm nào, và có tính lại khi ước lượng di chuyển không? Từ “một lần, không bao giờ sửa” (EKF) tới “mỗi vòng lặp, cho mọi biến” (full batch).

Hai trục ấy giải thích *tất cả* khác biệt về chi phí, độ chính xác và tính nhất quán. Và quan trọng hơn, chúng cho một cách suy nghĩ: khi thiết kế hệ thống, bạn không chọn giữa bốn cái tên mà chọn một điểm trên một mặt phẳng hai chiều.

Ai gặp bài toán này? Mọi người thiết kế back-end, và quyết định này định hình yêu cầu tính toán, độ trễ, và khả năng đóng vòng. Nó cũng là quyết định khó đảo ngược nhất trong cả hệ thống — đổi detector thì mất một tuần, đổi kiến trúc back-end thì viết lại gần hết.

TRỰC GIÁC

Hình dung bạn đang ghi chép một chuyến đi dài.

Cách bộ lọc: bạn chỉ giữ *một* trang giấy, ghi “tôi đang ở đây, và tôi chắc chắn chừng này”. Mỗi khi có thông tin mới, bạn cập nhật trang đó rồi *xoá* thông tin cũ — vì nó đã được gộp vào con số hiện tại. Rất rẻ, bộ nhớ không đổi. Cái giá: nếu ba giờ sau bạn phát hiện một suy luận lúc sáng là sai, bạn không sửa được. Trang giấy không còn ghi suy luận đó nữa; nó chỉ còn kết quả.

Cách full batch: bạn giữ *toàn bộ* nhật ký, mọi quan sát từ đầu chuyến đi. Mỗi khi có thông tin mới, bạn đọc lại tất cả và suy luận lại từ đầu. Kết quả tốt nhất có thể, vì một thông tin muộn có thể sửa một hiểu lầm sớm. Cái giá: chi phí tăng theo độ dài chuyến đi, nên tới ngày thứ mười thì không kịp nữa.

Cách của sổ trượt: bạn giữ nhật ký của *ba ngày gần nhất* và tóm tắt mọi thứ cũ hơn thành một đoạn ghi chú. Chi phí không đổi, và bạn vẫn sửa được các hiểu lầm trong ba ngày qua. Cái giá nằm ở đoạn tóm tắt: một khi đã tóm tắt, bạn không mở lại được, và nếu lúc tóm tắt bạn đang hiểu sai thì cái hiểu sai ấy bị đóng băng.

Cách smoothing gia tăng: bạn giữ toàn bộ nhật ký nhưng tổ chức nó *rất khéo*, sao cho khi có thông tin mới bạn chỉ phải đọc lại đúng những trang bị ảnh hưởng. Nếu thông tin mới chỉ liên quan tới hôm nay thì bạn chỉ đọc hôm nay. Nhưng nếu bạn nhận ra hôm nay mình đang ở đúng chỗ đã đi qua tuần trước — một loop closure — thì bạn phải đọc lại cả tuần, và lúc đó chi phí vọt lên.

Bốn cách, một bài toán. Khác biệt duy nhất: giữ lại bao nhiêu, và có cho phép sửa lại suy luận cũ hay không.

3.2 Bộ lọc và tối ưu là cùng một bài toán

A. Chứng minh sự tương đương

Xét bài toán MAP với các trạng thái $\mathbf{x}_{0:k}$ và phép đo tới thời điểm k . Full batch giải cho tất cả:

$$\hat{\mathbf{x}}_{0:k} = \arg \min \left[\underbrace{\|\mathbf{x}_0 - \bar{\mathbf{x}}_0\|_{\Sigma_0}^2}_{\text{prior}} + \sum_i \|\mathbf{r}_i^{\text{do}}\|_{\Omega_i}^2 + \sum_i \|\mathbf{r}_i^{\text{chuyển}}\|_{Q_i}^2 \right].$$

Bây giờ áp dụng marginalization của A/04 mục 5 một cách cực đoan: sau mỗi bước thời gian, marginalize *mọi* trạng thái trừ trạng thái mới nhất. Theo mục 5A ở chương đó, marginalize một khối biến Gauss chính là phần bù Schur, và kết quả là một nhân tử prior trên các biến còn lại. Vì chỉ còn một trạng thái, prior ấy là một Gauss đơn giản: một trung bình và một hiệp phương sai.

Đó *chính xác* là những gì EKF giữ. Bước dự đoán của EKF là việc lan truyền prior qua nhân tử chuyển động; bước cập nhật là việc gộp nhân tử đo mới vào prior. Cùng công thức, cùng kết quả.

Nhưng có một khác biệt thứ hai, và nó là khác biệt quan trọng hơn. Trong full batch, khi ta thêm phép đo mới, ta *tuyến tính hoá lại* mọi phần dư tại ước lượng mới nhất. Trong EKF, Jacobian của một phép đo được tính *một lần* tại ước lượng lúc đó và không bao giờ sửa. Khi ước lượng di chuyển — và nó luôn di chuyển — Jacobian cũ trở nên lỗi thời, nhưng thông tin đã được nén vào hiệp phương sai và không tách ra được nữa.

CẦN NHỚ

Bộ lọc = bài toán tối ưu đã marginalize toàn bộ quá khứ, và chỉ tuyến tính hoá một lần.

Hai vế của câu này giải thích mọi thứ còn lại.

Về *marginalize toàn bộ quá khứ* giải thích chi phí: trạng thái không lớn lên theo thời gian, nên bộ lọc chạy được vô hạn. Nó cũng giải thích một hệ quả ít được nói: bộ lọc *không thể* sửa lại một ước lượng cũ, vì ước lượng cũ không còn tồn tại như một biến.

Về *tuyến tính hoá một lần* giải thích độ chính xác và tính nhất quán. Với mô hình phi tuyến mạnh — và phép chiếu camera phi tuyến mạnh — Jacobian tính tại một ước lượng tệ sẽ sai, và cái sai ấy bị đóng băng vĩnh viễn. Tệ hơn, như A/05 mục 6 chỉ ra, các Jacobian tính tại các điểm khác nhau *không tương thích* và tạo ra thông tin giả theo hướng không quan sát được. Đó là gốc của bệnh quá tự tin, và là lý do FEJ tồn tại.

Hệ quả thiết kế: nếu bạn cần chi phí không đổi và chấp nhận không đóng vòng, dùng bộ lọc và **bắt buộc** phải xử lý nhất quán (FEJ hoặc InEKF). Nếu bạn cần đóng vòng và sửa lại quá khứ, bạn cần giữ quá khứ, tức smoothing ở một dạng nào đó.

B. Vì sao EKF-SLAM có chi phí bậc hai

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và nó là một nghịch lý đáng hiểu.

Trong bài toán MAP, ma trận *thông tin* Λ của SLAM *thừa* — đó là toàn bộ nội dung của A/04 mục 4B. Nhưng EKF không giữ Λ ; nó giữ $\Sigma = \Lambda^{-1}$. Và như A/06 mục 2B đã nêu, *nghịch đảo của một ma trận thừa nói chung là đặc*.

Lý do trực giác: hai landmark chưa bao giờ được nhìn thấy cùng lúc vẫn *tương quan* với nhau, vì cả hai đều tương quan với quỹ đạo nối chúng. Thông tin “chúng không nối trực tiếp” nằm trong Λ ; thông tin “chúng vẫn tương quan” nằm trong Σ . Bộ lọc chọn biểu diễn thứ hai, và phải trả giá.

Hậu quả: với n landmark, Σ có $O(n^2)$ phần tử, và mỗi bước cập nhật EKF phải chạm vào tất cả — chi phí $O(n^2)$ mỗi bước, $O(n^2)$ bộ nhớ. Với vài nghìn landmark là hết. Đây là lý do kỹ thuật khiến EKF-SLAM không mở rộng được, và là lý do MSCKF quan trọng: bằng cách khử landmark qua *structureless* (B/08 mục 3C), nó giữ trạng thái chỉ gồm một cửa sổ tư thế, và chi phí không còn phụ thuộc số landmark.

Có một họ thứ ba đáng biết: *information filter*, giữ Λ thay vì Σ . Nó giữ được tính thừa — nhưng để lấy ra ước lượng thì phải giải một hệ, và để lấy hiệp phương sai thì phải nghịch đảo. Nói cách khác, nó chuyển chi phí từ lúc cập nhật sang lúc truy vấn. Với SLAM, nơi ta cập nhật liên tục nhưng chỉ thỉnh thoảng cần hiệp phương sai đầy đủ, đây thường là đánh đổi tốt hơn — và đó chính là cách mà các phương pháp smoothing hiện đại làm việc.

I 3.3 Bốn kiến trúc trên hai trục

Bảng 3.1. Bốn kiến trúc đặt trên hai trục: giữ lại bao nhiêu quá khứ, và tuyến tính hoá lại bao nhiêu. Mọi khác biệt về chi phí và nhất quán đều là hệ quả của hai cột đầu.

Kiến trúc	Giữ lại	Tuyến tính hoá lại	Hệ quả	Đại diện
Bộ lọc (EKF)	Chỉ trạng thái hiện tại	Không bao giờ	Chi phí không đổi theo thời gian; bắt nhất nếu không FEJ; không sửa được quá khứ	EKF-SLAM, ESKF
Bộ lọc cửa sổ (MSCKF)	Cửa sổ tư thế, không có landmark	Không bao giờ	Chi phí không phụ thuộc số landmark; không có bản đồ nên không đóng vòng	MSCKF, OpenVINS
Cửa sổ trượt	Cửa sổ tư thế + landmark; cũ hơn thì marginalize	Trong cửa sổ, mỗi vòng lặp	Chi phí không đổi; sửa được quá khứ gần; prior bị đóng băng gây fill-in và bắt nhất	VINS-Mono, OKVIS, Basalt
Smoothing gia tăng	Toàn bộ lịch sử	Phần bị ảnh hưởng, mỗi lần	Chính xác như full batch; chi phí thấp khi cập nhật cục bộ, vọt lên khi đóng vòng	iSAM2, GTSAM
Full batch	Toàn bộ	Tất cả, mỗi vòng lặp	Nghiệm tốt nhất có thể; không chạy thời gian thực	BA toàn cục, COLMAP

Ba nhận xét đọc ra từ bảng, và chúng quan trọng hơn bản thân bảng.

Thứ nhất, hai cột đầu là nguyên nhân, cột thứ ba là kết quả. Không có gì trong cột “hệ quả” là một lựa chọn thiết kế độc lập — tất cả đều suy ra từ hai lựa chọn trước. Điều này có nghĩa là nếu bạn muốn một hệ có tính chất nào đó ở cột ba, bạn phải tìm nó ở hai cột đầu.

Thứ hai, MSCKF không phải SLAM. Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Vì landmark bị khử ngay sau khi dùng, không có bản đồ nào tồn tại, nên không có gì để nhận ra lại khi quay về chỗ cũ. MSCKF là một hệ *odometry* xuất sắc: trôi chậm, rẻ, ổn định. Nhưng trôi của nó là đơn điệu — nó không bao giờ tự sửa. Với sản phẩm chạy trong một không gian hữu hạn và lặp lại — robot lau nhà, AGV kho hàng — điều đó không chấp nhận được, và ta cần một lớp SLAM đầy đủ ở trên. Cách phổ biến trong thực tế là ghép: MSCKF hoặc cửa sổ trượt cho odometry tần số cao, cộng một pose graph cho nhất quán toàn cục (B/11).

Thứ ba, iSAM2 vẫn phải trả giá. Đây là câu trả lời cho câu hỏi mở đầu thứ tư. Nó giữ toàn bộ lịch sử và chỉ tính lại phần bị ảnh hưởng, nhờ Bayes tree tổ chức biến theo một thứ tự khử tốt [28]. Khi cập nhật là cục bộ theo thời gian — thêm một khung, thêm vài landmark — phần bị ảnh hưởng nhỏ và chi phí thấp. Nhưng một *loop closure* nối một tư thế mới với một tư thế rất cũ, và nó làm mất hiệu lực một phần lớn của cây; chi phí vọt lên. Với bản đồ lớn và loop closure thường xuyên, cái giá đó trở nên nghiêm trọng, và đó là lý do các hệ thực tế thường tách loop closure ra một luồng riêng chạy chậm hơn (C/13).

3.4 Pose graph: bài toán MAP đã nén

Pose graph là một trường hợp riêng quan trọng tới mức đáng một mục riêng.

Ý tưởng: khử *mọi* landmark, giữ chỉ tư thế. Mỗi cạnh của đồ thị là một ràng buộc tư thế tương đối $\hat{T}_{ij}$ kèm hiệp phương sai, và phần dư là

$$\mathbf{r}_{ij} = \log(\hat{T}_{ij}^{-1} T_i^{-1} T_j)^\vee.$$

Các ràng buộc ấy đến từ hai nguồn: odometry (giữa hai tư thế liên tiếp) và loop closure (giữa hai tư thế xa nhau về thời gian).

Ba lý do pose graph quan trọng.

Một — kích thước. Với m tư thế và n landmark, bài toán đầy đủ có $6m + 3n$ ẩn với $n \gg m$; pose graph chỉ có $6m$. Điều này biến một bài toán không chạy nổi thành một bài toán giải được trong vài giây, kể cả với hàng chục nghìn tư thế.

Hai — tách mỗi quan tâm. Việc sửa trôi tích lũy là bài toán *toàn cục* và không cần độ chính xác cao ở quy mô nhỏ; việc tinh chỉnh cấu trúc cục bộ là bài toán *địa phương*. Tách chúng ra cho phép chạy hai luồng ở hai tần số khác nhau — một kiến trúc mà mọi hệ SLAM thật đều dùng.

Ba — nền cho certifiable. Vì pose graph chỉ chứa tư thế, nó có cấu trúc đại số đặc biệt cho phép các phương pháp bảo đảm nghiệm toàn cục (mục 5B).

Số bậc tự do của pose graph phải khớp với cấu hình cảm biến, đúng như A/05 mục 3B: $Sim(3)$ cho mono [29], $SE(3)$ cho stereo, và bốn bậc cho VIO vì trọng lực ghim roll với pitch. Đây không phải tối ưu hoá mà là tính đúng đắn: tối ưu một bậc tự do mà dữ liệu đã xác định chỉ tạo thêm đường cho sai số tích lũy.

Cái giá của pose graph, và nó đáng nói rõ: *thông tin bị nén một cách không phục hồi được*. Một cạnh với hiệp phương sai 6×6 không mô tả đầy đủ những gì hàng trăm quan sát landmark đã nói. Cụ thể, các cạnh khác nhau *tương quan* với nhau (vì chúng chia nhau các landmark), và pose graph chuẩn coi chúng độc lập — đúng nguồn số một của bệnh quá tự tin ở A/06 mục 4B. Đây là một xấp xỉ được chấp nhận rộng rãi vì lợi ích tính toán quá lớn, nhưng nên biết rằng nó là một xấp xỉ.

3.5 Hai hướng phát triển đáng biết

A. Dạng căn bậc hai

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó là một trong những cải tiến có tỉ lệ lợi ích trên chi phí cao nhất trong cả cây.

Thay vì dựng và phân tích $\Lambda = \sum H^\top \Omega H$, làm việc trực tiếp với một thừa số R sao cho $\Lambda = R^\top R$, thu được bằng phân tích QR của ma trận Jacobian đã whitened.

Lợi ích số học định lượng được, và đó là điểm mạnh của lập luận. Số điều kiện của Λ là *bình phương* số điều kiện của R :

$$\text{cond}(\Lambda) = \text{cond}(R^\top R) = \text{cond}(R)^2.$$

Nghĩa là việc dựng Λ tương minh *phá huỷ một nửa số chữ số có nghĩa*. Với bài toán có số điều kiện 10^8 — hoàn toàn bình thường trong BA có điểm ở nhiều thang độ sâu khác nhau — ta mất tám chữ số, và số học độ chính xác kép chỉ có mười sáu. Làm việc với R giữ được tám chữ số đó. (Cửa kiểm chứng a: suy dẫn trực tiếp từ định nghĩa số điều kiện qua giá trị kỳ dị.)

Lợi ích này lớn nhất đúng ở chỗ bài toán gần suy biến — tức đúng các cấu hình ở A/05. Basalt [30] và $\sqrt{\text{BA}}$ [31] khai thác điều này, và $\sqrt{\text{BA}}$ còn cho thấy nó cho phép dùng số học *độ chính xác đơn* trong nhiều trường hợp — tức nhanh gấp đôi và tốn nửa bộ nhớ, một lợi ích thật cho hệ nhúng (D/20).

Cái giá: phân tích QR đắt hơn Cholesky khoảng hai lần cho cùng bài toán, và cài đặt phức tạp hơn, đặc biệt khi kết hợp với phần bù Schur. Đây là loại đánh đổi mà một hệ sản phẩm nên cân nhắc nghiêm túc, vì tính ổn định thường quan trọng hơn tốc độ.

B. Certifiable: bảo đảm nghiệm toàn cục

Gauss-Newton và LM chỉ tìm được cực tiểu *địa phương*, và chúng không bao giờ cho biết nghiệm tìm được có phải toàn cục hay không. Với pose graph có loop closure, hàm mục tiêu *rất* không lồi và cực tiểu địa phương xấu tồn tại thật — một bản đồ “gập” vào chính nó là một cực tiểu địa phương hoàn toàn hợp lệ về mặt toán học.

SE-Sync [32] khai thác một cấu trúc đặc biệt: bài toán pose graph có thể viết lại thành một bài toán tối ưu trên đa tạp các ma trận bán xác định, và trong điều kiện nhiễu vừa phải, một nơi lồi của nó có nghiệm *trùng* với nghiệm của bài toán gốc. Khi điều kiện thoả, thuật toán trả về nghiệm kèm một *chứng chỉ* rằng đó là cực tiểu toàn cục.

Điều kiện “nhiều vừa phải” là chỗ đáng chú ý và cũng là giới hạn: với nhiễu lớn hoặc với outlier, bảo đảm mất hiệu lực. Nên certifiable không thay thế được back-end bền ở B/11 — hai thứ giải hai bài toán khác nhau, một cái lo cực tiểu địa phương và một cái lo dữ liệu sai.

Một hướng liên quan và thực dụng hơn: **rotation averaging** [33]. Tách bài toán thành hai bước — giải cho mọi phép xoay trước (bài toán này dễ hơn nhiều và có nghiệm gần toàn cục), rồi giải cho tịnh tiến với xoay đã cố định (bài toán này gần tuyến tính). Đây là nền của SfM toàn cục, đối lập với SfM tuần tự của COLMAP, và nó cho thấy rằng việc *chọn thứ tự giải* có thể quan trọng ngang việc chọn bộ giải.

CẠM BẦY

Đừng chọn kiến trúc back-end theo độ chính xác công bố. Ba lý do.

Thứ nhất, các con số không so sánh được: chúng chạy trên các bộ dữ liệu khác nhau, với front-end khác nhau, và front-end thường ảnh hưởng tới ATE nhiều hơn back-end.

Thứ hai, và quan trọng hơn, *độ chính xác trung bình không phải tiêu chí quyết định*. Bốn tiêu chí sau thường quyết định hơn trong sản phẩm: (a) **chi phí xấu nhất** — một hệ nhanh trung bình nhưng thỉnh thoảng mất một giây là không dùng được trong vòng điều khiển;

(b) **có đóng vòng được không** — quyết định hệ có trôi vô hạn hay không, và đó là khác biệt giữa dùng được và không dùng được trong không gian lặp lại; (c) **tính nhất quán** — hiệp phương sai có đáng tin không (A/06); (d) **hành vi khi hỏng** — nó phân kỳ ồn ào hay trôi im lặng?

Thứ ba, quyết định này khó đảo ngược nhất trong cả hệ thống. Đổi detector mất một tuần; đổi kiến trúc back-end là viết lại gần hết. Nên nó xứng đáng được quyết định dựa trên yêu cầu sản phẩm, không dựa trên bảng xếp hạng.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OpenVINS [34]	MSCKF với FEJ, hiệu chuẩn trực tuyến, mô phỏng có chân trị	Cài đặt bộ lọc sạch nhất; đọc để thấy cái giá của “tuyến tính hoá một lần”
VINS-Fusion [10]	Cửa sổ trượt + marginalization + pose graph riêng	Kiến trúc hai tầng điển hình: odometry nhanh + nhất quán toàn cục chậm
GTSAM [28]	iSAM2, Bayes tree, khử biến gia tăng	Cài đặt tham chiếu của smoothing gia tăng; đọc để hiểu vì sao loop closure đắt
Basalt [30]	Cửa sổ trượt dạng căn bậc hai; chất lượng kỹ thuật rất cao	Mục 5A trong một hệ thật; cũng là ví dụ tốt về kỹ thuật phần mềm
SE-Sync [32]	Pose graph certifiable với chứng chỉ toàn cục	Mục 5B; đọc để biết ranh giới của điều kiện “nhiều vừa phải”
ORB-SLAM3 [16]	Local BA + pose graph + BA toàn cục, ba luồng	Cho thấy các paradigm <i>kết hợp</i> chứ không loại trừ nhau

MINI PROJECT SLAM · Cài cùng một bài toán bằng ba kiến trúc, rồi đo cái mà bảng xếp hạng không đo · 4 ngày

Mục tiêu	tự thấy rằng ba kiến trúc cho cùng nghiệm khi mọi thứ thuận lợi, và thấy chúng phân kỳ ở <i>đâu</i> — vì chỗ phân kỳ mới là thứ quyết định lựa chọn.
Dữ liệu	tự sinh để có chân trị, cộng một chuỗi EuRoC để có dữ liệu thật.
Các bước	cài ba back-end trên cùng front-end và cùng dữ liệu: (a) EKF với toàn bộ landmark trong trạng thái, (b) cửa sổ trượt có marginalization, (c) full batch chạy lại từ đầu mỗi khung (chậm, làm chuẩn vàng); đo bốn thứ theo thời gian — ATE, thời gian mỗi bước (<i>cả trung bình lẫn phân vị 99</i>), bộ nhớ, và NEES so với chân trị; rồi bật/tắt FEJ cho (a) và (b) và đo lại NEES; cuối cùng thêm một loop closure và xem cái nào sửa được trôi.
Xong khi	bạn có bốn đường cong theo thời gian cho ba kiến trúc. Bạn phải quan sát được: (1) chi phí của (a) tăng bậc hai theo số landmark trong khi (b) không đổi; (2) NEES của (a) và (b) lệch lên khi không có FEJ và gần số chiều khi có — xác nhận A/05 mục 6; (3) chỉ (c) và (b) sửa được trôi khi có loop closure, còn (a) thì không nếu landmark đã ra khỏi trạng thái. Điểm quan trọng nhất: phân vị 99 của thời gian có thể kể một câu chuyện rất khác với trung bình.
Nối với	B/11-loop-closure-va-nhat-quan-toan-cuc xây trên pose graph ở mục 4; D/20-toi-uu-hieu-nang-va-he-thong-lay-phân-vị-99-làm-tiêu-chí-thiết-kế-chứ-không-phải-trung-bình .

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Chúng cực tiểu hoá cùng một hàm mục tiêu MAP. Hai lựa chọn phân biệt chúng: (1) *marginalize những gì* — EKF marginalize toàn bộ quá khứ sau mỗi bước, BA giữ tất cả; (2) *tuyến tính hoá lại bao nhiêu lần* — EKF tính Jacobian một lần tại ước lượng lúc đó và không bao giờ sửa, BA tính lại mọi Jacobian tại ước lượng tốt nhất hiện có ở mỗi vòng lặp. Về một giải thích chi phí (trạng thái không lớn lên) và giới hạn (không sửa được quá khứ). Về hai giải thích độ chính xác và tính nhất quán — Jacobian lỗi thời bị đóng băng, và các Jacobian tại các điểm khác nhau không tương thích nên tạo ra thông tin giả.
- A2. Vì EKF giữ $\Sigma = \Lambda^{-1}$ chứ không giữ Λ , và *ngịch đảo của một ma trận thưa nói chung là đặc*. Trực giác: hai landmark chưa bao giờ nhìn thấy cùng lúc vẫn tương quan, vì cả hai đều tương quan với quỹ đạo nổi chúng; Λ ghi “không nối trực tiếp”, còn Σ ghi “vẫn tương quan”. Với n landmark, Σ có $O(n^2)$ phần tử và mỗi cập nhật chạm tất cả. Có một lối thoát là information filter (giữ Λ , thưa) nhưng nó chuyển chi phí sang lúc truy vấn — và đó chính là cách các phương pháp smoothing hiện đại làm.
- A3. Vì landmark bị khử ngay sau khi dùng (structureless, B/08 mục 3C), nên *không có bản đồ nào tồn tại* — không có gì để nhận ra lại khi quay về chỗ cũ. MSCKF là odometry xuất sắc: trôi chậm, rẻ, ổn định, chi phí không phụ thuộc số landmark. Nhưng trôi của nó *đơn điệu* — nó không bao giờ tự sửa. Với sản phẩm chạy trong không gian hữu hạn và lặp lại (robot lau nhà, AGV kho), điều đó không chấp nhận được. Cách xử lý thực tế là ghép hai tầng: MSCKF hoặc cửa sổ trượt cho odometry tần số cao, cộng một pose graph cho nhất quán toàn cục.
- A4. Nó trả giá ở *loop closure*. Bayes tree cho phép chỉ tính lại phần bị ảnh hưởng, và khi cập nhật cục bộ theo thời gian — thêm một khung — phần đó nhỏ. Nhưng một loop closure nổi tư thế mới với tư thế rất cũ, làm mất hiệu lực một phần lớn cây, và chi phí vọt lên. Cái giá nghiêm trọng khi bản đồ lớn và loop closure thường xuyên — đúng tình huống của robot chạy lâu trong không gian lặp lại. Đó là lý do các hệ thực tế tách loop closure ra một luồng riêng chạy chậm hơn thay vì để nó chặn luồng tracking.
- A5. Tốt hơn một cách định lượng được: số điều kiện của R là căn bậc hai của số điều kiện của Λ , vì $\Lambda = R^T R$. Dựng Λ tường minh phá huỷ một nửa số chữ số có nghĩa. Với bài toán có $\text{cond} = 10^8$ — bình thường trong BA có điểm ở nhiều thang độ sâu — ta mất tám chữ số trong khi độ chính xác kép chỉ có mười sáu. Lợi ích lớn nhất đúng ở chỗ bài toán gần suy biến, tức các cấu hình ở A/05. Nó còn cho phép dùng độ chính xác đơn trong nhiều trường hợp [31], tức nhanh gấp đôi và tốn nửa bộ nhớ. Cái giá: QR đắt hơn Cholesky khoảng hai lần và cài đặt phức tạp hơn.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đang đi bộ đường dài và ghi lại mình đã đi qua đâu. Có bốn cách ghi, và chúng khác nhau đúng ở một điểm: bạn giữ lại bao nhiêu, và bạn có cho phép mình sửa lại những gì đã ghi hay không.

Cách một: chỉ giữ một mẫu giấy ghi “tôi đang ở đây”. Mỗi lần biết thêm điều gì, bạn sửa mẫu giấy rồi quên thông tin cũ đi. Rất nhẹ — mẫu giấy không dày lên dù bạn đi bao lâu. Nhưng chiều nay nếu bạn nhận ra rằng buổi sáng mình đã hiểu nhầm một ngã rẽ, bạn không sửa lại được, vì trên giấy không còn buổi sáng nữa.

Cách hai: giữ toàn bộ nhật ký từ đầu. Mỗi lần biết thêm điều gì, bạn đọc lại tất cả và nghĩ lại từ đầu. Kết quả luôn tốt nhất có thể. Nhưng đến ngày thứ mười, đọc lại hết mất cả buổi, và bạn không còn đi được nữa.

Cách ba: giữ nhật ký ba ngày gần nhất, còn cũ hơn thì tóm tắt thành một đoạn. Nhẹ, và vẫn sửa được những gì trong ba ngày. Cái bẫy nằm ở đoạn tóm tắt: một khi viết xong thì không mở lại được, và nếu lúc viết bạn đang hiểu nhầm thì cái hiểu nhầm đó đóng băng luôn ở đó.

Cách bốn: giữ hết, nhưng sắp xếp cực kỳ khéo — đánh dấu, phân mục — sao cho khi biết thêm điều gì, bạn chỉ phải đọc lại đúng những trang liên quan. Thường thì chỉ vài trang. Nhưng có một loại thông tin đặc biệt đắt: khi bạn nhận ra *chỗ mình đang đứng hôm nay chính là chỗ đã đi qua tuần trước*. Lúc đó mọi thứ ở giữa đều bị ảnh hưởng, và bạn phải đọc lại cả tuần.

Bốn cách, một việc. Và điều đáng nhớ nhất là: không có cách nào tốt hơn tuyệt đối. Nếu bạn chỉ cần biết mình đang ở đâu ngay bây giờ, cách một là đủ và rẻ nhất. Nếu bạn cần một tấm bản đồ đúng của cả chuyến đi, bạn buộc phải giữ lại quá khứ, và buộc phải trả giá cho nó.

Chỗ không rút gọn được: chỗ không rút gọn được là cái bẫy của đoạn tóm tắt ở cách ba. Ý tưởng “nén quá khứ lại nhưng giữ nguyên điều đã học được” nghe đơn giản, nhưng khi nén, các trang còn lại trở nên dính vào nhau theo những cách khó theo dõi, và điều đã học được thì ghi bằng một hệ quy chiếu đã lỗi thời. Đó là lý do các hệ cửa sổ trượt phức tạp hơn nhiều so với vẻ ngoài, và là lý do FEJ tồn tại.

Một câu: Bộ lọc là bài toán tối ưu đã quên hết quá khứ và chỉ được nghĩ một lần — mọi ưu và nhược của nó đều là hệ quả của câu đó.

3.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *bộ lọc là MAP đã marginalize toàn bộ quá khứ và tuyến tính hoá một lần* — là một tương đương toán học dưới giả thiết Gauss và mô hình đúng (cửa a, đối chiếu [35] và [36], cửa c). Nó bị bác bỏ nếu ai đó chỉ ra một bộ lọc mà hành vi của nó *không* tái tạo được bằng một bài toán MAP với marginalization tương ứng. Các bộ lọc phi Gauss — particle filter — nằm ngoài phạm vi này, và đó là một giới hạn thật của khung ở đây chứ không phải một ngoại lệ nhỏ.

Mệnh đề thứ hai — *EKF-SLAM có chi phí bậc hai* — là kết quả đếm phép toán, kiểm được bằng mini project (cửa b). Nó bị bác bỏ nếu một cài đặt khai thác cấu trúc nào đó trong Σ để đạt chi phí thấp hơn — và thực tế có những công trình làm điều này bằng cách xấp xỉ Σ bằng một ma trận thưa, với cái giá là mất tính chính xác của hiệp phương sai.

Mệnh đề thứ ba — *dạng căn bậc hai giảm số điều kiện xuống căn bậc hai* — là một đồng nhất thức đại số. Điều *có thể* sai là kết luận thực dụng rằng điều đó đáng kể: với bài toán có số điều kiện nhỏ, lợi ích không đo được. Mini project ở trên không kiểm điều này; đó là một thí nghiệm riêng đáng làm, và nó nằm trong 99-chua-biet.md.

Một mệnh đề mềm hơn: chương này khẳng định *bốn tiêu chí ở khối Cam bẫy quyết định hơn độ chính xác trung bình trong sản phẩm*. Đó là một phán đoán về ưu tiên kỹ thuật, không phải một sự kiện đo được, và nó có thể sai trong ứng dụng mà độ chính xác là yêu cầu chính — chẳng hạn quét đo đạc, nơi độ trễ không quan trọng và chỉ độ chính xác cuối cùng mới tính. Nêu rõ điều này vì cây này viết với mục tiêu sản phẩm robot, và ưu tiên ấy không phổ quát (*tôi suy ra từ mục tiêu đã nêu ở 00-map, chưa đổi chiều nguồn*).

3.7 Kết nối

Nối lên trên. A/04-uoc-luong-map-va-bundle-adjustment là khung mà toàn bộ chương này nằm trong; đặc biệt mục 5 ở đó về marginalization là công cụ chính của mục 2 ở đây. A/05-observability-gauge-va-suy-bien mục 6 giải thích bệnh bất nhất mà FEJ chữa — không có chương đó thì FEJ trông như một thủ thuật kỳ quặc. A/06-bat-dinh-va-lan-truyen-sai-so giải thích vì sao Σ đặc còn Λ thưa, tức gốc của chi phí bậc hai ở mục 2B. B/08-mo-hinh-phan-du mục 3C cho structureless, kỹ thuật làm MSCKF khả thi.

Nối xuống dưới. B/10-khoi-tao là chỗ mọi kiến trúc đều cần một khởi đầu, và yêu cầu về chất lượng khởi đầu khác nhau giữa chúng — bộ lọc nhạy hơn vì nó tuyến tính hoá một lần. B/11-loop-closure-va-nhat-quan-toan-cuc xây trực tiếp trên pose graph ở mục 4, và nó là chương giải thích vì sao khả năng đóng vòng — thứ mà MSCKF không có — lại quyết định tính dùng được của một hệ trong không gian lặp lại. C/13-quan-ly-ban-do là chỗ kiến trúc nhiều luồng trở thành một vấn đề kỹ thuật thật: tracking, local mapping và loop closing chạy ở ba tần số khác nhau và phải chia nhau một bản đồ.

Nói xa hơn. C/15-hop-nhat-da-cam-bien cho thấy lựa chọn kiến trúc ở đây quyết định việc thêm cảm biến dễ hay khó: với đồ thị nhân tử thì thêm một loại nhân tử là đủ, còn với bộ lọc thì phải sửa cả vector trạng thái và các ma trận. D/20-toi-uu-hieu-nang-va-he-thong lấy phân vị thời gian — không phải trung bình — làm tiêu chí, và giải thích vì sao một hệ nhanh trung bình nhưng thỉnh thoảng chậm là không dùng được trong vòng điều khiển.

Nói ngang: ../IMU_Fusion/ chương 8 và 9 bàn cùng chủ đề với trọng tâm quán tính, và nói kỹ hơn về ESKF, InEKF và cấu trúc nhân tử tiền tích phân. Chỗ nào cần chi tiết về bộ lọc bất biến thì đọc ở đó; chương này cố ý dừng ở mức nêu nó tồn tại và nó chữa cái gì.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu hai trục phân biệt mọi kiến trúc back-end, và đặt EKF, MSCKF, VINS-Mono và iSAM2 lên hai trục đó.
2. Vì sao Λ thừa mà Σ đặc? Trả lời bằng một câu về ý nghĩa vật lý, không bằng đại số.
3. MSCKF rẻ và chính xác nhưng không phải SLAM. Vì sao, và hậu quả gì cho một robot chạy tám tiếng trong một nhà kho?
4. iSAM2 giữ toàn bộ lịch sử mà vẫn chạy thời gian thực. Loại cập nhật nào làm nó đắt, và vì sao?
5. Tự dẫn lại vì sao $\text{cond}(\Lambda) = \text{cond}(R)^2$, rồi nói điều đó nghĩa là mất bao nhiêu chữ số với bài toán có $\text{cond}(\Lambda) = 10^{10}$.
6. Pose graph nén thông tin một cách không phục hồi được. Cụ thể cái gì bị mất, và nó liên hệ thế nào với bệnh quá tự tin ở A/06?
7. Nêu bốn tiêu chí quyết định lựa chọn back-end trong sản phẩm, và nói vì sao độ chính xác trung bình không nằm trong bốn cái đó.

Nguồn

- [1] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, mức tin A.
- [2] J. Shi and C. Tomasi, “Good Features to Track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 1994.
- [3] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2006.
- [4] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2011.
- [5] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [6] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [7] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [8] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [9] J.-Y. Bouguet, “Pyramidal Implementation of the Affine Lucas Kanade Feature Tracker: Description of the Algorithm,” Intel Corporation, tech. rep., 2001, mức tin B; báo cáo kỹ thuật, là tài liệu tham chiếu de-facto cho KLT thấp.
- [10] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [11] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; nguồn của inverse compositional.
- [12] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [13] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [14] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2005.
- [15] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2020.
- [16] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.

- [17] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [18] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [19] C. Forster, Z. Zhang, M. Gassner, et al., "SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.
- [20] G. Klein and D. Murray, "Parallel Tracking and Mapping for Small AR Workspaces," in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A; công trình tách tracking khỏi mapping thành hai luồng, 2007.
- [21] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," in *European Conference on Computer Vision (ECCV)*, mức tin A, 2014.
- [22] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [23] J. Civera, A. J. Davison, and J. M. M. Montiel, "Inverse Depth Parametrization for Monocular SLAM," *IEEE Transactions on Robotics*, vol. 24, no. 5, pp. 932–945, 2008, mức tin A; nguồn của lập luận vì sao inverse depth tuyến tính hoá tốt hơn.
- [24] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của cách khử landmark bằng chiếu lên null space, 2007.
- [25] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2024.
- [26] L. Piccinelli, Y.-H. Yang, C. Sakaridis, et al., "UniDepth: Universal Monocular Metric Depth Estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [27] A. Rosinol, M. Abate, Y. Chang, et al., "Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [28] M. Kaess, H. Johannsson, R. Roberts, et al., "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [29] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên Sim(3), 2010.
- [30] V. Usenko, N. Demmel, D. Schubert, et al., "Visual-Inertial Mapping with Non-Linear Factor Recovery," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [31] N. Demmel, C. Sommer, D. Cremers, et al., "Square Root Bundle Adjustment for Large-Scale Reconstruction," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [32] D. M. Rosen, L. Carlone, A. S. Bandeira, et al., "SE-Sync: A Certifiably Correct Algorithm for Synchronization over the Special Euclidean Group," *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019, mức tin A.
- [33] R. Hartley, J. Trunpf, Y. Dai, et al., "Rotation Averaging," *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.

- [34] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [35] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [36] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.
- [37] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012, mức tin A.
- [38] D. Nistér and H. Stewénus, “Scalable Recognition with a Vocabulary Tree,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2006.
- [39] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018, mức tin A.
- [40] M. J. Milford and G. F. Wyeth, “SeqSLAM: Visual Route-Based Navigation for Sunny Summer Days and Stormy Winter Nights,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [41] Y. Latif, C. Cadena, and J. Neira, “Robust Loop Closing over Time for Pose Graph SLAM,” *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013, mức tin A; thuật toán RRR.
- [42] N. Sünderhauf and P. Protzel, “Switchable Constraints for Robust Pose Graph SLAM,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2012.
- [43] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., “Robust Map Optimization Using Dynamic Covariance Scaling,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2013.
- [44] H. Yang, P. Antonante, V. Tzoumas, et al., “Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [45] M. J. Black and A. Rangarajan, “On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision,” *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [46] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [47] E. Brachmann and C. Rother, “Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022, mức tin B.
- [48] E. Brachmann, T. Cavallari, and V. A. Prisacariu, “Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2023.
- [49] A. Kendall, M. Grimes, and R. Cipolla, “PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2015.
- [50] T. Sattler, Q. Zhou, M. Pollefeys, et al., “Understanding the Limitations of CNN-Based Absolute Camera Pose Regression,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của khẳng định APR không tốt hơn image retrieval, 2019.

Khởi tạo (Initialization)

PHỤ THUỘC A/01-hình-hoc-da-thi (E so với H , bốn cấu hình suy biến — chương này là ứng dụng trực tiếp nhất của mục 6 ở đó); A/05-observability-gauge-va-suy-bien (điều kiện kích thích, và nguyên tắc từ chối khi chưa đủ); B/07-dac-trung-va-data-association (tương ứng đầu vào); B/09-paradigm-uoc-luong-back-end (mỗi kiến trúc có yêu cầu khởi tạo khác nhau).

TÓM TẮT

Mọi bộ tối ưu phi tuyến cần một điểm khởi đầu đủ gần, và SLAM bắt đầu từ chỗ không biết gì. Chương này về khoảnh khắc chuyển từ “không biết gì” sang “có một ước lượng đủ tốt để Gauss-Newton hội tụ”. Nội dung có ba phần. Một: khởi tạo mono hai khung — bài toán khó nhất — với kỹ thuật model selection chạy song song E và H rồi để dữ liệu quyết định; đây là chỗ bốn cấu hình suy biến ở A/01 chuyển từ lý thuyết thành mã. Hai: các trường hợp dễ hơn — stereo, RGB-D, thị giác-quán tính, prior học được — và điều đáng chú ý là mỗi trường hợp dễ hơn *vì cùng một lý do*: có thêm một nguồn thông tin phá vỡ một gauge. Ba: phần quan trọng nhất và bị bỏ qua nhiều nhất, là *tiêu chí từ chối* và cơ chế phát hiện khởi tạo hỏng. Nếu chỉ nhớ một điều: *một bộ khởi tạo tốt không phải bộ luôn trả về kết quả mà là bộ biết từ chối khi dữ liệu chưa đủ* — vì một khởi tạo sai không tự sửa mà nhiễm độc toàn bộ bản đồ về sau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **khởi tạo (initialization)** — quá trình tạo ra ước lượng đầu tiên của tư thế và cấu trúc từ chỗ không có tiên nghiệm nào. Khác với tracking ở chỗ tracking luôn có một dự đoán từ khung trước.
- **model selection** — chạy song song nhiều mô hình hình học (E và H), tính điểm số cho mỗi mô hình dựa trên phần dư của inlier, rồi chọn mô hình thắng. Cần thiết vì hai mô hình suy biến ở những chỗ bù nhau.
- **parallax** — góc giữa hai tia nhìn tới cùng một điểm từ hai vị trí camera. Đây là đại lượng quyết định độ sâu có xác định được không, và là tiêu chí đúng để đánh giá một cặp khung khởi tạo.
- **bootstrapping** — giai đoạn trước khi khởi tạo thành công, trong đó hệ thống thu thập khung và theo dõi đặc trưng nhưng chưa có bản đồ 3D nào.
- **khởi tạo thị giác-quán tính** — quá trình giải cho hướng trọng lực, tỉ lệ metric, vận tốc ban đầu và độ lệch con quay bằng cách căn chỉnh quỹ đạo thị giác (không có tỉ lệ) với tiền tích phân IMU. Chi tiết ở ../IMU_Fusion/ chương 10.
- **khởi tạo lại (re-initialization)** — quá trình bắt đầu lại sau khi hệ thống mất bám hoàn toàn. Khác với relocalization ở chỗ relocalization tìm lại vị trí *trong bản đồ cũ*, còn khởi tạo lại tạo một bản đồ mới.

CÂU HỎI MỞ ĐẦU

- Q1. Bộ khởi tạo của bạn thành công 100% số lần được gọi — nó chưa bao giờ từ chối. Vì sao đó là một tin xấu chứ không phải tin tốt?
- Q2. ORB-SLAM chạy song song hai mô hình E và H rồi chọn. Vì sao không đơn giản là luôn dùng thuật toán năm điểm, vốn không suy biến với cảnh phẳng?

- Q3.** Khởi tạo stereo gần như tầm thường: có độ sâu ngay từ khung đầu. Nêu chính xác điều gì stereo cung cấp mà mono không có, và nói điều đó liên hệ thế nào với A/05.
- Q4.** Một khởi tạo sai với tỉ lệ lệch 20% nhưng hình học tự nhất quán. Bundle adjustment chạy sau đó sẽ sửa được không, và vì sao?
- Q5.** Hệ thống của bạn khởi tạo tốt trong phòng thí nghiệm và thất bại ở hành lang khách hàng. Nêu ba nguyên nhân hình học khác nhau, và nêu đại lượng phân biệt chúng.

4.1 Đặt vấn đề

A/04 dựng một bộ giải mạnh và nêu một điều kiện mà nó không tự thoả được: Gauss-Newton cần một điểm khởi đầu đủ gần nghiệm. Với bài toán không lồi như BA, “đủ gần” không phải một hình thức — xa quá thì bộ giải hội tụ về một cực tiểu địa phương hoàn toàn sai, và nó làm điều đó một cách hoàn toàn tự tin.

Bài toán gốc: hệ thống vừa được bật lên. Nó có một chuỗi ảnh và không có gì khác — không tư thế, không bản đồ, không biết mình đang ở đâu hay cảnh trông thế nào. Từ chỗ đó, phải tạo ra một ước lượng đủ tốt.

Điều làm chương này quan trọng hơn về ngoài là *hậu quả của việc làm sai không tự giới hạn*. Một khung tracking sai sẽ được sửa ở khung sau. Một khởi tạo sai thì không: bản đồ ban đầu trở thành hệ quy chiếu cho mọi thứ sau đó, và một bản đồ khởi tạo méo sẽ làm mọi landmark thêm vào sau bị méo theo cho khớp với nó. Hệ thống hội tụ, phần dư nhỏ, và toàn bộ bản đồ sai. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: BA *không* sửa được, vì BA tìm cầu hình nhất quán nhất với dữ liệu, và một bản đồ méo với tỉ lệ sai là nhất quán — đó chính là gauge tỉ lệ ở A/05 mục 3.

Ai gặp bài toán này? Mọi hệ SLAM, và nó gay gắt nhất với mono. Với stereo hay RGB-D thì gần như tầm thường, và *vì sao* nó tầm thường là một bài học đáng rút ra chứ không phải một may mắn.

Trước khi có model selection, cách làm phổ biến là chọn sẵn một mô hình — thường là E — và chờ tới khi nó cho kết quả trông hợp lý. Cách đó hỏng trong nhà, nơi cảnh phẳng là quy luật chứ không phải ngoại lệ: camera nhìn vào tường, sàn, trần, cửa. Với cảnh phẳng, E không xác định duy nhất (A/01 mục 6B), và nghiệm trả về là một tổ hợp tùy tiện do nhiễu quyết định — nhưng số inlier vẫn cao, nên “trông hợp lý”.

Đó là lý do sâu xa để chạy hai mô hình. Và có một lợi ích thứ hai ít được nhắc nhưng quan trọng hơn cho sản phẩm: khi H thắng, hệ thống *biết* nó đang nhìn một mặt phẳng hoặc đang xoay thuần, và nó có thể đổi *hành vi* — chờ thêm, báo cho người dùng di chuyển — chứ không chỉ đổi công thức.

TRỰC GIÁC

Hình dung bạn tỉnh dậy trong một căn phòng lạ và phải vẽ bản đồ của nó chỉ bằng cách nhìn.

Nhìn một lần không cho biết gì về chiều sâu — mọi thứ có thể là một bức tranh tường phẳng. Bạn phải *đi vài bước* rồi nhìn lại, và so sánh: thứ gì dịch nhiều thì gần, thứ gì gần như đứng im thì xa.

Ba chuyện có thể làm hỏng việc này, và biết ba chuyện đó là biết cả chương.

Bạn xoay đầu thay vì bước đi. Cảnh đổi hoàn toàn, nhưng không có gì dịch tương đối với nhau, nên bạn không học được gì về chiều sâu. Tệ hơn, bạn có cảm giác đã quan sát rất nhiều.

Bạn đang nhìn vào một bức tường phẳng. Bạn bước đi, mọi thứ dịch — nhưng chúng dịch giống hệt nhau, vì chúng ở cùng khoảng cách. Bạn không đọc được gì từ sự khác biệt, vì không có khác biệt nào.

Bạn đi thẳng về phía thứ mình đang nhìn. Nó to dần lên nhưng không lệch sang bên. Bạn có một chút thông tin, rất ít, và ước lượng của bạn chậm chạp.

Trong cả ba trường hợp, một người khôn ngoan sẽ nói “khoan đã, tôi chưa đủ dữ liệu” rồi bước sang bên vài bước. Một cái máy được lập trình kém sẽ vẽ ra một bản đồ, vì người ta bảo nó phải vẽ ra một bản đồ.

Toàn bộ chương này là về việc dạy cái máy nói câu “khoan đã”.

4.2 Khởi tạo mono: bài toán khó nhất

A. Quy trình

Bốn bước, và bước thứ tư mới là bước phân biệt một bộ khởi tạo tốt với một bộ tồi.

Bước một — thu thập và theo dõi. Chọn một khung tham chiếu, theo dõi đặc trưng qua các khung sau (B/07). Tích lũy cho tới khi có đủ tương ứng và đủ parallax.

Bước hai — chạy song song hai mô hình. Với cùng tập tương ứng, chạy RANSAC cho E (hoặc F) và cho H . Mỗi mô hình cho một tập inlier và một tổng phần dư.

Bước ba — chọn mô hình. Tính một điểm số cho mỗi mô hình phản ánh cả số inlier lẫn độ khớp, rồi chọn. ORB-SLAM [12] dùng một tỉ số điểm số; tôi không chép con số ngưỡng cụ thể của họ vào đây, vì ngưỡng ấy phụ thuộc vào thang phần dư, vào ngưỡng RANSAC, và vào loại đặc trưng — chép nó sang một hệ khác mà không hiểu là cách nhanh nhất để có một hằng số kỳ bí. Nguyên tắc thì chép được: nếu H khớp tốt hơn đáng kể thì cảnh phẳng hoặc xoay thuận, và trong cả hai trường hợp bạn chưa nên khởi tạo bằng E .

Bước bốn — kiểm tra và quyết định. Phân rã mô hình thẳng, triangulate, rồi kiểm kết quả trước khi chấp nhận. Đây là mục 4.

B. Vì sao không chỉ dùng thuật toán năm điểm

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và câu trả lời có ba tầng.

Tầng một: thuật toán năm điểm không suy biến với cảnh phẳng *theo nghĩa nó vẫn trả về một E xác định*. Nhưng nó không giải quyết được vấn đề thật — với cảnh phẳng, *bài toán* có nhiều nghiệm, không phải *thuật toán* có nhiều nghiệm. Thuật toán năm điểm chọn một nghiệm trong họ đó dựa trên năm điểm được lấy mẫu, và nghiệm ấy nhảy với nhiều theo cách không kiểm soát được.

Tầng hai, và đây là tầng quan trọng: với cảnh *gần* phẳng — một bức tường có vài vật nhô ra — bài toán không suy biến hoàn toàn nhưng có số điều kiện rất lớn. Đây là suy biến số học ở A/05 mục 2C, và đúng kiểu khó chẩn đoán nhất. Homography cho một cách *đo* mức độ phẳng: nếu H khớp gần như hoàn hảo thì cảnh gần phẳng, và ta biết mình đang ở vùng nguy hiểm.

Tầng ba, thực dụng nhất: khi H thẳng, ta *vẫn khởi tạo được* bằng cách phân rã H — nó cho $(R, \mathbf{t}/d, \mathbf{n})$, tức tư thế tương đối và pháp tuyến mặt phẳng, với tỉ lệ tương đối so với khoảng cách tới mặt phẳng. Đó là một khởi tạo hợp lệ cho cảnh phẳng, và nó tốt hơn nhiều so với việc ép E hoạt động. Nói cách khác, model selection không chỉ là một bộ phát hiện suy biến mà còn là một bộ chọn *thuật toán phù hợp*.

C. Tiêu chí chọn cặp khung

Đây là chỗ A/01 mục 6 trả công trực tiếp nhất.

Cần bao nhiêu parallax? Không có ngưỡng phổ quát, vì nó phụ thuộc vào σ của front-end (B/07 mục 4B) và vào độ sâu cảnh. Nhưng có một *cách tính* ngưỡng thay vì đoán nó, và cách ấy đáng biết.

Từ A/06, bất định độ sâu của một điểm triangulate với góc parallax α nhỏ tỉ lệ xấp xỉ $\sigma_z/z \approx \sigma_\theta/\alpha$, với $\sigma_\theta \approx \sigma_{px}/f$ là bất định góc của một quan sát. Đặt yêu cầu $\sigma_z/z \leq \epsilon$ — chẳng hạn $\epsilon = 0,1$ cho 10% sai số độ sâu tương đối — ta được

$$\alpha \geq \frac{\sigma_{px}}{f \epsilon}.$$

Với $\sigma_{px} = 0,2$, $f = 400$, $\epsilon = 0,1$: $\alpha \geq 0,005 \text{ rad} \approx 0,29^\circ$. (Tôi tự dẫn quan hệ này từ hình học tam giác nhỏ; chưa đối chiếu một nguồn cụ thể, và nó là xấp xỉ bậc nhất.)

Điều đáng nói không phải con số mà là *phương pháp*: ngưỡng parallax nên được *tính ra* từ σ đã đo và từ yêu cầu độ chính xác của ứng dụng, chứ không nên là một hằng số chép từ mã nguồn của người khác. Đây là một ví dụ nhỏ của nguyên tắc lớn hơn ở D/21: mỗi hằng số trong hệ thống nên truy được về một phép đo hoặc một yêu cầu.

Hai tiêu chí nữa. *Số lượng tương ứng*: cần đủ để RANSAC có ý nghĩa thống kê, và đủ để bản đồ ban đầu không quá nghèo. *Phân bố*: các điểm khởi tạo nên trải đều khung hình, vì lý do ở B/07 mục 4A — điều kiện số của bài toán khởi tạo cũng chịu ảnh hưởng như mọi bài toán khác.

I 4.3 Các trường hợp dễ hơn, và vì sao chúng dễ hơn

Điều đáng chú ý về mục này: mọi trường hợp dễ hơn đều dễ hơn vì *cùng một lý do*, và nhận ra điều đó là hiểu được cả chương.

A. Stereo và RGB-D

Với stereo đã hiệu chỉnh, khung đầu tiên đã cho một đám mây điểm có đơn vị mét: khớp dọc hàng, tính disparity, lấy $z = bf/d$. Không cần chuyển động, không cần parallax theo thời gian, không cần model selection. Hệ thống có bản đồ ngay ở khung một.

Vì sao dễ như vậy? Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Theo A/05 mục 3B, đường cơ sở stereo là một *độ dài đã biết bằng mét*, nên nó phá vỡ gauge tỉ lệ ngay lập tức. Bảy bậc tự do của mono rút xuống sáu, và bậc bị mất chính là bậc khó nhất — bậc mà mono cần chuyển động để lấy. Nói khác đi: stereo mua thông tin bằng *phân cứng* thay vì bằng *chuyển động*.

Đây là một trong những lập luận mạnh nhất cho stereo trong sản phẩm, và nó lớn hơn lợi ích “có độ sâu trực tiếp” thường được nêu. Cả một lớp chế độ hồng — toàn bộ mục 2 của chương này, cộng bốn cấu hình suy biến ở A/01 — biến mất. Với robot phải khởi động ở một vị trí bất kỳ do khách hàng đặt, trong một môi trường mà ta không kiểm soát được, đó là khác biệt giữa dùng được và không.

Cái giá, đã nêu ở A/02 mục 5C: đường cơ sở phải giữ ổn định, và lợi ích tắt dần theo bình phương khoảng cách. Ngoài tầm hữu ích của stereo, ta trở lại chính bài toán mono — nên hệ stereo vẫn cần logic của mục 2, chỉ là ít khi phải dùng tới.

B. Thị giác-quán tính

Khởi tạo VI giải đồng thời cho hướng trọng lực, tỉ lệ metric, vận tốc ban đầu và độ lệch con quay, bằng cách căn chỉnh một quỹ đạo thị giác không có tỉ lệ với tiền tích phân IMU. VINS-Mono làm theo cách tuyến tính từng bước [10]; ORB-SLAM3 làm bằng một bài toán MAP thuần quán tính [16]. Chi tiết đầy đủ nằm ở `../IMU_Fusion/` chương 10 và không lặp lại ở đây.

Điều cần nhớ cho chương này là điều kiện: *gia tốc phải thay đổi*. Nếu gia tốc thật là hằng — đứng yên, hoặc chuyển động thẳng đều — thì thành phần gia tốc nhập hoàn toàn vào vectơ trọng lực chưa biết, và tỉ lệ không quan sát được dù có IMU. Đây là cùng nguyên lý ở A/05: cảm biến cho thông tin chỉ khi chuyển động kích thích nó.

Hệ quả thực hành đáng nhớ: một hệ VI khởi tạo trên một drone đang đậu trên bàn sẽ thất bại, và nó thất bại một cách *im lặng* nếu không có kiểm tra kích thích. Đây là ví dụ mở đầu của `../IMU_Fusion/` chương 5, và nó là lý do mọi bộ khởi tạo VI nghiêm túc đều có một cổng kiểm tra phương sai gia tốc.

C. Prior độ sâu học được

Một mô hình độ sâu metric đơn ảnh cho một ước lượng độ sâu ngay từ khung đầu, nên về nguyên tắc nó chơi vai trò của stereo. Điều này hữu ích và ngày càng được dùng.

Nhưng theo B/08 mục 4C, phải cẩn thận về cách đưa vào. Dùng đúng: *khởi tạo* tỉ lệ và độ sâu từ mô hình, rồi để hình học tiếp quản khi đã có đủ parallax. Dùng sai: giữ prior với trọng số cao mãi mãi, lúc đó một sai lệch có hệ thống của mô hình — và mọi mô hình đều có, phụ thuộc vào loại cảnh — sẽ chảy vào toàn bộ bản đồ.

Có một lợi ích thứ hai ít được nhắc và có thể quan trọng hơn: mô hình học được cho một ước lượng *ngay lập tức*, nên hệ thống có thể bắt đầu tracking trong khi chờ đủ parallax. Điều này biến giai đoạn bootstrapping từ “mù hoàn toàn” thành “ước lượng thô”, và với trải nghiệm người dùng thì đó là khác biệt lớn.

Bảng 4.1. Bốn cấu hình và cái mà mỗi cấu hình cung cấp. Cột thứ hai là cột giải thích tất cả: mỗi trường hợp dễ hơn vì có một nguồn phá vỡ gauge tỉ lệ.

Cấu hình	Nguồn phá gauge tỉ lệ	Điều kiện	Chế độ hỏng còn lại
Mono	Không có — tỉ lệ là gauge tự do	Cần parallax; cần cảnh không phẳng	Cả bốn cấu hình ở A/01 mục 6
Stereo	Đường cơ sở đã biết bằng mét	Không có — khung đầu là đủ	Ngoài tầm hữu ích thì thoái hoá về mono
RGB-D	Cảm biến độ sâu	Trong tầm và bề mặt hợp tác	Kính, gương, bề mặt đen, ngoài trời
Thị giác-quán tính	Gia tốc kế (qua tích phân)	Gia tốc phải thay đổi	Đứng yên hoặc thẳng đều thì thất bại im lặng
Prior học được	Tiên nghiệm về kích thước vật thể	Cảnh nằm trong phân bố huấn luyện	Sai lệch có hệ thống, không có bất định đã hiệu chuẩn

4.4 Tiêu chí từ chối: phần quan trọng nhất

A. Vì sao từ chối quan trọng hơn thành công

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là luận điểm trung tâm của chương.

Một bộ khởi tạo có hai loại lỗi. *Từ chối khi lẽ ra chấp nhận được*: hệ thống chờ thêm vài khung. Chi phí là một chút độ trễ, và người dùng thấy hệ thống “hơi chậm khởi động”. *Chấp nhận khi lẽ ra phải từ chối*: hệ thống tạo ra một bản đồ sai, và mọi thứ sau đó xây trên bản đồ ấy. Chi phí là một quỹ đạo sai không tự sửa, và thường là mất bám hoàn toàn sau vài chục khung — nhưng đôi khi tệ hơn, là một quỹ đạo trông hợp lý mà sai tỉ lệ.

Hai chi phí ấy chênh nhau nhiều bậc độ lớn. Nên ngưỡng chấp nhận nên đặt *bảo thủ*, và một bộ khởi tạo từ chối thường xuyên là dấu hiệu của thiết kế đúng chứ không phải thiết kế kém. Một hệ chưa bao giờ từ chối thì hoặc nó chỉ chạy trong điều kiện lý tưởng, hoặc nó đang chấp nhận những thứ lẽ ra không nên — và bạn không biết là cái nào.

Nguyên tắc này lặp lại ở B/12 với relocalization, nơi chênh lệch còn lớn hơn, và ở B/11 với loop closure, nơi nó lớn nhất.

B. Bốn phép kiểm

Kiểm một — *parallax trung vị*. Không phải trung bình: trung vị bền hơn với vài điểm ở rất gần. Ngưỡng tính theo mục 2C từ σ đã đo. Đây là phép kiểm bắt được xoay thuần và chuyển động quá nhỏ.

Kiểm hai — *tỉ số điểm số giữa H và E*. Bắt được cảnh phẳng. Khi H thắng rõ rệt, hoặc chuyển sang khởi tạo bằng phân rã H, hoặc chờ thêm.

Kiểm ba — *tỉ lệ cheirality*. Sau khi phân rã và triangulate, đếm tỉ lệ điểm có độ sâu dương ở cả hai khung. Với nghiệm đúng, tỉ lệ này phải áp đảo. Nếu hai trong bốn nghiệm cho tỉ lệ xấp xỉ nhau thì đó *không* phải lúc chọn cái nhiều phiếu hơn — đó là dấu hiệu parallax quá nhỏ (A/01 mục 3E), và câu trả lời đúng là từ chối.

Kiểm bốn — *số điều kiện hoặc phổ giá trị riêng của Λ* . Sau khi dựng bài toán BA nhỏ cho hai khung, kiểm xem có bao nhiêu hướng gần suy biến ngoài bảy bậc gauge. Đắt hơn ba phép trên nhưng là phép kiểm tổng quát nhất — nó bắt được cả những trường hợp mà ba phép kia bỏ sót. A/05 khối *Cạm bẫy* liệt kê đầy đủ.

CẠM BÃY

Số inlier không nằm trong bốn phép kiểm, và đó là có chủ ý.

Như A/01 mục 6 đã lập luận, trong cả bốn cấu hình suy biến, số inlier vẫn cao. Nó đo mức nhất quán của dữ liệu với mô hình, không đo mức xác định của mô hình. Dùng nó làm tiêu chí chấp nhận là đúng loại nhầm lẫn mà A/05 mô tả.

Một biểu hiện cụ thể và rất hay gặp: hệ thống khởi tạo “thành công” với năm trăm inlier khi camera đang xoay tại chỗ trước một bức tường có vân. Mọi điểm khớp hoàn hảo — vì chúng thật sự khớp. Bản đồ tạo ra có mọi điểm ở những độ sâu ngẫu nhiên, và nó hoàn toàn nhất quán với dữ liệu.

Số inlier *vẫn* hữu ích, nhưng như một *điều kiện cần*, không phải điều kiện đủ: quá ít inlier thì chắc chắn không khởi tạo được, nhưng nhiều inlier không nói gì về việc có nên khởi tạo hay không.

C. Phát hiện khởi tạo hỏng sau khi đã chấp nhận

Bốn phép kiểm trên chạy *trước* khi chấp nhận. Cần thêm một tầng chạy *sau*, vì không bộ kiểm nào hoàn hảo.

Ba dấu hiệu cho thấy bản đồ ban đầu có vấn đề, quan sát được trong vài chục khung đầu. *Tỉ lệ tracking sụt nhanh*: nếu bản đồ ban đầu sai, các landmark không chiếu về đúng chỗ ở khung sau và bị loại dần. *Phần dư trung bình tăng dần* thay vì ổn định. *Nhiều landmark bị loại vì cheirality* — điểm rơi ra sau camera khi camera di chuyển, dấu hiệu độ sâu ban đầu sai.

Khi phát hiện, hành động đúng là *vứt bản đồ và khởi tạo lại* chứ không phải cố cứu. Đây là một quyết định kiến trúc quan trọng: hệ thống phải có khả năng vứt bỏ trạng thái của chính nó và bắt đầu lại sạch sẽ. Nhiều hệ không có khả năng đó và phải khởi động lại cả tiến trình — chấp nhận được trong nghiên cứu, không chấp nhận được trong sản phẩm (D/21).

ORB-SLAM3 đi xa hơn với kiến trúc Atlas: khi mất bám, nó bắt đầu một bản đồ *mới* và giữ bản đồ cũ; nếu sau này phát hiện hai bản đồ chồng lấn, nó ghép chúng [16]. Đây là cách xử lý tình tế hơn nhiều so với vứt đi, và nó là chủ đề của C/13.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [16]	Model selection H so với E ; từ chối khi parallax không đủ; Atlas cho nhiều bản đồ	Cài đặt đầy đủ nhất của mục 2 và mục 4C
VINS-Mono [10]	SfM thuần thị giác rồi căn với IMU; kiểm phương sai gia tốc trước khi khởi tạo	Mục 3B — và công kiểm tra kích thích là một ví dụ tốt
DSO [22]	Khởi tạo direct coarse-to-fine với độ sâu ngẫu nhiên ban đầu	Cho thấy direct cần chiến lược khác hẳn: không có tương ứng rời rạc để dùng E hay H
SVO-Pro [19]	Khởi tạo bằng homography cho cảnh phẳng (giả định mặt đất)	Ví dụ về việc dùng tiên nghiệm ứng dụng — hợp lệ khi tiên nghiệm đúng
COLMAP [17]	Chọn cặp khung khởi tạo theo tiêu chí hình học, thử nhiều cặp	Ngoại tuyến nên có thể thử lại — một xa xỉ mà hệ thời gian thực không có

MINI PROJECT SLAM • Xây một bộ khởi tạo biết từ chối, rồi đo tỉ lệ từ chối đúng và sai • 3 ngày

Mục tiêu	biến “khởi tạo” từ một bước bạn hy vọng thành công thành một bước có tiêu chí đo được và có tỉ lệ lỗi biết trước.
Dữ liệu	một chuỗi EuRoC hoặc TUM-RGBD có chân trị, cộng <i>ba chuỗi tự quay</i> : một chuỗi xoay thuần, một chuỗi nhìn tường phẳng, một chuỗi đi thẳng về phía trước. Ba chuỗi sau là phần quan trọng nhất của bài này.
Các bước	cài khởi tạo mono hai khung với model selection; cài cả bốn phép kiểm ở mục 4B; chạy trên cả bốn chuỗi và ghi lại quyết định (chấp nhận/từ chối) kèm giá trị của bốn đại lượng kiểm; với các lần chấp nhận trên chuỗi có chân trị, đo sai số tỉ lệ và sai số tư thế tương đối; vẽ đường cong ROC — tỉ lệ chấp nhận sai so với tỉ lệ từ chối sai — khi quét ngưỡng parallax.
Xong khi	bạn có một đường cong ROC và một ngưỡng parallax chọn từ đường cong đó chứ không chép từ đâu. Bạn phải quan sát được rằng trên ba chuỗi suy biến, số inlier vẫn cao trong khi ba đại lượng kiểm kia thì không — xác nhận khối <i>Cạm bẫy</i> bằng dữ liệu của chính bạn. Và bạn có một con số: tỉ lệ chấp nhận sai của hệ thống bạn ở ngưỡng đã chọn.
Nối với	B/12-relocalization dùng cùng nguyên tắc từ chối với chênh lệch chi phí còn lớn hơn; D/21-tu-prototype-len-mass-production biến tỉ lệ chấp nhận sai thành một chỉ tiêu nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì một bộ khởi tạo có hai loại lỗi với chi phí chênh nhau nhiều bậc. Từ chối sai chỉ tốn một chút độ trễ; chấp nhận sai tạo ra một bản đồ sai mà mọi thứ sau đó xây trên nó, và nó *không tự sửa*. Một hệ chưa bao giờ từ chối thì hoặc chỉ chạy trong điều kiện lý tưởng, hoặc đang chấp nhận những thứ lẽ ra không nên — và bạn không biết là cái nào, tức bạn đang bay mù. Thiết kế đúng là đặt ngưỡng bảo thủ và *đo* tỉ lệ chấp nhận sai, chứ không phải tối đa hoá tỉ lệ thành công.
- A2. Ba tầng. (a) Thuật toán năm điểm vẫn trả về một E xác định với cảnh phẳng, nhưng vấn đề là *bài toán* có nhiều nghiệm chứ không phải thuật toán — nghiệm nó chọn nhảy với nhiễu theo cách không kiểm soát được. (b) Với cảnh *gần* phẳng, bài toán có số điều kiện rất lớn — suy biến số học, khó chẩn đoán nhất — và homography cho một cách *đo* mức độ phẳng. (c) Khi H thẳng, ta vẫn khởi tạo được bằng phân rã H , vốn cho $(R, t/d, n)$ và là một khởi tạo hợp lệ cho cảnh phẳng. Nên model selection không chỉ phát hiện suy biến mà còn chọn thuật toán phù hợp.
- A3. Stereo cung cấp một *độ dài đã biết bằng mét* — đường cơ sở. Theo A/05 mục 3B, nó phá vỡ gauge tỉ lệ ngay lập tức, đưa bảy bậc tự do của mono xuống sáu, và bậc bị mất chính là bậc khó nhất — bậc mà mono phải dùng chuyển động để lấy. Nói khác đi: stereo mua thông tin bằng *phần cứng* thay vì bằng *chuyển động*. Hệ quả là cả một lớp chế độ hồng biến mất. Cái giá: đường cơ sở phải ổn định, và lợi ích tắt dần theo bình phương khoảng cách — ngoài tầm hữu ích thì ta trở lại đúng bài toán mono.
- A4. Không, BA *không* sửa được, và lý do là A/05: tỉ lệ là một gauge tự do của SLAM mono. BA tìm cấu hình nhất quán nhất với dữ liệu, và một bản đồ cơ giản đều với tỉ lệ sai là hoàn toàn nhất quán — mọi phần dư chiếu đều bằng đúng như với tỉ lệ đúng. Hàm mục tiêu phẳng theo hướng đó, nên BA không có tín hiệu nào để sửa. Muốn sửa thì phải *thêm thông tin*: một cảm biến có đơn vị mét, một vật có kích thước đã biết, hoặc một loop closure $Sim(3)$ nối về một phần bản đồ có tỉ lệ đúng.
- A5. Ba nguyên nhân: (a) *xoay thuận* — robot quay để vào hành lang, phát hiện bằng parallax trung vị thấp dù số inlier cao; (b) *cảnh phẳng* — camera nhìn vào tường cuối hành lang, phát hiện bằng tỉ số điểm số H thẳng E ; (c) *đi dọc trục quang* — robot đi thẳng dọc hành lang, epipole nằm trong ảnh, phát hiện bằng phân bố parallax rất lệch theo bán kính và bằng số điều kiện lớn. Ba đại lượng phân biệt chúng là ba đại lượng kiểm khác nhau ở mục 4B, và không cái nào là số inlier. Lưu ý thêm: hành lang thường có *cả ba* cùng lúc, và đó là lý do nó là môi trường khó nhất cho SLAM mono.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn bị bịt mắt đưa vào một căn phòng lạ, rồi tháo băng ra và bảo vẽ bản đồ căn phòng. Nhìn một lần không đủ. Một bức tranh vẽ cảnh sâu hun hút và một cái hành lang thật, từ một chỗ đứng, trông y hệt nhau. Bạn phải bước sang bên vài bước rồi nhìn lại: thứ gì dịch nhiều là gần, thứ gì gần như đứng im là xa.

Ba chuyện có thể làm hỏng việc này. Bạn *quay người* thay vì bước đi — cảnh đổi hoàn toàn nhưng chẳng học được gì. Bạn đang nhìn vào một *bức tường phẳng* — bước đi thì mọi thứ dịch, nhưng dịch giống hệt nhau nên chẳng đọc được gì từ sự khác biệt. Hoặc bạn *đi thẳng* về phía đang nhìn — mọi thứ to dần chứ không lệch sang bên, nên bạn chỉ có rất ít thông tin.

Điều mà một người tỉnh táo làm trong cả ba trường hợp: nói “khoan đã”, rồi bước sang bên vài bước nữa.

Điều mà một cái máy lập trình kém làm: vẽ ra một bản đồ. Vì người ta bảo nó phải vẽ ra một bản đồ, và nó không có khái niệm “chưa đủ dữ liệu”.

Và đây là chỗ nguy hiểm: bản đồ sai ấy không tự hỏng. Mọi thứ máy nhìn thấy về sau sẽ được đặt vào cho *khớp* với bản đồ sai ban đầu, nên bản đồ vẫn luôn tự nhất quán. Nó chỉ không giống căn phòng thật.

Nên phần khó nhất của chương này không phải cách vẽ bản đồ. Đó là cách dạy cái máy nhận ra lúc nó chưa nên vẽ — và chịu đựng việc phải chờ.

Chỗ không rút gọn được: chỗ không rút gọn được là *ngưỡng*: bước sang bên bao nhiêu thì đủ? Câu trả lời phụ thuộc vào camera của bạn chính xác tới đâu, cảnh xa hay gần, và bạn cần bản đồ chính xác

tới mức nào. Mục 2C cho một cách *tính* nó thay vì đoán, và đó là cách đúng — nhưng nó đòi bạn phải biết σ của chính mình, và đó là một phép đo mà ít ai làm.

Một câu: Một bộ khởi tạo tốt là bộ biết nói “chưa đủ dữ liệu” — vì cái nó tạo ra sẽ là hệ quy chiếu cho mọi thứ về sau.

4.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *khởi tạo sai không tự sửa được* — là hệ quả của A/05: tỉ lệ là gauge tự do nên BA không có tín hiệu để sửa nó. Nó bị bác bỏ nếu ai đó cho thấy một hệ mono thuần thị giác phục hồi được tỉ lệ đúng sau một khởi tạo sai tỉ lệ, mà không dùng thêm thông tin nào. Điều đó bất khả theo lý thuyết; điều *có thể* xảy ra và cần phân biệt là hệ phục hồi được sau một khởi tạo sai về *hình học* (không phải tỉ lệ) — chẳng hạn khi các landmark sai bị loại dần và thay bằng landmark mới đúng. Đó là một cơ chế thật và nó làm mệnh đề cần được phát biểu chặt hơn: *sai tỉ lệ không sửa được; sai hình học thì có thể, nếu hệ có cơ chế loại landmark tốt*.

Mệnh đề thứ hai — *số inlier không phải tiêu chí chấp nhận* — là mệnh đề kế thừa từ A/01 mục 6 và kiểm được bằng mini project (cửa b). Nó bị bác bỏ nếu trên một tập đủ lớn các lần khởi tạo gồm cả cấu hình suy biến, số inlier tương quan chặt với sai số tỉ lệ thật — chặt ngang hoặc hơn parallax trung vị.

Mệnh đề thứ ba — *ngưỡng parallax tính được từ σ và yêu cầu độ chính xác* — dựa trên một xấp xỉ bậc nhất mà tôi tự dẫn ở mục 2C và *chưa đối chiếu nguồn*. Nó có thể sai ở chỗ: quan hệ $\sigma_z/z \approx \sigma_\theta/\alpha$ bỏ qua ảnh hưởng của số lượng quan sát (nhiều quan sát bù được parallax nhỏ ở mức độ nào đó) và bỏ qua hình dạng phân bố lệch ở A/06. Một công thức đúng hơn nên tính từ ma trận thông tin đầy đủ. Đây là một mục trong 99-chua-biet.md.

Một mệnh đề mềm: chương này khẳng định *tỉ lệ từ chối cao là dấu hiệu thiết kế đúng*. Điều đó có thể sai trong ứng dụng mà độ trễ khởi động là yêu cầu cứng — một thiết bị AR mà người dùng không chờ quá hai giây. Ở đó, đánh đổi dịch chuyển, và cách đúng là dùng một cấu hình cảm biến khác (stereo, hoặc prior học được) chứ không phải nới ngưỡng. Nêu rõ điều này vì nguyên tắc ở mục 4A không phổ quát mà phụ thuộc vào tỉ số chi phí của hai loại lỗi.

4.6 Kết nối

Nối lên trên. A/01-hình-hoc-da-thi là chương mà chương này áp dụng trực tiếp nhất — mục 6 ở đó liệt kê bốn cấu hình suy biến, và mục 4B ở đây biến chúng thành bốn phép kiểm chạy được. A/05-observability-gauge-va-suy-bien cho nguyên tắc lớn hơn: tỉ lệ là gauge nên không sửa được, và điều kiện kích thích quyết định khi nào được phép tin. A/06-bat-dinh-va-lan-truyen-sai-so cho công cụ tính ngưỡng parallax từ σ . B/07-dac-trung-va-data-association cho σ đó.

Nối xuống dưới. B/11-loop-closure-va-nhat-quan-toan-cuc dùng cùng nguyên tắc từ chối với chênh lệch chi phí còn lớn hơn: một loop closure sai phá cả bản đồ. B/12-relocalization cũng vậy, và cả ba chương — 10, 11, 12 — có chung một cấu trúc lập luận: hai loại lỗi với chi phí bất đối xứng, nên ngưỡng phải bảo thủ và tỉ lệ lỗi phải được đo. C/13-quan-ly-ban-do nhận vấn đề ở mục 4C: hệ thống phải vứt bỏ được bản đồ của chính nó, và Atlas là cách xử lý tinh tế hơn.

Nối xa hơn. C/15-hop-nhat-da-cam-bien là chỗ Bảng 4.1 được đọc như một bảng thiết kế: chọn cảm biến nào chính là chọn phá gauge nào. D/19-ben-vung-trong-the-gioi-thuc bàn về việc khởi tạo trong điều kiện xấu — ít vân, ánh sáng kém, vật động.

D/21-tu-prototype-len-mass-production biến tỉ lệ chấp nhận sai thành một chỉ tiêu nghiệm thu, và lập luận rằng khởi động nhanh và đáng tin là một yêu cầu sản phẩm chứ không phải một tính năng.

Nổi ngang: ../IMU_Fusion/ chương 10 là chương đầy đủ về khởi tạo thị giác-quán tính, với suy dẫn các bước giải cho trọng lực, tỉ lệ và vận tốc. Chương này chỉ nêu kết quả và điều kiện.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao một bộ khởi tạo chưa bao giờ từ chối là một tin xấu? Nêu tỉ số chi phí giữa hai loại lỗi.
2. Nêu ba lý do để chạy song song E và H thay vì chỉ dùng thuật toán năm điểm.
3. Tự dẫn lại ngưỡng parallax từ σ_{px} , f và yêu cầu ϵ , rồi tính cho $\sigma = 0,3$ px, $f = 600$, $\epsilon = 0,05$.
4. Điều gì stereo cung cấp mà mono không có? Trả lời bằng ngôn ngữ gauge, không bằng ngôn ngữ “có độ sâu”.
5. Khởi tạo VI cần gia tốc thay đổi. Vì sao, và chuyện gì xảy ra nếu bỏ qua điều kiện đó?
6. Nêu bốn phép kiểm trước khi chấp nhận khởi tạo, và nói mỗi phép bắt được cấu hình suy biến nào.
7. Bản đồ khởi tạo sai tỉ lệ 20%. Nêu chính xác vì sao BA không sửa được, và nêu ba cách thêm thông tin để sửa.

Nguồn

- [1] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, mức tin A.
- [2] J. Shi and C. Tomasi, “Good Features to Track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 1994.
- [3] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2006.
- [4] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2011.
- [5] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [6] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [7] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [8] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [9] J.-Y. Bouguet, “Pyramidal Implementation of the Affine Lucas Kanade Feature Tracker: Description of the Algorithm,” Intel Corporation, tech. rep., 2001, mức tin B; báo cáo kỹ thuật, là tài liệu tham chiếu de-facto cho KLT thấp.
- [10] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [11] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; nguồn của inverse compositional.
- [12] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [13] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [14] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2005.
- [15] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2020.
- [16] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.

- [17] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [18] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [19] C. Forster, Z. Zhang, M. Gassner, et al., "SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.
- [20] G. Klein and D. Murray, "Parallel Tracking and Mapping for Small AR Workspaces," in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A; công trình tách tracking khỏi mapping thành hai luồng, 2007.
- [21] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," in *European Conference on Computer Vision (ECCV)*, mức tin A, 2014.
- [22] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [23] J. Civera, A. J. Davison, and J. M. M. Montiel, "Inverse Depth Parametrization for Monocular SLAM," *IEEE Transactions on Robotics*, vol. 24, no. 5, pp. 932–945, 2008, mức tin A; nguồn của lập luận vì sao inverse depth tuyến tính hoá tốt hơn.
- [24] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của cách khử landmark bằng chiếu lên null space, 2007.
- [25] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2024.
- [26] L. Piccinelli, Y.-H. Yang, C. Sakaridis, et al., "UniDepth: Universal Monocular Metric Depth Estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [27] A. Rosinol, M. Abate, Y. Chang, et al., "Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [28] M. Kaess, H. Johannsson, R. Roberts, et al., "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [29] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên Sim(3), 2010.
- [30] V. Usenko, N. Demmel, D. Schubert, et al., "Visual-Inertial Mapping with Non-Linear Factor Recovery," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [31] N. Demmel, C. Sommer, D. Cremers, et al., "Square Root Bundle Adjustment for Large-Scale Reconstruction," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [32] D. M. Rosen, L. Carlone, A. S. Bandeira, et al., "SE-Sync: A Certifiably Correct Algorithm for Synchronization over the Special Euclidean Group," *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019, mức tin A.
- [33] R. Hartley, J. Trumpf, Y. Dai, et al., "Rotation Averaging," *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.

- [34] P. Geneva, K. Eickenhoff, W. Lee, et al., "OpenVINS: A Research Platform for Visual-Inertial Estimation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [35] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [36] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.
- [37] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012, mức tin A.
- [38] D. Nistér and H. Stewénus, "Scalable Recognition with a Vocabulary Tree," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2006.
- [39] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018, mức tin A.
- [40] M. J. Milford and G. F. Wyeth, "SeqSLAM: Visual Route-Based Navigation for Sunny Summer Days and Stormy Winter Nights," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [41] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013, mức tin A; thuật toán RRR.
- [42] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2012.
- [43] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2013.
- [44] H. Yang, P. Antonante, V. Tzoumas, et al., "Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [45] M. J. Black and A. Rangarajan, "On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision," *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [46] T. Schneider, M. Dymczyk, M. Fehr, et al., "maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [47] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022, mức tin B.
- [48] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2023.
- [49] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2015.
- [50] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-Based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của khẳng định APR không tốt hơn image retrieval, 2019.

Loop closure và nhất quán toàn cục (Loop Closure and Global Consistency)

PHỤ THUỘC A/03-lie-group-va-toi-uu-tren-da-tap ($Sim(3)$ và log của sai lệch tư thế); A/04-uoc-luong-map-va-bundle-adjustment (M-estimator, GNC); A/05-observability-gauge-va-suy-bien (số bậc tự do của pose graph theo cấu hình cảm biến); B/07-dac-trung-va-data-association (perceptual aliasing — ở đây nó xuất hiện lại với hậu quả nghiêm trọng hơn nhiều); B/09-paradigm-uoc-luong-back-end (pose graph).

TÓM TẮT

Odometry trôi, và trôi đơn điệu — không cơ chế nào trong B/09 tự sửa được điều đó. Loop closure là cơ chế duy nhất: khi hệ thống nhận ra nó đã quay lại một chỗ từng đi qua, nó thêm một ràng buộc nổi hiện tại với quá khứ, và ràng buộc ấy phân phối lại sai số tích lũy trên toàn quỹ đạo. Đây là thứ biến *odometry* thành *SLAM*. Nội dung có bốn phần. Một: nhận ra chỗ cũ — từ túi từ trực quan tới các mô hình học được — và vì sao đây là một bài toán truy hồi chứ không phải một bài toán khớp. Hai: kiểm định hình học, tăng bắt buộc biến một ứng viên thành một ràng buộc, và vì sao ngưỡng của nó phải bảo thủ hơn mọi ngưỡng khác trong hệ thống. Ba: áp dụng ràng buộc — $Sim(3)$ cho mono, pose graph, và việc hoà nhập kết quả với luồng tracking đang chạy. Bốn, và là phần quan trọng nhất: back-end bền, tức làm sao sống sót khi một ràng buộc sai lọt qua. Nếu chỉ nhớ một điều: *đây là chương duy nhất trong cây mà một lỗi đơn lẻ phá huỷ toàn bộ sản phẩm của hàng giờ làm việc* — nên chi phí của hai loại lỗi bất đối xứng tới mức mọi thiết kế phải xoay quanh sự bất đối xứng đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **loop closure** — việc nhận ra rằng vị trí hiện tại trùng với một vị trí đã đi qua, và thêm một ràng buộc tương ứng vào bài toán ước lượng. Gồm hai bước tách rời: *phát hiện* (nhận ra chỗ cũ) và *đóng* (áp dụng ràng buộc).
- **trôi (drift)** — sai số tích lũy của odometry, tăng đơn điệu theo quãng đường đi. Không cơ chế odometry nào tự sửa được trôi, vì nó không có thông tin nào về vị trí tuyệt đối.
- **túi từ trực quan (bag of visual words)** — biểu diễn một ảnh bằng histogram tần suất của các “từ” — các cụm descriptor đã học trước. Cho phép so sánh hai ảnh bằng một vectơ thưa thay vì bằng hàng trăm descriptor.
- **TF-IDF** — trọng số cho mỗi từ: từ xuất hiện nhiều trong ảnh này (term frequency) và hiếm trong toàn bộ kho (inverse document frequency) thì mang nhiều thông tin phân biệt hơn.
- **perceptual aliasing** — hai địa điểm khác nhau trông giống nhau tới mức không phân biệt được bằng vẻ ngoài. Đây là nguyên nhân gốc của loop closure sai, và nó là quy luật chứ không phải ngoại lệ trong môi trường nhân tạo.
- **kiểm định hình học (geometric verification)** — bước kiểm một ứng viên loop closure bằng cách tìm một phép biến đổi nhất quán giữa hai khung. Đây là tầng phòng thủ chính chống lại perceptual aliasing.
- **back-end bền (robust back-end)** — họ kỹ thuật cho phép bài toán tối ưu sống sót khi một số cạnh là sai, bằng cách cho phép bộ giải “tắt” chúng thay vì buộc phải thoả mãn chúng.

CÂU HỎI MỞ ĐẦU

- Q1. Loop closure sai làm gập cả bản đồ; loop closure bỏ lỡ chỉ làm hệ thống trôi thêm một chút. Ước lượng tỉ số chi phí giữa hai loại lỗi, rồi nói nó quyết định gì về thiết kế.
- Q2. Kiểm định hình học đã cho một phép biến đổi với hai trăm inlier. Vì sao đó vẫn chưa đủ để chấp nhận, và nêu phép kiểm bổ sung.
- Q3. Hành lang kho hàng có ba mươi kệ giống hệt nhau, cách nhau ba mét. Mọi phương pháp dựa trên vẽ ngoài đều thất bại ở đây. Nêu hai nguồn thông tin khác vẫn phân biệt được, và nói vì sao chúng không phải lời giải hoàn chỉnh.
- Q4. SLAM mono cần $Sim(3)$ cho loop closure còn SLAM stereo chỉ cần $SE(3)$. Nếu bạn dùng $SE(3)$ cho mono thì chuyện gì xảy ra cụ thể — không phải “kém chính xác hơn” mà là gì?
- Q5. Back-end bền cho phép bộ giải tắt các cạnh nghi ngờ. Nghe như nó giải quyết hoàn toàn bài toán loop closure sai. Nêu chính xác điều kiện để nó hoạt động, và nói khi nào điều kiện đó bị vi phạm.

5.1 Đặt vấn đề

Mọi bộ ước lượng ở B/09 có chung một tính chất: sai số của chúng *tích lũy*. Mỗi bước thêm một chút sai số, và không có gì kéo nó về. Với một robot chạy tám tiếng trong một nhà kho, sai số cuối ngày có thể là hàng chục mét — đủ để bản đồ trở nên vô dụng.

Điều làm vấn đề này đặc biệt khó chịu: *robot thường xuyên quay lại chỗ cũ*. Nó đi qua cùng một lối đi mười lần một ngày. Ở lần thứ mười, nó “biết” rằng nó đang ở chỗ đã đi qua — theo nghĩa thông tin đó có trong dữ liệu — nhưng nếu không ai nói cho nó, nó sẽ tiếp tục vẽ một lối đi thứ mười cách lối đi thứ nhất năm mét.

Bài toán gốc: phát hiện việc quay lại, và dùng nó để sửa. Đây là chữ “M” trong SLAM — không có nó, ta chỉ có odometry.

Ai gặp bài toán này? Mọi hệ chạy lâu trong không gian hữu hạn. Với một xe tự lái đi một chiều trên đường cao tốc, loop closure ít quan trọng; với robot dịch vụ, nó là yêu cầu sống còn.

Trước khi có các phương pháp truy hồi hiệu quả, cách làm là so khung hiện tại với *mọi* khung quá khứ. Chi phí tuyến tính theo số khung mỗi lần, tức bậc hai theo thời gian — không khả thi. Tút từ trực quan giải quyết bằng cách biến mỗi ảnh thành một vectơ thưa và dùng chỉ mục ngược, đưa chi phí truy hồi xuống gần hằng số [37].

Nhưng vấn đề thật không phải chi phí. Đó là *perceptual aliasing*, và nó nghiêm trọng hơn ở đây so với ở B/07 vì hai lý do. Thứ nhất, không gian tìm kiếm lớn hơn nhiều — ta so với toàn bộ bản đồ, không chỉ với khung trước. Thứ hai, và quan trọng hơn, *hậu quả lớn hơn nhiều*. Một khớp điểm sai giữa hai khung liền kề tạo ra một phần dư lớn mà M-estimator hạ trọng số. Một loop closure sai tạo ra một ràng buộc nói “hai chỗ cách nhau năm mươi mét này thực ra là một chỗ”, và bộ giải — vốn tin mọi ràng buộc — sẽ gập bản đồ lại để thỏa mãn nó.

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và nó định hình cả chương: chi phí của một dương tính giả lớn hơn chi phí của một âm tính giả nhiều bậc độ lớn. Bỏ lỡ một loop closure nghĩa là trôi thêm một chút, và sẽ có cơ hội khác ở vòng sau. Chấp nhận một loop closure sai nghĩa là phá huỷ sản phẩm của hàng giờ làm việc, và thường không phục hồi được.

TRỰC GIÁC

Hình dung bạn đi bộ quanh một khu phố, vừa đi vừa vẽ bản đồ bằng cách đếm bước và ghi hướng rẽ.

Sau một vòng, bạn quay về điểm xuất phát — nhưng bản đồ của bạn nói bạn đang ở cách đó ba mươi mét. Sai số đếm bước đã tích lũy.

Bây giờ bạn *nhận ra* cái cửa hàng trước mặt chính là cửa hàng bạn xuất phát. Đây là thông tin cực kỳ quý: nó nói điểm đầu và điểm cuối trùng nhau. Bạn lấy bản đồ ra và *kéo* nó cho hai điểm khớp, phân phối ba mươi mét sai lệch đều ra cả vòng. Bản đồ đột nhiên đúng hơn nhiều — không chỉ ở điểm cuối mà ở *mọi* chỗ.

Đó là loop closure, và nó là phép thuật thật sự của SLAM.

Nhưng bây giờ hình dung bạn nhầm. Cái cửa hàng trước mặt thuộc cùng một chuỗi, cùng biển hiệu, cùng cách bày hàng — nhưng nó ở một khu phố khác cách đó nửa cây số. Bạn kéo bản đồ cho hai điểm khớp.

Kết quả không phải một bản đồ hơi sai. Đó là một bản đồ *gập đôi*: nửa cây số bị nén thành không, hai con phố khác nhau chồng lên nhau, và mọi thứ bạn đã ghi đúng suốt buổi sáng giờ nằm sai chỗ. Và bạn không có cách nào biết, vì bản đồ mới vẫn tự nhất quán — nó nhất quán với một giả thuyết sai.

Chênh lệch giữa hai kịch bản ấy là toàn bộ lý do chương này dài như vậy. Việc nhận ra chỗ cũ thì dễ. Việc *chắc chắn* rằng mình không nhầm mới là bài toán.

5.2 Phát hiện: nhận ra chỗ cũ

A. Vì sao đây là bài toán truy hồi

Điểm khái niệm quan trọng đầu tiên: phát hiện loop closure *không* phải bài toán khớp điểm mà là bài toán *truy hồi* (retrieval). Ta không hỏi “điểm này khớp với điểm nào” mà hỏi “khung này giống khung nào nhất trong hàng chục nghìn khung đã lưu”.

Khác biệt ấy quyết định công cụ. Khớp điểm dùng khoảng cách descriptor và ratio test; truy hồi dùng một *biểu diễn toàn cục* cho mỗi ảnh, một chỉ mục để tìm nhanh, và một thang điểm tương đồng. Đây là bài toán mà cộng đồng tìm kiếm thông tin đã giải cho văn bản, và túi từ trực quan là việc mượn nguyên giải pháp đó.

B. Túi từ trực quan

Quy trình [37], [38]. *Ngoại tuyến*: gom một tập lớn descriptor từ dữ liệu đại diện, phân cụm chúng thành một *từ điển* có cấu trúc cây, mỗi lá là một “từ”. *Trực tuyến*: với mỗi khung, gán mỗi descriptor vào từ gần nhất, thu được một histogram thưa; tính điểm tương đồng giữa hai histogram bằng khoảng cách có trọng số TF-IDF.

Hai chi tiết kỹ thuật đáng biết vì chúng là lý do phương pháp này khả thi.

Chỉ mục ngược (inverse index): với mỗi từ, lưu danh sách các khung chứa nó. Khi truy vấn, chỉ cần duyệt các khung chia sẻ ít nhất một từ với khung hiện tại — thường là một phần rất nhỏ của bản đồ. Đây là thứ đưa chi phí từ tuyến tính theo số khung xuống gần hằng số.

Chỉ mục thuận (direct index): với mỗi khung, lưu descriptor nào thuộc nhánh nào của cây. Khi đã có ứng viên, chỉ so khớp các descriptor thuộc cùng nhánh thay vì so tất cả với tất cả — tăng tốc bước kiểm định hình học ở mục 3 lên nhiều lần. Chi tiết này ít được nhắc nhưng nó là lý do DBoW2 nhanh trong thực tế.

TF-IDF đáng một câu giải thích vì nó là chỗ trực giác đẹp. Một từ xuất hiện trong *mọi* khung — chẳng hạn một mẫu vân sàn phổ biến — không mang thông tin phân biệt nào, nên IDF hạ trọng số của nó gần về không. Một từ hiếm — một vật đặc biệt trong phòng — mang rất nhiều thông

tin. Đây chính là cơ chế mà một người dùng khi tìm đường: bạn không định vị bằng “có tường trắng” mà bằng “có cái cây bonsai kỳ lạ ở góc”.

C. Place recognition học được

NetVLAD [39] và các hệ sau học một biểu diễn toàn cục trực tiếp từ dữ liệu, tối ưu cho việc nhận ra cùng một địa điểm qua các điều kiện khác nhau — mùa, thời tiết, ngày đêm. Đây là cải thiện rõ rệt so với BoW ở điều kiện thay đổi mạnh, vì BoW dựa trên descriptor cục bộ vốn không bền với thay đổi về ngoài lớn.

Tôi *không* dẫn số precision/recall so sánh các hệ ở đây, vì cùng lý do ở B/07 khối *Cận bầy*: các con số ấy phụ thuộc cực mạnh vào bộ dữ liệu, vào cách định nghĩa “cùng một địa điểm”, và vào ngưỡng. Điều có thể nói trung thực: học được thắng rõ ở *thay đổi về ngoài lớn* (mùa, ngày đêm) và cái giá là GPU cùng tính khó dự đoán khi ra ngoài phân bố huấn luyện.

D. Nhất quán chuỗi

Một kỹ thuật đơn giản và rất hiệu quả, dựa trên một quan sát tầm thường: robot di chuyển liên tục, nên nếu khung t khớp với khung s trong bản đồ, thì khung $t + 1$ nên khớp với khung gần $s + 1$.

SeqSLAM [40] khai thác điều này triệt để: thay vì tìm khớp cho từng ảnh, nó tìm khớp cho cả một *chuỗi* ảnh. Kết quả là nó chịu được những ảnh riêng lẻ rất kém — và nó hoạt động trong điều kiện mà khớp từng ảnh hoàn toàn thất bại.

Ý tưởng chung đáng nhớ hơn thuật toán cụ thể: *ràng buộc thời gian là một nguồn thông tin miễn phí chống lại perceptual aliasing*. Ba mươi cái kệ giống hệt nhau, nhưng chỉ có *một* cái mà robot có thể tới được từ chỗ nó đứng một giây trước. Hầu hết các hệ khai thác điều này ở dạng đơn giản — yêu cầu vài khung liên tiếp cùng đề xuất một vùng — và đó là một trong những phép kiểm rỏ nhất có sẵn.

I 5.3 Kiểm định hình học: tầng phòng thủ chính

A. Quy trình

Một ứng viên từ truy hồi *chưa phải* một loop closure. Nó phải qua kiểm định hình học, và đây là câu trả lời một phần cho câu hỏi mở đầu thứ hai.

Quy trình: khớp descriptor giữa khung hiện tại và khung ứng viên (dùng chỉ mục thuận để tăng tốc); chạy RANSAC để tìm phép biến đổi — PnP nếu ứng viên có landmark 3D, hoặc $Sim(3)$ nếu hai bên đều có cấu trúc; đếm inlier và đo phần dư.

Vì sao hai trăm inlier vẫn chưa đủ? Ba lý do, và cả ba đều là bài học đã gặp.

Một — perceptual aliasing thật sự. Nếu hai địa điểm giống nhau về mặt hình học *lẫn* về ngoài — hai hành lang xây giống hệt nhau — thì một phép biến đổi nhất quán tồn tại thật, và RANSAC tìm ra nó. Đây là trường hợp mà kiểm định hình học không cứu được, và nó là lý do cần thêm các tầng ở mục B.

Hai — cấu hình suy biến. Như A/01 mục 6 đã nói, số inlier cao tương thích hoàn hảo với một mô hình không xác định. Nếu hai khung đều nhìn vào một bức tường phẳng, phép biến đổi tìm được có thể có một hướng tự do.

Ba — chất lượng phân bố. Hai trăm inlier tụ ở một góc ảnh cho một phép biến đổi có số điều kiện tẻ, đúng như B/07 mục 4A.

B. Bốn tầng phòng thủ

Thiết kế đúng có nhiều tầng, mỗi tầng bắt một loại lỗi khác nhau. Đây là phần còn lại của câu trả lời cho câu hỏi thứ hai.

Tầng một — ngưỡng inlier bảo thủ. Cần thiết nhưng không đủ. Đặt cao hơn nhiều so với ngưỡng dùng cho tracking, vì chi phí lỗi khác nhau.

Tầng hai — nhất quán chuỗi. Yêu cầu nhiều khung liên tiếp cùng đề xuất một vùng bản đồ, với các tư thế tương đối nhất quán với odometry giữa chúng. Rẻ và mạnh — nó tấn công trực tiếp vào perceptual aliasing bằng ràng buộc thời gian.

Tầng ba — nhất quán với hiệp phương sai hiện tại. Nếu hệ thống nghĩ nó đang ở A với bất định ± 5 m, thì một loop closure nói nó đang ở cách A năm mươi mét là *không nhất quán* với chính ước lượng của nó. Về nguyên tắc, đây là phép kiểm mạnh nhất: nó dùng toàn bộ thông tin hệ thống có. Trong thực tế nó bị hạn chế bởi bệnh quá tự tin ở $A/06$ — nếu hiệp phương sai nhỏ hơn sự thật, phép kiểm này sẽ loại cả các loop closure đúng. Cách dùng an toàn là đặt ngưỡng rất rộng, chấp nhận rằng nó chỉ bắt được các trường hợp sai rành rành.

Tầng bốn — nhất quán với các loop closure khác. Nếu một cạnh mâu thuẫn với một nhóm cạnh đã được chấp nhận, nó đáng ngờ. Đây là ý tưởng của RRR [41]: nhóm các loop closure thành các cụm và kiểm tính nhất quán giữa các cụm, loại cả cụm nếu nó mâu thuẫn với phần còn lại. Mạnh nhất trong bốn tầng và cũng đắt nhất.

CẦN NHỚ

Chi phí của hai loại lỗi bất đối xứng tới mức nó phải chi phối mọi quyết định thiết kế trong chương này.

Bỏ lỡ một loop closure: hệ thống trôi thêm một chút, và sẽ có cơ hội khác ở vòng sau. Chi phí tuyến tính và có thể phục hồi.

Chấp nhận một loop closure sai: bản đồ gấp lại, hàng giờ làm việc bị phá huỷ, và thường không phục hồi được vì không có cơ chế nào phát hiện sau đó — bản đồ gấp vẫn tự nhất quán.

Ba hệ quả thiết kế trực tiếp. Một: **ngưỡng phải bảo thủ**, và tỉ lệ dương tính giả nên được đặt làm mục tiêu chứ không phải tỉ lệ phát hiện. Hai: **nhiều tầng phòng thủ độc lập** tốt hơn một tầng tốt, vì các tầng bắt các loại lỗi khác nhau. Ba: **luôn có back-end bền** như tầng cuối, vì không bộ phát hiện nào hoàn hảo.

Và một hệ quả về cách đo: khi đánh giá một hệ loop closure, đường cong precision-recall quan trọng hơn một con số duy nhất, và **vùng precision rất cao** mới là vùng đáng quan tâm. Một hệ có recall 80% ở precision 99% tốt hơn nhiều so với một hệ có recall 95% ở precision 90% — dù con số thứ hai trông đẹp hơn. Chi tiết ở C/18.

5.4 Đóng vòng: áp dụng ràng buộc

A. Vì sao mono cần $Sim(3)$

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và nó là ứng dụng trực tiếp nhất của A/05 mục 3B.

Trong SLAM mono, tỉ lệ là gauge tự do, và nó *trôi*: mỗi lần triangulate một landmark mới từ các landmark cũ, một sai số tỉ lệ nhỏ được đưa vào, và các sai số ấy tích lũy. Sau một vòng dài, tỉ lệ ở cuối vòng có thể lệch vài phần trăm so với ở đầu.

Hệ quả cụ thể nếu dùng $SE(3)$: phép biến đổi $SE(3)$ bảo toàn khoảng cách, nên nó *không thể* ghép hai đầu vòng có tỉ lệ khác nhau. Bộ giải sẽ tìm một thoả hiệp — và thoả hiệp ấy không phải

một sai số nhỏ phân bố đều mà là một *biến dạng* của bản đồ: một phần bị nén, một phần bị kéo, và các ràng buộc odometry bị vi phạm một cách có hệ thống. Đây không phải “kém chính xác hơn” mà là một chế độ hỏng khác về chất.

Với $Sim(3)$ [29], phép biến đổi có bảy bậc tự do gồm cả tỉ lệ, nên nó ghép được hai đầu và *đồng thời* sửa trôi tỉ lệ. Quy trình đầy đủ: tính $Sim(3)$ giữa hai khung, áp dụng nó để hiệu chỉnh các tư thế và landmark trong vùng ảnh hưởng, rồi chạy pose graph $Sim(3)$ để phân phối sai số.

Với stereo hoặc VIO, tỉ lệ quan sát được nên nó không trôi, và $SE(3)$ là đủ — và với VIO thì còn rút xuống bốn bậc vì trọng lực ghim roll với pitch. Số bậc tự do lại một lần nữa là hệ quả của cấu hình cảm biến, không phải một lựa chọn.

B. Pose graph và phân phối sai số

Sau khi có ràng buộc, chạy tối ưu pose graph (B/09 mục 4). Điều quan trọng cần hiểu: tối ưu này *không* làm cho sai số biến mất — nó *phân phối lại* sai số một cách tối ưu theo trọng số.

Trực giác: ba mươi mét sai lệch ở điểm cuối không phải lỗi của riêng bước cuối. Nó là tổng của hàng nghìn sai số nhỏ dọc đường. Pose graph phân bổ phần sửa cho từng cạnh theo tỉ lệ nghịch với độ tin cậy của nó — cạnh có hiệp phương sai lớn nhận phần sửa nhiều hơn. Đây chính là bài toán MAP đang làm đúng việc của nó, và nó cho thấy vì sao hiệp phương sai của các cạnh odometry phải đúng: nếu chúng sai, phần sửa được phân bổ sai chỗ.

C. Hoà nhập với luồng tracking

Một vấn đề kỹ thuật thật mà ít tài liệu học thuật nói tới, và nó thuộc về C/13 nhiều hơn nhưng cần nêu ở đây.

Tối ưu pose graph mất từ hàng chục mili giây tới hàng giây tùy kích thước; BA toàn cục sau đó còn lâu hơn. Trong lúc đó, luồng tracking *vẫn phải chạy* — không được dừng vì camera vẫn gửi khung và robot vẫn di chuyển. Nhưng tracking đang dùng bản đồ mà loop closing sắp sửa đổi.

Ba cách xử lý trong thực tế. *Đóng băng*: dừng tracking trong lúc cập nhật — đơn giản, và không chấp nhận được nếu vòng điều khiển phụ thuộc vào tư thế. *Sao chép*: loop closing làm việc trên một bản sao, rồi hoán đổi nhanh; tốn bộ nhớ và cần xử lý các khung xử lý trong lúc đó. *Cập nhật gia tăng*: áp dụng phần hiệu chỉnh dần dần. ORB-SLAM3 dùng một biến thể của cách thứ hai với một giai đoạn hoà nhập cẩn thận [16].

Điều đáng nhớ: đây là loại vấn đề *không xuất hiện trong bài báo nhưng chiếm phần lớn thời gian kỹ thuật*. Nó là một trong những lý do khoảng cách giữa một bài báo và một sản phẩm lớn hơn về ngoài, và là chủ đề của D/20.

I 5.5 Back-end bền: sống sót khi đã nhầm

Không bộ phát hiện nào hoàn hảo. Tầng cuối là làm cho bài toán tối ưu chịu đựng được các cạnh sai.

A. Ba họ kỹ thuật

Switchable constraints [42]: gán cho mỗi cạnh loop closure một biến “công tắc” $s \in [0, 1]$ và đưa nó vào bài toán tối ưu, với một prior kéo s về một. Nếu một cạnh mâu thuẫn mạnh với phần còn lại, bộ giải thấy rẻ hơn khi hạ s xuống — tức tắt cạnh đó. Ý tưởng đẹp: *để bộ giải tự quyết định tin cạnh nào*, thay vì quyết định trước.

DCS [43]: cùng ý tưởng nhưng loại bỏ biến công tắc bằng cách giải cho nó ở dạng đóng và thay vào — kết quả là một hàm mất mát bền có dạng cụ thể. Rẻ hơn và dễ cài hơn, cho hành vi tương tự.

GNC [44]: bắt đầu với hàm mất mát gần lỗi rồi làm nó không lỗi dần (A/04 mục 6C). Giải bài toán dễ trước để có khởi tạo tốt, rồi dùng nó cho bài toán khó hơn. Mạnh hơn hai cách trên khi tỉ lệ outlier cao.

Cả ba là các biến thể của cùng một ý tưởng, và đối ngẫu Black–Rangarajan [45] là chỗ sự thống nhất ấy được phát biểu: tối ưu với một hàm mất mát bền *tương đương* với tối ưu đồng thời trên biến gốc và một tập biến trọng số outlier. Switchable constraints làm điều đó một cách tường minh; DCS và M-estimator làm ngầm.

B. Điều kiện để nó hoạt động

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó là giới hạn quan trọng nhất của mục này.

Back-end bền hoạt động khi các cạnh sai là *thiếu số* và *không nhất quán với nhau*. Lúc đó, đa số đúng tạo ra một “sự thật” mà thiếu số sai mâu thuẫn với, và bộ giải tắt thiếu số.

Điều kiện ấy bị vi phạm trong đúng tình huống khó nhất: *perceptual aliasing có hệ thống*. Trong hành lang kho hàng với ba mươi kệ giống nhau, các loop closure sai không ngẫu nhiên — chúng đều nói “bạn đang ở kệ nào đó”, và chúng *nhất quán với nhau* theo một cách sai. Nếu số cạnh sai vượt số cạnh đúng, back-end bền sẽ tắt các cạnh *đúng*. Nó không chỉ thất bại mà thất bại theo hướng ngược lại.

Đây là lý do back-end bền là tầng *cuối cùng* chứ không phải tầng duy nhất, và là lý do bốn tầng ở mục 3B không thể bỏ qua. Nó cũng là lý do vì sao perceptual aliasing được liệt kê ở D/22 như một bài toán chưa giải quyết: nó phá vỡ giả định nền của mọi kỹ thuật bền.

C. Hai nguồn thông tin ngoài về ngoài

Đây là câu trả lời cho câu hỏi mở đầu thứ ba. Trong môi trường lặp lại có hệ thống, hai nguồn vẫn phân biệt được.

Một — ràng buộc thời gian và odometry. Robot không dịch chuyển tức thời. Nếu một giây trước nó ở kệ thứ năm, nó không thể đang ở kệ thứ hai mươi. Đây là thông tin mạnh và miễn phí, và nó là cơ sở của tầng hai và tầng ba ở mục 3B. Giới hạn: nó chỉ giúp khi odometry còn đáng tin — tức đúng lúc ta chưa cần loop closure. Sau tám tiếng trôi, bất định đã lớn tới mức ràng buộc này không loại được gì.

Hai — neo tuyệt đối. Một marker fiducial đặt ở các vị trí đã biết, một tag UWB, một tín hiệu wifi. Đây là cách mà phần lớn hệ thống kho hàng thật giải quyết vấn đề, và nó là một lời thú nhận đáng chú ý: *bài toán thuần thị giác trong môi trường lặp lại chưa có lời giải tốt, nên công nghiệp giải nó bằng cách thay đổi môi trường*. Cây `.. /marker/` bàn về nhánh này. Giới hạn: nó đòi sửa đổi cơ sở hạ tầng, không phải lúc nào cũng làm được, và nó chuyển bài toán sang việc bảo trì các neo đó.

Cả hai đều không phải lời giải hoàn chỉnh, và đó là phát biểu trung thực nhất tôi đưa ra được.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [16]	DBoW2 + kiểm định $Sim(3)$ + pose graph + BA toàn cục; Atlas ghép nhiều bản đồ	Cài đặt đầy đủ nhất; đọc phần hoà nhập với tracking ở mục 4C
VINS-Fusion [10]	Pose graph 4-DoF riêng biệt với luồng VIO	Kiến trúc hai tầng; 4-DoF vì có IMU, đúng A/05 mục 3B

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
DBoW2 [37]	Từ điển nhị phân, chỉ mục ngược và thuận	Đọc để hiểu chỉ mục thuận — chỉ tiết ít được nhắc nhưng quyết định tốc độ
RRR [41]	Phân cụm loop closure và kiểm nhất quán giữa các cụm	Tầng bốn ở mục 3B — mạnh nhất và đắt nhất
GTSAM [44]	GNC cài sẵn cho pose graph	Mục 5A; dễ thử nhất trong ba họ
maplab [46]	Loop closure đa phiên; quản lý bản đồ dài hạn	Hiếm framework xử lý tốt mảng nhiều phiên — xem C/13

MINI PROJECT SLAM · Đo đường cong precision-recall của hệ loop closure, rồi cố tình tìm cạnh sai · 4 ngày

Mục tiêu	chuyển từ “hệ của tôi đóng vòng được” sang “hệ của tôi có tỉ lệ dương tính giả là X ở ngưỡng Y , và nó sống sót được $Z\%$ cạnh sai”.
Dữ liệu	một chuỗi có loop thật và có chân trị (KITTI hoặc EuRoC); cộng một chuỗi tự quay trong môi trường lặp lại — một hành lang, một dãy phòng giống nhau. Chuỗi thứ hai là phần quan trọng.
Các bước	<i>Phần một:</i> cài phát hiện bằng BoW, ghi lại điểm số cho mọi cặp khung, dùng chân trị để gán nhãn đúng/sai, vẽ đường cong precision-recall; chú ý vùng precision > 99%. <i>Phần hai:</i> thêm từng tầng phòng thủ ở mục 3B và vẽ lại đường cong sau mỗi tầng. <i>Phần ba:</i> chạy pose graph với các cạnh đã thêm thêm $k\%$ cạnh sai ngẫu nhiên, với k từ 0 tới 50; so ATE giữa pose graph thường, với Huber, và với GNC.
Xong khi	bạn có ba kết quả. (1) Một đường cong precision-recall và một ngưỡng chọn từ vùng precision cao chứ không từ điểm F1 tối đa — và bạn giải thích được vì sao. (2) Một bảng cho thấy mỗi tầng phòng thủ đóng góp bao nhiêu, và tầng nào rẻ nhất trên mỗi đơn vị cải thiện. (3) Một đường cong ATE theo tỉ lệ cạnh sai, cho thấy GNC giữ được tới một ngưỡng rồi sụp — và bạn xác định được ngưỡng đó, vốn chính là điều kiện ở mục 5B.
Nối với	C/13-quan-ly-ban-do xử lý vấn đề hoà nhập ở mục 4C; C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo precision-recall; D/22-huong-nghien-cuu-con-trong lấy perceptual aliasing làm một trong tám khoảng trống.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Tỉ số ít nhất vài bậc độ lớn, và nó bất đối xứng về *bản chất* chứ không chỉ về độ lớn. Bỏ lỡ: trôi thêm một chút, có cơ hội khác ở vòng sau, phục hồi được. Chấp nhận sai: bản đồ gấp lại, hàng giờ làm việc bị phá, và *không phục hồi được* vì bản đồ gấp vẫn tự nhất quán nên không cơ chế nào phát hiện sau đó. Ba hệ quả thiết kế: ngưỡng phải bảo thủ và mục tiêu nên là tỉ lệ dương tính giả chứ không phải tỉ lệ phát hiện; nhiều tầng phòng thủ độc lập tốt hơn một tầng tốt; luôn phải có back-end bền lâu làm tầng cuối. Và khi đó, vùng precision rất cao mới là vùng đáng quan tâm, không phải điểm F1 tối đa.
- A2. Vì ba lý do. (a) *Perceptual aliasing thật sự*: hai hành lang xây giống hệt nhau thì một phép biến đổi nhất quán tồn tại thật, và RANSAC tìm ra nó — kiểm định hình học không cứu được. (b) *Cấu hình suy biến*: nếu cả hai khung nhìn vào tường phẳng, phép biến đổi có thể có hướng tự do, và số inlier cao tương thích hoàn hảo với điều đó (A/01 mục 6). (c) *Phân bố kém*: hai trăm inlier tự một góc cho phép biến đổi có số điều kiện tẻ. Phép kiểm bổ sung, theo thứ tự rẻ: nhất quán chuỗi (nhiều khung liên tiếp cùng đề xuất một vùng, với tư thế tương đối khớp odometry), nhất quán với hiệp phương sai hiện tại, và nhất quán với các loop closure đã chấp nhận (RRR).
- A3. Hai nguồn: (a) *ràng buộc thời gian và odometry* — robot không dịch chuyển tức thời, nên chỉ một trong ba mươi kè là tới được từ chỗ nó đứng một giây trước; (b) *neo tuyệt đối* — marker, UWB, wifi. Cả hai đều không hoàn chỉnh. Nguồn (a) chỉ giúp khi odometry còn đáng tin, tức đúng lúc ta chưa cần loop closure; sau tám tiếng trôi, bất định đã lớn tới mức nó không loại được gì. Nguồn (b) đòi sửa đổi cơ sở hạ tầng và chuyển bài toán sang việc bảo trì các neo. Việc công

nghiệp chủ yếu dùng (b) là một lời thú nhận đáng chú ý: bài toán thuần thị giác trong môi trường lặp lại chưa có lời giải tốt.

- A4. Cụ thể là bản đồ bị *biến dạng*, không phải “kém chính xác hơn”. Trong mono, tỉ lệ trôi, nên hai đầu vòng có tỉ lệ khác nhau vài phần trăm. Phép biến đổi $SE(3)$ bảo toàn khoảng cách nên nó không ghép được hai đầu có tỉ lệ khác nhau; bộ giải tìm một thoả hiệp, và thoả hiệp ấy nén một phần bản đồ và kéo phần khác, vi phạm các ràng buộc odometry một cách có hệ thống. $Sim(3)$ có bảy bậc gồm cả tỉ lệ nên nó ghép được và *đồng thời* sửa trôi tỉ lệ [29]. Với stereo hoặc VIO thì tỉ lệ quan sát được nên không trôi, và $SE(3)$ — hoặc bốn bậc với VIO — là đủ.
- A5. Điều kiện: các cạnh sai phải là *thiếu số* và *không nhất quán với nhau*. Lúc đó đa số đúng tạo ra một “sự thật” mà thiếu số sai mâu thuẫn với, và bộ giải tắt thiếu số. Điều kiện bị vi phạm trong đúng tình huống khó nhất: *perceptual aliasing có hệ thống*. Trong hành lang ba mươi kệ giống nhau, các cạnh sai không ngẫu nhiên — chúng nhất quán với nhau theo một cách sai. Nếu chúng vượt số cạnh đúng, back-end bền sẽ tắt các cạnh *đúng*, tức thất bại theo hướng ngược lại. Đó là lý do nó là tầng cuối chứ không phải tầng duy nhất, và là lý do perceptual aliasing nằm trong danh sách bài toán chưa giải ở D/22.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đi bộ quanh khu phố và vẽ bản đồ bằng cách đếm bước. Quay về chỗ xuất phát, bản đồ nói bạn còn cách đó ba mươi mét — vì đếm bước có sai số và sai số cộng dồn.

Nhưng bạn *nhận ra* cửa hàng trước mặt chính là cửa hàng lúc xuất phát. Thế là bạn lấy bản đồ và kéo nó cho hai điểm trùng nhau, chia đều ba mươi mét sai lệch ra cả vòng. Bản đồ đột nhiên đúng hơn hẳn — không chỉ ở điểm cuối mà ở mọi chỗ. Đây là phép thuật thật sự, và nó là lý do chữ M trong SLAM tồn tại.

Bây giờ hình dung bạn nhầm. Cửa hàng trước mặt cùng chuỗi, cùng biển hiệu, nhưng ở khu phố khác cách đó nửa cây số. Bạn vẫn kéo bản đồ cho hai điểm trùng.

Kết quả không phải một bản đồ hơi sai. Đó là một bản đồ *gập đôi*: nửa cây số bị nén thành không, hai con phố chồng lên nhau, và mọi thứ bạn ghi đúng suốt buổi sáng giờ nằm sai chỗ. Tệ nhất: bạn không có cách nào biết, vì bản đồ mới *vẫn tự nhất quán* — chỉ là nó nhất quán với một giả thuyết sai.

Đó là lý do chương này dài. Nhận ra chỗ cũ thì dễ. Chắc chắn rằng mình không nhầm mới là bài toán.

Và cách chắc chắn không phải nhìn kỹ hơn vào cửa hàng. Đó là hỏi thêm vài câu khác: *Tôi có thể đi từ chỗ tôi đứng một phút trước tới đây trong một phút không? Nếu đây thật là cửa hàng xuất phát, thì mọi thứ khác trong bản đồ có còn hợp lý không? Mấy phút vừa qua tôi có liên tục thấy những thứ quen thuộc, hay chỉ đúng một khoảnh khắc này?*

Còn một chỗ mà mọi câu hỏi đều thất bại: một dãy kho hàng có ba mươi cửa giống hệt nhau, cách nhau ba mét. Ở đó bạn không phân biệt được, và cũng không có gì trong khung cảnh giúp bạn phân biệt. Đó là bài toán khó nhất trong lĩnh vực này, và câu trả lời thật thà là nó chưa được giải.

Chỗ không rút gọn được: chỗ không rút gọn được là chính cái dãy kho hàng ấy. Mọi kỹ thuật trong chương này đều giả định rằng “số đồng thì đúng”. Nhưng khi môi trường lặp lại một cách có hệ thống, số đồng cũng có thể sai một cách nhất quán, và toàn bộ lập luận sụp đổ. Cách duy nhất thoát ra là thêm thông tin từ bên ngoài thị giác — và đó là lý do các nhà kho thật dán marker lên tường.

Một câu: *Đóng vòng đúng sửa cả một ngày làm việc; đóng vòng sai phá cả một ngày làm việc — và hai việc đó trông giống hệt nhau ở thời điểm quyết định.*

I 5.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *chi phí của dương tính giả lớn hơn âm tính giả nhiều bậc* — là một lập luận cấu trúc, và nó bị bác bỏ nếu một hệ có cơ chế *phát hiện và hủy* một loop closure sai sau khi đã áp dụng. Điều đó không bất khả: RRR [41] làm gần như vậy bằng cách kiểm nhất quán giữa các cụm và

có thể loại cả cụm đã chấp nhận. Nếu một cơ chế như vậy đủ tin cậy, sự bất đối xứng giảm và ngưỡng có thể nới. Tôi cho rằng nó chưa đủ tin cậy trong thực tế, nhưng đây là chỗ tôi có ít bằng chứng nhất (*tôi suy ra, chưa đối chiếu nguồn*).

Mệnh đề thứ hai — *mono cần $Sim(3)$, stereo cần $SE(3)$, VIO cần bốn bậc* — kế thừa từ A/05 mục 3B và kiểm được bằng cách chạy pose graph với số bậc sai rồi xem biến dạng. Nó bị bác bỏ nếu một hệ mono dùng $SE(3)$ mà không biểu hiện biến dạng — điều sẽ xảy ra nếu trôi tỉ lệ đủ nhỏ, chẳng hạn vòng rất ngắn hoặc có prior tỉ lệ ngầm từ đâu đó.

Mệnh đề thứ ba — *back-end bền cần cạnh sai là thiếu số và không nhất quán* — là điều kiện được nêu trong các công trình gốc. Điều tôi *chưa* kiểm là ngưỡng định lượng: bao nhiêu phần trăm cạnh sai thì GNC sụp? Mini project phần ba đo đúng điều đó, và con số thu được sẽ phụ thuộc mạnh vào cấu trúc đồ thị. Đây là một mục trong 99-chua-biet.md.

Một mệnh đề mềm và là mệnh đề tôi ít chắc nhất: chương này khẳng định *perceptual aliasing có hệ thống chưa có lời giải tốt*. Nó bị bác bỏ nếu một hệ đạt precision cao trong môi trường lặp lại có hệ thống mà chỉ dùng thị giác, được kiểm định trên dữ liệu thật chứ không phải benchmark. Các mô hình học được gần đây khai thác ngữ cảnh rộng hơn, nên ranh giới đang dịch chuyển, và tôi không chắc phát biểu này còn đúng trong hai năm nữa.

5.7 Kết nối

Nổi lên trên. B/07-dac-trung-va-data-association là chương mà bài toán này xuất hiện lần đầu ở quy mô nhỏ; điều đáng chú ý là *cùng một nguyên lý* — thông tin để phát hiện khớp sai nằm trong quan hệ với các khớp khác — áp dụng ở cả hai quy mô, chỉ khác hậu quả. B/09-paradigm-uoc-luong-back-end cho pose graph và giải thích vì sao MSCKF không đóng vòng được. A/03-lie-group-va-toi-uu-tren-da-tap cho $Sim(3)$; A/04-uoc-luong-map-va-bundle-adjustment cho GNC và M-estimator; A/05-observability-gauge-va-suy-bien cho số bậc tự do và cho lý do trôi tỉ lệ tồn tại.

Nổi xuống dưới. B/12-relocalization dùng cùng công nghệ phát hiện — BoW, kiểm định hình học — cho một mục đích khác, và có cùng cấu trúc chi phí bất đối xứng. C/13-quan-ly-ban-do xử lý vấn đề hoà nhập ở mục 4C và mở rộng sang ghép nhiều bản đồ, nhiều phiên. C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo precision-recall và lập luận vì sao vùng precision cao mới quan trọng.

Nổi xa hơn. D/19-ben-vung-trong-the-gioi-thuc bàn về loop closure khi môi trường đã thay đổi — cùng một chỗ nhưng đồ đạc đã dịch, ánh sáng đã khác. D/22-huong-nghien-cuu-con-trong lấy perceptual aliasing làm một trong tám khoảng trống, và mục 5B của chương này là lập luận vì sao nó khó: nó phá vỡ giả định nền của mọi kỹ thuật bền.

Nổi ngang: ../marker/ là nhánh giải bài toán này bằng cách thay đổi môi trường — đặt các neo có danh tính duy nhất. Đây là lời giải thực dụng nhất hiện có cho môi trường lặp lại, và đọc cây đó cho thấy cái giá của nó: hiệu chuẩn, bảo trì, và giới hạn tầm nhìn.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ước lượng tỉ số chi phí giữa loop closure sai và loop closure bỏ lỡ, rồi nêu ba hệ quả thiết kế của nó.
2. Vì sao phát hiện loop closure là bài toán truy hồi chứ không phải bài toán khớp, và điều đó quyết định công cụ thể nào?
3. Nêu bốn tầng phòng thủ chống loop closure sai, và nói mỗi tầng bắt được loại lỗi nào mà ba tầng kia bỏ sót.

4. Dùng $SE(3)$ thay cho $Sim(3)$ trong SLAM mono. Mô tả *cụ thể* bản đồ sẽ trông thế nào, không dùng cụm từ “kém chính xác hơn”.
5. Pose graph không làm sai số biến mất mà phân phối lại nó. Theo nguyên tắc nào, và điều gì phải đúng thì việc phân phối mới đúng?
6. Nêu điều kiện để back-end bền hoạt động, và mô tả một tình huống thực tế mà điều kiện đó bị vi phạm.
7. Khi đánh giá hệ loop closure, vì sao điểm F1 tối đa là tiêu chí sai, và tiêu chí nào đúng?

Nguồn

- [1] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, mức tin A.
- [2] J. Shi and C. Tomasi, “Good Features to Track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 1994.
- [3] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2006.
- [4] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2011.
- [5] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [6] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [7] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [8] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [9] J.-Y. Bouguet, “Pyramidal Implementation of the Affine Lucas Kanade Feature Tracker: Description of the Algorithm,” Intel Corporation, tech. rep., 2001, mức tin B; báo cáo kỹ thuật, là tài liệu tham chiếu de-facto cho KLT thấp.
- [10] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [11] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; nguồn của inverse compositional.
- [12] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [13] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [14] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2005.
- [15] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2020.
- [16] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.

- [17] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [18] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [19] C. Forster, Z. Zhang, M. Gassner, et al., "SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.
- [20] G. Klein and D. Murray, "Parallel Tracking and Mapping for Small AR Workspaces," in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A; công trình tách tracking khỏi mapping thành hai luồng, 2007.
- [21] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," in *European Conference on Computer Vision (ECCV)*, mức tin A, 2014.
- [22] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [23] J. Civera, A. J. Davison, and J. M. M. Montiel, "Inverse Depth Parametrization for Monocular SLAM," *IEEE Transactions on Robotics*, vol. 24, no. 5, pp. 932–945, 2008, mức tin A; nguồn của lập luận vì sao inverse depth tuyến tính hoá tốt hơn.
- [24] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của cách khử landmark bằng chiếu lên null space, 2007.
- [25] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2024.
- [26] L. Piccinelli, Y.-H. Yang, C. Sakaridis, et al., "UniDepth: Universal Monocular Metric Depth Estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [27] A. Rosinol, M. Abate, Y. Chang, et al., "Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [28] M. Kaess, H. Johannsson, R. Roberts, et al., "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [29] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên Sim(3), 2010.
- [30] V. Usenko, N. Demmel, D. Schubert, et al., "Visual-Inertial Mapping with Non-Linear Factor Recovery," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [31] N. Demmel, C. Sommer, D. Cremers, et al., "Square Root Bundle Adjustment for Large-Scale Reconstruction," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [32] D. M. Rosen, L. Carlone, A. S. Bandeira, et al., "SE-Sync: A Certifiably Correct Algorithm for Synchronization over the Special Euclidean Group," *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019, mức tin A.
- [33] R. Hartley, J. Trumpf, Y. Dai, et al., "Rotation Averaging," *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.

- [34] P. Geneva, K. Eickenhoff, W. Lee, et al., "OpenVINS: A Research Platform for Visual-Inertial Estimation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [35] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [36] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.
- [37] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012, mức tin A.
- [38] D. Nistér and H. Stewénus, "Scalable Recognition with a Vocabulary Tree," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2006.
- [39] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018, mức tin A.
- [40] M. J. Milford and G. F. Wyeth, "SeqSLAM: Visual Route-Based Navigation for Sunny Summer Days and Stormy Winter Nights," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [41] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013, mức tin A; thuật toán RRR.
- [42] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2012.
- [43] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2013.
- [44] H. Yang, P. Antonante, V. Tzoumas, et al., "Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [45] M. J. Black and A. Rangarajan, "On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision," *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [46] T. Schneider, M. Dymczyk, M. Fehr, et al., "maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [47] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022, mức tin B.
- [48] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2023.
- [49] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2015.
- [50] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-Based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của khẳng định APR không tốt hơn image retrieval, 2019.

Relocalization

PHỤ THUỘC A/01-hình-học-da-thi (PnP — bước cuối của mọi pipeline); A/06-bat-dinh-va-lan-truyen-sai-so (hiệp phương sai để đặt ngưỡng chấp nhận); B/07-dac-trung-va-data-association (khớp điểm giữa hai khung rất xa nhau); B/11-loop-closure-va-nhat-quan-toan-cuc (cùng công nghệ phát hiện, cùng cấu trúc chi phí bất đối xứng).

TÓM TẮT

Có ba khoảnh khắc đặc biệt trong đời một hệ SLAM: lúc bắt đầu (B/10), lúc quay lại chỗ cũ (B/11), và lúc lạc đường — chương này. Relocalization trả lời câu hỏi “tôi đang ở đâu trong bản đồ này” khi không có tiên nghiệm nào, và nó là chương gần sản phẩm nhất trong Phần B vì nó quyết định điều mà khách hàng thật sự quan tâm: hệ thống có tự phục hồi được không, và mất bao lâu. Nội dung có bốn phần. Một: hai bài toán khác nhau thường bị gộp làm một — phục hồi sau mất bám ngắn, và định vị từ đầu trong một bản đồ đã có — với các yêu cầu rất khác nhau. Hai: kiến trúc phân cấp *truy hồi* → *khớp* → *PnP* đã trở thành chuẩn công nghiệp, và vì sao nó thắng. Ba: các hướng học được, gồm cả một trường hợp đáng học là hồi quy tư thế tuyệt đối — một ý tưởng hấp dẫn hoá ra không làm điều người ta tưởng. Bốn: kiểm định chấp nhận, với cùng lập luận bất đối xứng như B/11 nhưng gay gắt hơn. Nếu chỉ nhớ một điều: *một relocalization sai hại hơn không relocalization* — vì hệ thống đang lạc thì biết mình lạc, còn hệ thống tin sai thì không.

KIẾN THỨC CẦN CÓ TRƯỚC

- **relocalization** — xác định tư thế camera trong một bản đồ đã có, không dùng tiên nghiệm về vị trí. Khác với tracking ở chỗ tracking luôn có dự đoán từ khung trước.
- **kidnapped robot** — bài toán kinh điển: robot bị nhắc lên và đặt ở một chỗ khác trong khi tắt nguồn, rồi bật lên và phải tự tìm ra mình ở đâu. Tên gọi cho trường hợp khó nhất của relocalization.
- **cold start** — khởi động hệ thống trong một bản đồ đã lưu, không biết vị trí ban đầu. Cùng bài toán với kidnapped robot nhưng thường có ràng buộc thời gian gắt hơn vì người dùng đang chờ.
- **localization phân cấp (hierarchical localization)** — kiến trúc ba bước: truy hồi vài ứng viên toàn cục, khớp đặc trưng cục bộ với chúng, rồi PnP. Tách bài toán lớn thành hai bài toán nhỏ hơn mà mỗi cái đều có công cụ tốt.
- **scene coordinate regression** — học một hàm ánh xạ trực tiếp từ một mảng ảnh sang tọa độ 3D của nó trong bản đồ, rồi dùng PnP trên các tương ứng thu được. Bỏ qua bước truy hồi và bước mô tả.
- **hồi quy tư thế tuyệt đối (absolute pose regression, APR)** — học một mạng ánh xạ thẳng từ ảnh sang tư thế 6-DoF. Ý tưởng đơn giản nhất có thể, và mục 4B giải thích vì sao nó không hoạt động như kỳ vọng.

CÂU HỎI MỞ ĐẦU

- Q1. Hệ thống mất bám. Nó có thể (a) tiếp tục báo tư thế cuối cùng biết được, (b) báo “tôi không biết”, hoặc (c) relocalize và báo một tư thế mới. Xếp ba lựa chọn theo mức nguy hiểm, và giải thích thứ tự.

- Q2. HLoc tách bài toán thành truy hồi rồi khớp rồi PnP, thay vì khớp trực tiếp với toàn bộ bản đồ. Nêu hai lý do khác nhau vì sao việc tách đó thắng — một lý do về chi phí và một lý do về chất lượng.
- Q3. PoseNet học ánh xạ thẳng từ ảnh sang tư thế. Nó hoạt động trên tập kiểm tra. Nêu điều nó thực sự học được, và vì sao điều đó khác với điều người ta tưởng nó học được.
- Q4. Robot lau nhà bật lên trong một căn hộ đã được lập bản đồ tuần trước. Từ tuần trước, một cái ghế đã bị dịch và rèm cửa đã được kéo. Nêu ba thứ có thể làm relocalization thất bại, xếp theo mức khó xử lý.
- Q5. Relocalization của bạn trả về một tư thế với hai trăm inlier PnP. Nêu ba phép kiểm bổ sung trước khi chấp nhận nó, và nói mỗi phép bắt được lỗi gì.

6.1 Đặt vấn đề

Mọi hệ SLAM mất bám. Không phải nếu mà là khi nào: camera bị che, ánh sáng tắt, robot bị đẩy, chuyển động quá nhanh, một người đi ngang chắn hết khung hình. Câu hỏi thiết kế không phải làm sao để không bao giờ mất bám — điều đó bất khả — mà là *chuyện gì xảy ra sau đó*.

Bài toán gốc: cho một ảnh và một bản đồ, xác định tư thế camera mà không có tiên nghiệm nào về vị trí. Nghe giống loop closure, và công nghệ thì giống thật, nhưng bối cảnh khác ở một điểm quyết định: trong loop closure, hệ thống có một ước lượng vị trí (dù trôi) để kiểm chéo; trong relocalization, nó không có gì.

Mất đi phép kiểm chéo ấy là mất tầng phòng thủ thứ ba ở B/11 mục 3B — phép kiểm “có nhất quán với chỗ tôi nghĩ mình đang đứng không”. Nên relocalization khó hơn loop closure, dù dùng cùng công cụ.

Ai gặp bài toán này? Ba nhóm với ba yêu cầu khác nhau, và gộp chúng làm một là một lỗi thiết kế phổ biến.

Nhóm một — phục hồi sau mất bám ngắn. Hệ đang chạy, mất bám vài giây. Nó có một ước lượng thô từ trước khi mất, và nó chỉ cần tìm lại trong một vùng nhỏ. Yêu cầu: rất nhanh (người dùng không nên nhận ra), và có thể dùng tiên nghiệm.

Nhóm hai — cold start trong bản đồ đã có. Robot bật lên, không biết mình ở đâu. Không có tiên nghiệm, phải tìm trong toàn bộ bản đồ. Yêu cầu: nhanh vừa phải (vài giây chấp nhận được), và đúng — một khởi động sai vị trí là một lỗi mà khách hàng nhìn thấy ngay.

Nhóm ba — localization thuần túy. Không có SLAM, chỉ có một bản đồ lớn đã dựng sẵn và một ảnh cần định vị. Đây là bài toán của thị giác máy tính (định vị ảnh trong thành phố) hơn là của robot, và nó có quy mô lớn hơn nhiều — hàng triệu ảnh trong bản đồ.

Ba nhóm chia nhau công nghệ nhưng khác nhau về ràng buộc, và một hệ sản phẩm thường cần cả nhóm một lẫn nhóm hai với hai đường xử lý khác nhau.

Trước khi có kiến trúc phân cấp, cách làm là khớp đặc trưng của ảnh truy vấn với *toàn bộ* bản đồ. Với bản đồ vài nghìn landmark thì được; với bản đồ vài triệu thì không — cả về chi phí lẫn về chất lượng, vì tỉ lệ khớp sai tăng theo kích thước không gian tìm kiếm. Đây là câu trả lời cho câu hỏi mở đầu thứ hai và là lý do kiến trúc ba bước tồn tại.

TRỰC GIÁC

Bạn tỉnh dậy trong một thành phố bạn đã đi qua, không nhớ mình đang ở đâu, và có một cuốn sổ đầy ảnh chụp các góc phố bạn từng đi.

Cách ngây thơ: giờ từng trang, so ảnh trong sổ với cảnh trước mặt. Với sổ mười trang thì được. Với sổ mười nghìn trang thì hết ngày — và tệ hơn, xác suất bạn gặp một trang *trông giống* mà không phải đúng chỗ tăng theo số trang.

Cách người ta thật sự làm có hai bước. Đầu tiên nhìn tổng thể: kiến trúc kiểu gì, cây gì, màu tường ra sao — và lật nhanh tới vài trang có *ấn tượng chung* giống nhất. Rồi với vài trang đó, so kỹ từng chi tiết: cái biển hiệu này, vết nứt kia.

Hai bước ấy dùng hai loại thông tin khác nhau. Bước một dùng *ấn tượng toàn cục* — thô, nhưng so được rất nhanh với mười nghìn trang. Bước hai dùng *chi tiết cục bộ* — chính xác, nhưng chỉ so được với vài trang.

Đó là kiến trúc phân cấp, và nó thắng vì hai lý do chứ không phải một. Rõ ràng là nó *nhANH hơn*. Ít rõ hơn nhưng quan trọng không kém: nó *chính xác hơn*, vì khi chỉ so chi tiết với ba trang thay vì mười nghìn, cơ hội nhầm giảm hàng nghìn lần.

Còn một chuyện nữa, và nó là chuyện quan trọng nhất của chương. Giả sử sau cả hai bước bạn vẫn không chắc — có hai góc phố trông giống nhau. Bạn có hai lựa chọn: đoán một cái và đi tiếp, hoặc thừa nhận “tôi chưa biết” và đứng nhìn thêm.

Đoán và đi tiếp nguy hiểm hơn nhiều. Vì một khi đã đi, bạn sẽ diễn giải mọi thứ thấy sau đó theo giả định sai, và bạn càng lúc càng tin vào nó. Còn nếu bạn thừa nhận chưa biết, ít nhất bạn vẫn đang tìm.

6.2 Kiến trúc phân cấp

A. Ba bước

HLoc [18] hệ thống hoá kiến trúc đã trở thành chuẩn:

Bước một — truy hồi toàn cục. Tính một biểu diễn toàn cục cho ảnh truy vấn (BoW, NetVLAD, hoặc tương đương) và lấy k khung giống nhất trong bản đồ, với k thường vài chục. Chi phí gần hằng số nhờ chỉ mục.

Bước hai — khớp cục bộ. Với mỗi ứng viên, khớp đặc trưng cục bộ giữa ảnh truy vấn và khung ấy (B/07). Vì chỉ có vài chục ứng viên, ta có thể dùng matcher đắt như SuperGlue mà vẫn chấp nhận được.

Bước ba — PnP. Các khớp cho tương ứng 2D–3D với landmark của bản đồ; chạy PnP trong RANSAC (A/01 mục 5) để lấy tư thế và tập inlier.

B. Vì sao tách lại thẳng

Hai lý do độc lập, và lý do thứ hai quan trọng hơn nhưng ít được nhắc.

Chi phí: bước một là $O(1)$ nhờ chỉ mục ngược; bước hai là $O(k)$ với k nhỏ và không phụ thuộc kích thước bản đồ. Tổng chi phí gần như không tăng khi bản đồ lớn lên — một tính chất thiết yếu cho hệ chạy lâu.

Chất lượng: đây là lý do tinh tế hơn. Khi khớp một đặc trưng với toàn bộ bản đồ, số ứng viên sai tăng tuyến tính theo kích thước bản đồ, nên xác suất một khớp sai lọt qua ratio test cũng tăng. Với mười nghìn landmark, ratio test đủ tốt; với mười triệu, nó không còn đủ. Bằng cách giới hạn tìm kiếm vào vài chục khung, ta giữ không gian tìm kiếm nhỏ và ratio test lấy lại hiệu lực. Nói cách khác: *bước truy hồi không chỉ tăng tốc mà còn khôi phục điều kiện hoạt động của bước khớp.*

Điều này là một trường hợp của một nguyên tắc lặp lại trong cả cây: *thu hẹp không gian tìm kiếm là cách rẻ nhất để cải thiện data association* — cùng nguyên tắc với tiên nghiệm chuyển động ở B/07 mục 5C.

C. Dùng tiên nghiệm thô khi có

Với nhóm một — phục hồi sau mất bám ngắn — ta có tiên nghiệm, và bỏ qua nó là lãng phí.

Tư thế cuối cùng biết được, cộng với thời gian đã trôi qua và một giới hạn vận tốc, cho một vùng khả dĩ. Chỉ tìm trong vùng đó: rẻ hơn nhiều, và tỉ lệ dương tính giả giảm mạnh vì phần lớn các ứng viên gây nhầm lẫn nằm ngoài vùng.

Các nguồn tiên nghiệm thô khác đáng biết trong sản phẩm: odometry bánh xe tiếp tục chạy trong lúc thị giác mất bám; IMU cho hướng và một ước lượng dịch chuyển ngắn hạn; wifi hoặc bluetooth beacon cho vị trí ở mức phòng; một marker trong tầm nhìn cho vị trí tuyệt đối (`.../marker/`). Mỗi nguồn thu hẹp không gian tìm kiếm, và chúng *cộng dồn*.

Đây là lý do thực dụng quan trọng nhất để có nhiều cảm biến, và nó khác với lý do thường được nêu (độ chính xác cao hơn). Cảm biến phụ có giá trị lớn nhất không phải khi mọi thứ chạy tốt mà khi thị giác thất bại — chúng cho hệ thống một chỗ đứng để phục hồi.

I 6.3 Các hướng học được

A. Scene coordinate regression

Thay vì truy hồi rồi khớp, học một hàm ánh xạ trực tiếp từ một mảng ảnh sang tọa độ 3D của nó trong bản đồ. DSAC* [47] và ACE [48] theo hướng này: mạng cho mỗi mảng một tọa độ 3D, và PnP trong RANSAC chạy trên các tương ứng đó.

Ưu điểm: bỏ được cả bước truy hồi lẫn bước mô tả; và vì mạng đã học cảnh cụ thể, nó xử lý được các vùng mà descriptor tổng quát thất bại. ACE còn giảm thời gian huấn luyện cho một cảnh mới xuống mức phút, thay vì giờ.

Giới hạn quan trọng: mô hình *gắn với một cảnh cụ thể*, phải huấn luyện lại cho mỗi bản đồ. Với ứng dụng có một bản đồ cố định — một nhà máy, một bảo tàng — điều đó chấp nhận được. Với robot phải hoạt động ở nhà bất kỳ khách hàng nào, nó là một rào cản triển khai thật: mỗi lần lập bản đồ lại là một lần huấn luyện lại, và điều đó phải chạy trên thiết bị hoặc trên đám mây với mọi hệ quả về quyền riêng tư (C/13).

B. Hồi quy tư thế tuyệt đối, và vì sao nó không làm điều ta tưởng

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và tôi đưa nó vào vì nó là một bài học phương pháp có giá trị vượt ra ngoài chủ đề.

Ý tưởng đơn giản nhất có thể: học một mạng ánh xạ thẳng từ ảnh sang sáu (hoặc bảy) con số tư thế. PoseNet [49] làm điều đó và hoạt động — trên tập kiểm tra.

Sattler và cộng sự [50] đặt câu hỏi mà lẽ ra phải đặt sớm hơn: *mạng thực sự học được gì?* Kết quả của họ là mạng hành xử gần với *nội suy giữa các tư thế huấn luyện* hơn là với một bộ ước lượng hình học. Cụ thể, độ chính xác của nó không vượt hơn đáng kể so với một đường cơ sở rất đơn giản: truy hồi ảnh huấn luyện giống nhất rồi trả về tư thế của ảnh đó.

Điều đó có nghĩa gì? Mạng học được một ánh xạ *từ vẻ ngoài sang tư thế* trên tập huấn luyện, và nó nội suy giữa chúng. Nó *không* học hình học chiều, nên nó không tổng quát hoá ra các tư thế xa tập huấn luyện — đúng chỗ mà một bộ ước lượng hình học lại mạnh nhất.

Bài học phương pháp đáng rút ra, và nó áp dụng rộng hơn nhiều so với chủ đề này: *một phương pháp đạt điểm số tốt trên benchmark không chứng minh rằng nó làm điều ta nghĩ nó làm*. Cách kiểm là so với một đường cơ sở tầm thường thiết kế để có cùng khả năng nội suy nhưng không có gì khác. Nếu phương pháp mới không thắng đường cơ sở ấy, thì phần “mới” chưa được chứng minh. Đây là đúng tinh thần của `research-rules.md` mục 7 về việc phải làm baseline tử tế trước, và đó là lý do tôi đưa trường hợp này vào một chương học tập.

CẠM BẦY

Tôi không dẫn số liệu benchmark relocalization ở đây, và cần nói rõ lý do.

Các con số trên Aachen Day-Night, 7-Scenes hay Cambridge phụ thuộc cực mạnh vào: bản đồ được dựng thế nào (SfM từ chính tập huấn luyện hay từ nguồn khác), ngưỡng nào được coi là “thành công” (5 cm/5° hay 25 cm/2° — các bài khác nhau dùng khác nhau), và có bao nhiêu ảnh trong bản đồ. Hai bài báo báo cáo cho cùng phương pháp trên cùng bộ dữ liệu thường không so sánh được.

Nghiêm trọng hơn, với ứng dụng robot thì các benchmark ấy *đo sai thứ*. Chúng đo độ chính xác trung bình trên các truy vấn có câu trả lời trong bản đồ. Chúng không đo: tỉ lệ trả về một tư thế *sai* một cách tự tin; hành vi khi truy vấn *không* có trong bản đồ; và thời gian xấu nhất. Với sản phẩm, ba thứ đó quyết định hơn.

Cách đúng, và nó lặp lại lời khuyên ở [B/07](#): đo trên dữ liệu của chính bạn, trong chính điều kiện triển khai, và đo *tỉ lệ dương tính giả ở ngưỡng bạn định dùng* chứ không phải độ chính xác trung bình. C/18 bàn cách thiết kế phép đo đó.

6.4 Kiểm định chấp nhận

A. Vì sao reloc sai hại hơn không reloc

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là luận điểm trung tâm của chương.

Xếp ba lựa chọn theo mức nguy hiểm, từ ít tới nhiều:

(b) Báo “tôi không biết” — an toàn nhất. Hệ thống ở trạng thái đã biết là xấu, và mọi module phía trên có thể phản ứng phù hợp: robot dừng lại, người dùng được thông báo, một chiến lược phục hồi được kích hoạt.

(a) Tiếp tục báo tư thế cuối cùng — nguy hiểm vừa. Tư thế ngày càng sai theo thời gian, nhưng sai một cách *dự đoán được*: nó cũ, và nếu hệ thống ghi lại độ tuổi của ước lượng thì module phía trên biết mà chiết khấu.

(c) Relocalize sai — nguy hiểm nhất. Hệ thống báo một tư thế *mới*, với độ tin cậy cao, và hoàn toàn sai. Mọi thứ phía trên tin nó. Tệ hơn cả: hệ thống sẽ bắt đầu tracking từ tư thế sai đó, khớp landmark của vùng sai, và *củng cố* niềm tin sai — vì các quan sát mới sẽ khớp với giả thuyết sai một cách nhất quán trong một thời gian. Nếu nó còn ghi cập nhật vào bản đồ thì bản đồ cũng bị hỏng.

Chênh lệch giữa (b) và (c) là lý do chương này tồn tại. Và nó gay gắt hơn [B/11](#) vì ở đó, hệ thống ít nhất còn một ước lượng để kiểm chéo; ở đây nó không có gì.

B. Bốn phép kiểm

Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

Một — số và tỉ lệ inlier PnP. Cần thiết, không đủ; cùng lý do ở [A/01](#) mục 6. Đặt ngưỡng cao hơn nhiều so với tracking.

Hai — phân bố không gian của inlier. Hai trăm inlier tụ ở một góc ảnh cho một tư thế có bất định lớn theo vài hướng, dù số đếm trông tốt. Kiểm bằng diện tích bao lồi của inlier hoặc bằng số ô lưới có inlier — rẻ và hiệu quả, và nó là B/07 mục 4A áp dụng ở đây.

Ba — hiệp phương sai của tư thế. Tính Λ của bài toán PnP và xem phổ của nó (A/06). Đây là phép kiểm đúng về nguyên tắc vì nó đo trực tiếp mức xác định. Giới hạn: nó chịu bệnh quá tự tin, nên ngưỡng phải bảo thủ.

Bốn — nhất quán thời gian. Đây là phép kiểm mạnh nhất và rẻ bất ngờ: *đừng chấp nhận một relocalization từ một khung duy nhất*. Yêu cầu vài khung liên tiếp cùng cho tư thế nhất quán với nhau và nhất quán với chuyển động đo được giữa chúng (từ odometry, IMU, hay optical flow). Một dương tính giả hiếm khi lặp lại nhất quán qua nhiều khung với chuyển động đúng.

Cái giá của phép kiểm bốn là độ trễ — vài trăm mili giây. Với nhóm hai (cold start) đó là chấp nhận được và nên làm mặc định. Với nhóm một (phục hồi nhanh) thì cần cân nhắc, và đó là một lý do nữa để tách hai đường xử lý.

C. Ba chế độ hồng trong sản phẩm

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, xếp theo mức khó xử lý.

Dễ nhất — thay đổi ánh sáng. Rèm kéo, đèn bật. Descriptor cổ điển kém bền với điều này; place recognition học được bền hơn nhiều. Có lời giải, dù tốn tài nguyên.

Khó vừa — vật thể dịch chuyển. Cái ghế đã dịch. Các landmark trên nó giờ ở sai chỗ, và chúng sẽ là outlier trong PnP. RANSAC xử lý được nếu chúng là thiểu số. Vấn đề thật là *tích lũy*: sau vài tháng, một tỉ lệ đáng kể bản đồ đã lỗi thời, và có lúc thiểu số thành đa số. Cần cơ chế cập nhật bản đồ (C/13), và đó là bài toán chưa được giải tốt.

Khó nhất — thay đổi cấu trúc. Bàn ghế được sắp lại hoàn toàn, tường được sơn lại, một phòng được cải tạo. Bản đồ cũ không còn mô tả thế giới. Không phép kiểm nào phân biệt được “tôi đang ở chỗ khác” với “tôi ở đúng chỗ nhưng chỗ này đã đổi”, và hai kết luận đòi hai hành động ngược nhau. Đây là bài toán *lifelong mapping*, và D/22 xếp nó là một trong tám khoảng trống lớn nhất.

CẦN NHỚ

Một hệ relocalization dùng được trong sản phẩm phải trả lời được ba câu, không phải một.

Câu một: *tôi ở đâu?* — đây là câu mà mọi bài báo trả lời.

Câu hai: *tôi có chắc không?* — một tư thế kèm hiệp phương sai đáng tin, hoặc ít nhất một mức tin cậy đã được hiệu chuẩn trên dữ liệu thật. Rất ít công trình trả lời câu này.

Câu ba: *tôi có nên nói gì không?* — tức khả năng trả về “không biết”. Đây là câu quan trọng nhất cho sản phẩm và là câu ít được nghiên cứu nhất.

Ba câu ấy tương ứng với ba chỉ tiêu đo khác nhau, và một hệ tối ưu cho câu một có thể rất tệ ở câu ba. Khi đánh giá, hãy đo cả ba — và với sản phẩm, hãy đặt chỉ tiêu cho câu ba trước, rồi tối ưu câu một trong ràng buộc đó.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [16]	Reloc bằng DBoW2 + PnP RANSAC + tối ưu tư thế; yêu cầu nhất quán qua nhiều khung	Cài đặt của mục 2 và phép kiểm bốn ở mục 4B
HLoc [18]	NetVLAD/SuperPoint/SuperGlue + PnP; pipeline chuẩn công nghiệp	Kiến trúc phân cấp đầy đủ; đọc để thấy mỗi bước tách bạch thể nào

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ACE [48]	Scene coordinate regression với huấn luyện nhanh cho cảnh mới	Mục 3A; chú ý rằng buộc “một mô hình một cảnh”
[50]	Phân tích cho thấy APR gần với truy hồi ảnh hơn là với ước lượng hình học	Mục 3B — bài học phương pháp quan trọng hơn kết quả cụ thể
RTAB-Map	Reloc và quản lý bộ nhớ cho bản đồ dài hạn	Một trong ít hệ mã nguồn mở nghĩ về hoạt động nhiều phiên

MINI PROJECT SLAM · Đo tỉ lệ dương tính giả của relocalization, rồi hạ nó xuống bằng bốn phép kiểm · 3 ngày

Mục tiêu	chuyển từ “hệ của tôi relocalize được” sang “hệ của tôi trả về tư thế sai $X\%$ số lần ở ngưỡng Y , và nói không biết $Z\%$ ”.
Dữ liệu	một chuỗi có chân trị để dựng bản đồ, và một chuỗi thứ hai cùng môi trường để truy vấn. Quan trọng: thêm <i>các truy vấn không có câu trả lời</i> — ảnh từ một phòng khác không có trong bản đồ. Phần này là phần bị bỏ qua trong mọi benchmark và là phần quyết định trong sản phẩm.
Các bước	cài pipeline ba bước ở mục 2A; chạy trên cả truy vấn có và không có câu trả lời; với ngưỡng inlier quét qua một dải, vẽ ba đường cong theo ngưỡng — tỉ lệ đúng, tỉ lệ sai, tỉ lệ “không biết”; rồi thêm từng phép kiểm ở mục 4B và vẽ lại; cuối cùng đo độ trễ thêm vào bởi phép kiểm bốn.
Xong khi	bạn có một bảng: mỗi phép kiểm giảm tỉ lệ dương tính giả bao nhiêu, tăng tỉ lệ “không biết” bao nhiêu, và tốn thêm bao nhiêu mili giây. Bạn chọn được một điểm làm việc <i>từ yêu cầu sản phẩm</i> — “tôi chấp nhận sai không quá $0,1\%$ ” — chứ không từ một ngưỡng chép ở đâu. Và bạn có con số cho các truy vấn không có câu trả lời, thứ mà không benchmark nào cho bạn.
Nối với	C/13-quan-ly-ban-do xử lý vấn đề bản đồ lỗi thời ở mục 4C; D/21-tu-prototype-len-mass-production biến điểm làm việc này thành một chỉ tiêu nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Từ an toàn tới nguy hiểm: (b) báo “không biết” — hệ ở trạng thái đã biết là xấu, mọi module phía trên phản ứng được; (a) báo tư thế cũ — sai dần nhưng sai *dự đoán được*, và nếu ghi kèm độ tuổi thì phía trên chiết khấu được; (c) reloc sai — nguy hiểm nhất vì hệ báo một tư thế *mới* với độ tin cậy cao và hoàn toàn sai. Tệ hơn cả, nó sẽ bắt đầu tracking từ đó, khớp landmark vùng sai, và *củng cố* niềm tin sai vì các quan sát mới khớp nhất quán với giả thuyết sai trong một thời gian. Nếu còn ghi vào bản đồ thì bản đồ cũng hỏng.
- A2. Lý do *chi phí*: bước truy hồi là $O(1)$ nhờ chỉ mục, bước khớp là $O(k)$ với k nhỏ và không phụ thuộc kích thước bản đồ — nên tổng chi phí gần không đổi khi bản đồ lớn lên. Lý do *chất lượng*, tinh tế hơn và quan trọng hơn: khi khớp với toàn bộ bản đồ, số ứng viên sai tăng tuyến tính theo kích thước, nên xác suất một khớp sai lọt qua ratio test cũng tăng — với mười triệu landmark, ratio test không còn đủ. Giới hạn tìm kiếm vào vài chục khung giữ không gian nhỏ và khôi phục điều kiện hoạt động của bước khớp. Đây là cùng nguyên tắc với tiên nghiệm chuyển động ở B/07 mục 5C.
- A3. Nó học một ánh xạ *từ vẻ ngoài sang tư thế* trên tập huấn luyện, rồi *nội suy* giữa chúng — chứ không học hình học chiều. Sattler và cộng sự [50] cho thấy nó không vượt hơn đáng kể một đường cơ sở rất đơn giản là truy hồi ảnh huấn luyện giống nhất rồi trả về tư thế của ảnh đó. Hệ quả: nó không tổng quát hoá ra các tư thế xa tập huấn luyện — đúng chỗ mà bộ ước lượng hình học mạnh nhất. Bài học phương pháp rộng hơn: điểm số tốt trên benchmark không chứng minh phương pháp làm điều ta nghĩ nó làm; cách kiểm là so với một đường cơ sở tầm thường có cùng khả năng nội suy nhưng không có gì khác.
- A4. Từ dễ tới khó: (a) *thay đổi ánh sáng* (rèm kéo) — descriptor cổ điển kém bền, place recognition học được bền hơn nhiều; có lời giải. (b) *vật thể dịch chuyển* (cái ghế) — landmark trên nó thành outlier, RANSAC xử lý được khi chúng là thiểu số; vấn đề thật là *tích lũy* qua nhiều tháng cho

tới khi thiếu số thành đa số, và cần cơ chế cập nhật bản đồ. (c) *thay đổi cấu trúc* — không phép kiểm nào phân biệt được “tôi ở chỗ khác” với “tôi ở đúng chỗ nhưng chỗ này đã đổi”, và hai kết luận đòi hai hành động ngược nhau. Đây là bài toán lifelong mapping, một trong tám khoảng trống ở D/22.

- A5. Bốn phép: (1) *số và tỉ lệ inlier* — cần nhưng không đủ, ngưỡng cao hơn tracking nhiều. (2) *phân bố không gian của inlier* — hai trăm inlier tụ một góc cho tư thế bất định theo vài hướng dù số đếm tốt; kiểm bằng số ô lưới có inlier. (3) *hiệp phương sai từ Λ của bài toán PnP* — đúng về nguyên tắc vì nó đo trực tiếp mức xác định, nhưng chịu bệnh quá tự tin nên ngưỡng phải bảo thủ. (4) *nhất quán thời gian* — mạnh nhất và rẻ bất ngờ: đừng chấp nhận từ một khung duy nhất, yêu cầu vài khung liên tiếp cho tư thế nhất quán với nhau và với chuyển động đo được giữa chúng. Cái giá của (4) là vài trăm mili giây độ trễ.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn tỉnh dậy giữa một thành phố từng đi qua, không nhớ mình ở đâu, và có một cuốn sổ ảnh các góc phố từng đi.

Cách làm hợp lý có hai bước. Đầu tiên nhìn tổng thể — kiến trúc, cây cối, màu tường — rồi lật nhanh tới vài trang có ấn tượng chung giống nhất. Rồi với vài trang đó, so kỹ từng chi tiết.

Hai bước, hai loại thông tin. Ấn tượng chung thì thô nhưng so được với mười nghìn trang trong vài giây. Chi tiết thì chính xác nhưng chỉ so được với vài trang. Ghép hai lại thì vừa nhanh vừa ít nhầm — và cái “ít nhầm” quan trọng hơn cái “nhanh”, vì so chi tiết với ba trang thay vì mười nghìn thì cơ hội nhầm giảm hàng nghìn lần.

Nhưng chuyện quan trọng nhất của chương không phải cách tìm. Đó là chuyện gì xảy ra khi bạn *không chắc*.

Giả sử có hai góc phố trông giống nhau và bạn không phân biệt được. Hai lựa chọn: đoán một cái rồi đi tiếp, hoặc nói “tôi chưa biết” rồi đứng nhìn thêm một lúc.

Đoán rồi đi tiếp tệ hơn nhiều, và không phải vì lý do hiển nhiên. Một khi đã tin mình ở góc phố A, bạn sẽ diễn giải *mọi thứ* thấy sau đó theo giả định ấy. Cái cửa hàng kia hơi khác? Chắc họ đổi biển. Con đường không rõ như mong đợi? Chắc mình nhớ nhầm. Bạn càng đi càng tin, và càng khó quay lại. Trong khi nếu bạn thừa nhận chưa biết, ít nhất bạn vẫn đang tìm.

Cái máy cũng hệt vậy, và tệ hơn: nó không có cảm giác gợn khi mọi thứ hơi không khớp. Nó chỉ thấy con số phù hợp đủ tốt và đi tiếp.

Nên câu hỏi thiết kế đúng không phải “làm sao để máy luôn tìm được chỗ đúng” mà là “làm sao để máy biết nói tôi chưa biết”. Và cách làm, hoá ra, đơn giản đến bất ngờ: *đừng quyết định từ một cái nhìn duy nhất*. Nhìn thêm vài giây, đi thêm vài bước, và kiểm xem những gì thấy được có khớp với cả việc mình vừa di chuyển thế nào không. Một sự nhầm lẫn hiếm khi sống sót qua phép kiểm đó.

Chỗ không rút gọn được: chỗ không rút gọn được là khi *chỗ cũ đã thay đổi*. Bạn về đúng nhà mình nhưng ai đó đã kê lại toàn bộ đồ đạc. Máy không có cách nào phân biệt “tôi đang ở nhầm nhà” với “tôi ở đúng nhà nhưng nhà đã đổi” — và hai kết luận đòi hai hành động trái ngược. Đó là bài toán khó nhất trong triển khai dài hạn, và nó chưa có lời giải tốt.

Một câu: Một hệ đang lạc thì biết mình lạc; một hệ tin sai thì không — và đó là toàn bộ lý do phải biết nói “chưa biết”.

I 6.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *reloc sai hại hơn không reloc* — là một lập luận cấu trúc dựa trên việc hệ thống sẽ củng cố niềm tin sai. Nó bị bác bỏ nếu một hệ có cơ chế *phát hiện và huỷ* một relocalization sai sau khi đã chấp nhận, đủ tin cậy để chi phí của (c) không còn vượt (b). Cơ chế như vậy có thể tồn tại — theo dõi tỉ lệ tracking và phần dư trong vài giây sau reloc, và huỷ nếu chúng xấu đi — và nó nên được cài. Tôi ngờ rằng nó không bắt được trường hợp khó nhất, là reloc vào một vùng *thật sự giống*, vì ở đó tracking sẽ chạy tốt một thời gian. Nhưng tôi chưa đo (*tôi suy ra, chưa đổi chiều nguồn*).

Mệnh đề thứ hai — *kiến trúc phân cấp thắng vì cả chi phí lẫn chất lượng* — kiểm được bằng cách so pipeline ba bước với khớp trực tiếp toàn bản đồ, ở nhiều kích thước bản đồ. Nó bị bác bỏ nếu chất lượng không khác nhau ở bản đồ nhỏ — điều tôi *dự đoán* là đúng, vì lập luận về chất lượng chỉ có hiệu lực khi bản đồ đủ lớn. Kiểm điều đó sẽ làm rõ ngưỡng kích thước mà kiến trúc phân cấp bắt đầu đáng giá.

Mệnh đề thứ ba — *APR hành xử như nội suy chứ không như ước lượng hình học* — là kết quả chính của [50] và là một mệnh đề đã được kiểm bởi người khác (cửa c). Nó có thể đã thay đổi với các kiến trúc mới hơn, và tôi chưa khảo sát tài liệu sau 2019 về điểm này. Đây là một mục trong 99-chua-biet.md.

Một mệnh đề mềm: chương này khẳng định *benchmark relocalization* đo sai thứ cho ứng dụng robot. Đó là một phán đoán về mức phù hợp của phép đo, không phải một sự kiện. Nó bị bác bỏ nếu ai đó cho thấy thứ hạng trên benchmark tương quan chặt với tỉ lệ dương tính giả trong triển khai thật. Tôi cho là không, vì benchmark không chứa truy vấn ngoài bản đồ, nhưng tôi không có dữ liệu.

6.6 Kết nối

Nổi lên trên. [B/11-loop-closure-va-nhat-quan-toan-cuc](#) chia sẻ gần như toàn bộ công nghệ và toàn bộ cấu trúc lập luận; khác biệt duy nhất và quyết định là ở đây không có ước lượng hiện tại để kiểm chéo, nên mất một tầng phòng thủ. [B/07-dac-trung-va-data-association](#) cho bước khớp và cho nguyên tắc thu hẹp không gian tìm kiếm. [A/01-hinh-hoc-da-thi](#) cho PnP. [A/06-bat-dinh-va-lan-truyen-sai-so](#) cho phép kiểm thứ ba và cảnh báo về giới hạn của nó.

Nổi xuống dưới. [C/13-quan-ly-ban-do](#) là chương xử lý ba vấn đề mà chương này nêu ra nhưng không giải: bản đồ lỗi thời theo thời gian, kích thước bản đồ tăng vô hạn, và việc ghép nhiều phiên. [C/18-danh-gia-va-phuong-phap-luan](#) chuẩn hoá cách đo ba câu hỏi ở khối *Cần nhớ*, đặc biệt câu ba vốn không có benchmark nào đo.

Nổi xa hơn. [D/19-ben-vung-trong-the-gioi-thuc](#) bàn về ba chế độ hồng ở mục 4C một cách đầy đủ hơn. [D/21-tu-prototype-len-mass-production](#) là chỗ chương này quan trọng nhất: khả năng tự phục hồi quyết định MTBF cảm nhận được của sản phẩm, và thời gian cold start là thứ khách hàng đo bằng đồng hồ. [D/22-huong-nghien-cuu-con-trong](#) lấy lifelong mapping — bài toán ở mục 4C — làm một trong tám khoảng trống.

Nổi ngang: `../marker/` cung cấp giải pháp thực dụng nhất cho cold start trong môi trường có thể sửa đổi: một marker trong tầm nhìn cho tư thế tuyệt đối ngay lập tức, với độ tin cậy cao hơn nhiều so với bất kỳ phương pháp thuần thị giác nào. Cái giá là phải đặt và bảo trì marker.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Xếp ba lựa chọn khi mất bám — báo tư thế cũ, báo không biết, relocalize — theo mức nguy hiểm, và giải thích cơ chế làm lựa chọn nguy hiểm nhất trở nên nguy hiểm.
2. Nêu hai lý do độc lập vì sao kiến trúc phân cấp thắng, và nói lý do nào chỉ có hiệu lực khi bản đồ đủ lớn.
3. PoseNet thực sự học được gì? Nêu đường cơ sở tầm thường dùng để phát hiện điều đó, và bài học phương pháp rộng hơn.
4. Nêu bốn phép kiểm trước khi chấp nhận một relocalization, và nói phép nào mạnh nhất và vì sao nó rẻ bất ngờ.
5. Ba chế độ hồng khi bản đồ cũ đi. Vì sao chế độ thứ ba không có lời giải tốt, và hai kết luận nào bị lẫn lộn?

6. Ba câu hỏi mà một hệ relocalization sản phẩm phải trả lời. Câu nào ít được nghiên cứu nhất, và vì sao nó quan trọng nhất?
7. Vì sao benchmark relocalization đo sai thứ cho ứng dụng robot? Nêu ba thứ chúng không đo.

Nguồn

- [1] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, mức tin A.
- [2] J. Shi and C. Tomasi, “Good Features to Track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 1994.
- [3] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2006.
- [4] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2011.
- [5] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [6] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [7] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [8] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [9] J.-Y. Bouguet, “Pyramidal Implementation of the Affine Lucas Kanade Feature Tracker: Description of the Algorithm,” Intel Corporation, tech. rep., 2001, mức tin B; báo cáo kỹ thuật, là tài liệu tham chiếu de-facto cho KLT thấp.
- [10] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [11] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; nguồn của inverse compositional.
- [12] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [13] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [14] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2005.
- [15] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2020.
- [16] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.

- [17] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [18] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [19] C. Forster, Z. Zhang, M. Gassner, et al., "SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.
- [20] G. Klein and D. Murray, "Parallel Tracking and Mapping for Small AR Workspaces," in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A; công trình tách tracking khỏi mapping thành hai luồng, 2007.
- [21] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," in *European Conference on Computer Vision (ECCV)*, mức tin A, 2014.
- [22] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [23] J. Civera, A. J. Davison, and J. M. M. Montiel, "Inverse Depth Parametrization for Monocular SLAM," *IEEE Transactions on Robotics*, vol. 24, no. 5, pp. 932–945, 2008, mức tin A; nguồn của lập luận vì sao inverse depth tuyến tính hoá tốt hơn.
- [24] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của cách khử landmark bằng chiếu lên null space, 2007.
- [25] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2024.
- [26] L. Piccinelli, Y.-H. Yang, C. Sakaridis, et al., "UniDepth: Universal Monocular Metric Depth Estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [27] A. Rosinol, M. Abate, Y. Chang, et al., "Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [28] M. Kaess, H. Johannsson, R. Roberts, et al., "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [29] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên $Sim(3)$, 2010.
- [30] V. Usenko, N. Demmel, D. Schubert, et al., "Visual-Inertial Mapping with Non-Linear Factor Recovery," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [31] N. Demmel, C. Sommer, D. Cremers, et al., "Square Root Bundle Adjustment for Large-Scale Reconstruction," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [32] D. M. Rosen, L. Carlone, A. S. Bandeira, et al., "SE-Sync: A Certifiably Correct Algorithm for Synchronization over the Special Euclidean Group," *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019, mức tin A.
- [33] R. Hartley, J. Trunpf, Y. Dai, et al., "Rotation Averaging," *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.

- [34] P. Geneva, K. Eickenhoff, W. Lee, et al., "OpenVINS: A Research Platform for Visual-Inertial Estimation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [35] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [36] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.
- [37] D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012, mức tin A.
- [38] D. Nistér and H. Stewénus, "Scalable Recognition with a Vocabulary Tree," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2006.
- [39] R. Arandjelović, P. Gronat, A. Torii, et al., "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018, mức tin A.
- [40] M. J. Milford and G. F. Wyeth, "SeqSLAM: Visual Route-Based Navigation for Sunny Summer Days and Stormy Winter Nights," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [41] Y. Latif, C. Cadena, and J. Neira, "Robust Loop Closing over Time for Pose Graph SLAM," *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013, mức tin A; thuật toán RRR.
- [42] N. Sünderhauf and P. Protzel, "Switchable Constraints for Robust Pose Graph SLAM," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2012.
- [43] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., "Robust Map Optimization Using Dynamic Covariance Scaling," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2013.
- [44] H. Yang, P. Antonante, V. Tzoumas, et al., "Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [45] M. J. Black and A. Rangarajan, "On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision," *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [46] T. Schneider, M. Dymczyk, M. Fehr, et al., "maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [47] E. Brachmann and C. Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022, mức tin B.
- [48] E. Brachmann, T. Cavallari, and V. A. Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2023.
- [49] A. Kendall, M. Grimes, and R. Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2015.
- [50] T. Sattler, Q. Zhou, M. Pollefeys, et al., "Understanding the Limitations of CNN-Based Absolute Camera Pose Regression," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của khẳng định APR không tốt hơn image retrieval, 2019.

Nguồn

- [1] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004, mức tin A.
- [2] J. Shi and C. Tomasi, “Good Features to Track,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 1994.
- [3] E. Rosten and T. Drummond, “Machine Learning for High-Speed Corner Detection,” in *European Conference on Computer Vision (ECCV)*, mức tin A, 2006.
- [4] E. Rublee, V. Rabaud, K. Konolige, et al., “ORB: An Efficient Alternative to SIFT or SURF,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2011.
- [5] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [6] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [7] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [8] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [9] J.-Y. Bouguet, “Pyramidal Implementation of the Affine Lucas Kanade Feature Tracker: Description of the Algorithm,” Intel Corporation, tech. rep., 2001, mức tin B; báo cáo kỹ thuật, là tài liệu tham chiếu de-facto cho KLT thấp.
- [10] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [11] S. Baker and I. Matthews, “Lucas-Kanade 20 Years On: A Unifying Framework,” *International Journal of Computer Vision*, vol. 56, no. 3, pp. 221–255, 2004, mức tin A; nguồn của inverse compositional.
- [12] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [13] M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981, mức tin A.
- [14] O. Chum and J. Matas, “Matching with PROSAC — Progressive Sample Consensus,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2005.
- [15] D. Barath, J. Noskova, M. Ivashechkin, et al., “MAGSAC++, a Fast, Reliable and Accurate Robust Estimator,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2020.
- [16] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.

- [17] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [18] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [19] C. Forster, Z. Zhang, M. Gassner, et al., "SVO: Semidirect Visual Odometry for Monocular and Multicamera Systems," *IEEE Transactions on Robotics*, vol. 33, no. 2, pp. 249–265, 2017, mức tin A.
- [20] G. Klein and D. Murray, "Parallel Tracking and Mapping for Small AR Workspaces," in *IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A; công trình tách tracking khỏi mapping thành hai luồng, 2007.
- [21] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," in *European Conference on Computer Vision (ECCV)*, mức tin A, 2014.
- [22] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, 2018, mức tin A.
- [23] J. Civera, A. J. Davison, and J. M. M. Montiel, "Inverse Depth Parametrization for Monocular SLAM," *IEEE Transactions on Robotics*, vol. 24, no. 5, pp. 932–945, 2008, mức tin A; nguồn của lập luận vì sao inverse depth tuyến tính hoá tốt hơn.
- [24] A. I. Mourikis and S. I. Roumeliotis, "A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; nguồn của cách khử landmark bằng chiếu lên null space, 2007.
- [25] L. Yang, B. Kang, Z. Huang, et al., "Depth Anything V2," in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2024.
- [26] L. Piccinelli, Y.-H. Yang, C. Sakaridis, et al., "UniDepth: Universal Monocular Metric Depth Estimation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [27] A. Rosinol, M. Abate, Y. Chang, et al., "Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping," in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [28] M. Kaess, H. Johannsson, R. Roberts, et al., "iSAM2: Incremental Smoothing and Mapping Using the Bayes Tree," *The International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, 2012, mức tin A.
- [29] H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," in *Robotics: Science and Systems (RSS)*, mức tin A; nguồn của loop closure trên Sim(3), 2010.
- [30] V. Usenko, N. Demmel, D. Schubert, et al., "Visual-Inertial Mapping with Non-Linear Factor Recovery," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [31] N. Demmel, C. Sommer, D. Cremers, et al., "Square Root Bundle Adjustment for Large-Scale Reconstruction," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [32] D. M. Rosen, L. Carlone, A. S. Bandeira, et al., "SE-Sync: A Certifiably Correct Algorithm for Synchronization over the Special Euclidean Group," *The International Journal of Robotics Research*, vol. 38, no. 2-3, pp. 95–125, 2019, mức tin A.
- [33] R. Hartley, J. Trumpf, Y. Dai, et al., "Rotation Averaging," *International Journal of Computer Vision*, vol. 103, no. 3, pp. 267–305, 2013, mức tin A.

- [34] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [35] T. D. Barfoot, *State Estimation for Robotics*. Cambridge University Press, 2017, mức tin A; nguồn chuẩn cho ước lượng trên đa tạp.
- [36] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005, mức tin A.
- [37] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012, mức tin A.
- [38] D. Nistér and H. Stewénus, “Scalable Recognition with a Vocabulary Tree,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2006.
- [39] R. Arandjelović, P. Gronat, A. Torii, et al., “NetVLAD: CNN Architecture for Weakly Supervised Place Recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 6, pp. 1437–1451, 2018, mức tin A.
- [40] M. J. Milford and G. F. Wyeth, “SeqSLAM: Visual Route-Based Navigation for Sunny Summer Days and Stormy Winter Nights,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2012.
- [41] Y. Latif, C. Cadena, and J. Neira, “Robust Loop Closing over Time for Pose Graph SLAM,” *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1611–1626, 2013, mức tin A; thuật toán RRR.
- [42] N. Sünderhauf and P. Protzel, “Switchable Constraints for Robust Pose Graph SLAM,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A, 2012.
- [43] P. Agarwal, G. D. Tipaldi, L. Spinello, et al., “Robust Map Optimization Using Dynamic Covariance Scaling,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2013.
- [44] H. Yang, P. Antonante, V. Tzoumas, et al., “Graduated Non-Convexity for Robust Spatial Perception: From Non-Minimal Solvers to Global Outlier Rejection,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1127–1134, 2020, mức tin A.
- [45] M. J. Black and A. Rangarajan, “On the Unification of Line Processes, Outlier Rejection, and Robust Statistics with Applications in Early Vision,” *International Journal of Computer Vision*, vol. 19, no. 1, pp. 57–91, 1996, mức tin A; nguồn của đối ngẫu Black-Rangarajan.
- [46] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [47] E. Brachmann and C. Rother, “Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022, mức tin B.
- [48] E. Brachmann, T. Cavallari, and V. A. Prisacariu, “Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2023.
- [49] A. Kendall, M. Grimes, and R. Cipolla, “PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2015.
- [50] T. Sattler, Q. Zhou, M. Pollefeys, et al., “Understanding the Limitations of CNN-Based Absolute Camera Pose Regression,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của khẳng định APR không tốt hơn image retrieval, 2019.